

근위 정책 최적화 기반 적응형 동적 가중치 학습을 통한 Triple-Hybrid RAG 프레임워크의 성능 최적화 연구

Performance Optimization of Triple-Hybrid RAG Framework via Proximal Policy Optimization-based Learned Dynamic Weighting

호서대학교 대학원 융합공학과

지도교수: 문남미

학위논문 제출자: 신동욱

제출일: 2026 년 4 월

학위: 박사

I. 서론

1. 연구 배경

검색 증강 생성(Retrieval-Augmented Generation, 이하 RAG)은 대형 언어 모델(LLM)의 환각과 지식 단절 문제를 외부 지식 주입으로 완화하는 핵심 기법으로, 2020 년 이래 산업·학계 양면에서 급속히 확산하였다. 단일 소스 RAG 의 한계가 실증되면서 복수의 지식 표현(텍스트 벡터, 지식 그래프, 형식 온톨로지)을 결합하는 하이브리드 RAG 가 대두되었으며, 본 연구의 저자는 선행 논문(Shin & Moon, 2025, JKSCI)에서 Triple-Hybrid RAG 프레임워크와 Rule-based Dynamic Weighting Algorithm (R-DWA) 을 제안한 바 있다.

선행 연구의 R-DWA 는 질의 유형 분류와 개체/관계/제약 밀도 신호를 2 단계로 결합하여 세 소스의 가중치 (α , β , γ)를 동적으로 결정한다. 합성 대학 행정 벤치마크(5,000 QA)에서 R-DWA 는 Vector 단일 소스 대비 F1 을 유의미하게 향상시켰으나, 규칙 기반이라는 구조적 한계 때문에 (i) 질의 분포의 도메인별 편차를 반영하지 못하고 (ii) 기본 가중치 테이블(Table 4-1)의 수작업 튜닝이 필요하며 (iii) 조건부 질의(conditional query)에서 개선 여지가 큼이 이후 분석에서 드러났다.

본 박사 논문은 이 구조적 한계를 학습형 동적 가중치 알고리즘(Learned DWA, 이하 L-DWA) 로 해소한다. 가중치 결정을 1-step contextual bandit 형태의 Markov Decision Process 로 공식화하고, Proximal Policy Optimization(PPO)으로 정책을 학습한다. 실험의 비용 장벽은 오프라인 보상 캐시로 해결하여, 학습 중 LLM API 호출이 0 이 되는 구조를 제시한다.

▶ [박스 1-1] Triple-Hybrid RAG — 한눈에 보기

>

▶ 정의. 하나의 자연어 질의에 대해 세 가지 지식 소스 를 병렬 조회하고 각 소스의 기여도를 가중치로 융합하여 최종 컨텍스트를 구성하는 RAG 아키텍처.

>

▶ 세 소스.

>

▶ | 소스 | 표현 | 가중치 | 장점 |

▶ | ---| ---| ---| ---|

▶ | Vector | 문서 임베딩 (FAISS) | α | 의미 유사도 |

▶ | Graph | 엔티티-관계 그래프 (NetworkX, BFS) | β | 다단계 관계 |

▶ | Ontology | OWL 클래스·제약 (Owlready2) | γ | 논리 제약 |

▶ | 제약 | $\alpha + \beta + \gamma = 1$, 모두 ≥ 0 (단순체 Δ^3) | | |

>

- ▶ 예시. "기계공학과 55 세 이하 교수는?" 질의에 대해:
- ▶ - $(\alpha, \beta, \gamma) = (0.33, 0.33, 0.33)$ 균등 $\rightarrow$ 평균 성능
- ▶ - $(\alpha, \beta, \gamma) = (0.1, 0.2, 0.7)$ Ontology-heavy $\rightarrow$ 55 세 제약 적용 $\uparrow$

>

- ▶ 본 논문의 핵심 질문. 이 (α, β, γ) 를 질의마다 어떻게 최적 결정하는가? — 선행의 R-DWA(규칙) 대비 학습된 L-DWA(PPO) 로 개선 가능한가?

2. 연구 문제

본 논문은 다음 세 가지 연구 질문(RQ)에 답한다.

RQ1. 규칙 기반 R-DWA 의 고정된 기본 가중치 테이블이 실제 벤치마크의 보상

지형(reward landscape)과 정렬되어 있는가? 정렬되지 않았다면 그 간극은 정량적으로 얼마인가?

RQ2. Triple-Hybrid 가중치 결정을 MDP 로 공식화하고 PPO 로 학습한 L-DWA 는 R-DWA 대비 어떤 정량적 개선을 제공하는가? 개선은 질의 유형별로 어떻게 분포하는가?

RQ3. 특정 도메인(합성 대학)에서 학습된 L-DWA 의 정책은 타 도메인(HotpotQA, MuSiQue, PubMedQA)으로 이전 가능한가? 이전 가능성/불가능성은 본 프레임워크의 가치 제안에 어떤 시사점을 주는가?

3. 연구 기여

이 연구가 기여하는 지점은 크게 네 갈래로 나눌 수 있다. 방법론, 실증, 엔지니어링, 그리고 평가 인프라 정비이다. 각 갈래를 차례로 서술한다.

가. 방법론적 기여

첫 번째 방법론적 기여는 Triple-Hybrid RAG 의 가중치 결정 문제를 Markov Decision Process 로 공식화한 점이다. 상태 공간은 질의에서 추출한 밀도 신호와 소스별 검색 통계를 담은 18 차원 벡터이고, 행동 공간은 3-simplex 위의 연속 가중치이다.

에피소드는 (s, a, R) 한 쌍으로 종료되는 1-step 구조이지만, GAE 와 PPO clipping 의 구현을 유지함으로써 이후 multi-turn 확장의 여지를 열어 두었다 (Ch.5 §1).

두 번째는 이 MDP 위에서 PPO 를 이용해 L-DWA 라는 학습형 정책을 제안한 점이다.

공유 백본 (18→64→64) 위에 Actor head 와 Critic head 를 얹은 5,636 파라미터의 경량 Actor-Critic 을 사용하며, clip ratio 0.2 와 10,000 에피소드의 표준적인 설정으로 학습한다. 서로 다른 세 시드 (42, 123, 999) 가 모두 Total Reward 0.215 ± 0.002 로 수렴한다 (Ch.5 §3, Ch.6 §5).

나. 실증적 기여

합성 대학 5,000 QA 벤치마크에서 측정한 핵심 수치는 다음과 같다. Corrected baseline (§3.4 참조) 조건에서 L-DWA 의 3-seed 평균 $F1_{\text{strict}}$ 는 0.562 ± 0.007 로, R-DWA 의 0.529 대비 6.2% 개선되었다. 더 흥미로운 점은 L-DWA 가 66-점 이산 격자에서의 per-query argmax Oracle (0.554) 을 $F1_{\text{strict}}$, $F1_{\text{substring}}$, $F1_{\text{char}}$, Faithfulness 모든 축에서 조금씩 넘어선다는 사실이다. 연속 Dirichlet 평균이 이산 격자 바깥의 공간을 활용하고 있다는 해석이 자연스럽다 (Ch.6 §3).

질의 유형별로 쪼개어 보면 조건부 질의(conditional query) 에서 개선 폭이 가장 크다.

R-DWA 0.223 대비 L-DWA 0.304 로 36.7% 향상되며, 이 영역에서는 Oracle 마저

넘어선다 (Ch.6 §4). 이는 per-query 학습형 가중치가 고정 규칙보다 조건 처리에 더 유연함을 보여준다.

교차 도메인에서는 반대 방향의 결과도 보고한다. 대학 도메인으로 학습한 L-DWA 를 HotpotQA/MuSiQue/PubMedQA 에 그대로 적용하면 Vector-only 기준선보다 30~35% 낮게 나온다. 이 실패의 원인은 언어 장벽, 도메인 어휘, Graph/Ontology 부재의 세 갈래로 분리되는데, 각각을 독립 실험으로 확인한 결과 언어와 도메인 어휘는 적절한 확장으로 해소 가능하지만 Graph/Ontology 부재는 Triple-Hybrid 프레임워크의 본질적 경계 조건이라는 결론에 도달하였다 (Ch.6 §7).

다. 엔지니어링 기여

L-DWA 학습의 비용 장벽을 낮추기 위해 오프라인 보상 캐시를 도입하였다. 5,000 질의 $\times$ 66 개 이산 가중치 = 330,000 엔트리를 사전 계산하여 SQLite 에 저장하는 방식으로, 최초 구축에 14 시간 / \$33 이 들어가지만 이후의 PPO 학습은 LLM 호출 없이 캐시 조회만으로 진행된다. 3-seed 학습 전체 기준으로 환산하면 예상 비용 \$288 대비 85% 가 절감되었다 (Ch.5 §4).

라. 평가 인프라 정비

연구를 진행하면서 선행 JKSCI 2025 논문의 수치가 현재 코드로는 그대로 재현되지 않는다는 사실을 확인하게 되었다. 원인을 추적해 보니 두 가지였다. 하나는 normalize_korean 에서 쉼표·마침표가 토큰화 이전에 제거되지 않던 작은 버그로, 콤마 구분 리스트 gold 의 토큰 분할이 잘못되어 F1 이 실제의 약 1/3 수준으로 산출되고 있었다. 다른 하나는 선행 저장소의 노트북에서 5,000 QA 전수 실행 셀이 주석 처리된 채

미실행 상태로 커밋되어 있었다는 점이다. 이 두 사항은 선행 저장소의 CORRIGENDUM에 공개 정정하였다 (Ch.6 §1.4, §3.3).

이 과정에서 추가로 도입한 평가 장치는 세 가지이다. 첫째, LLM 출력을 gold의 리스트 형식과 맞추는 PROMPT_TEMPLATE_LIST를 두어, 같은 평가기 아래에서도

F1_{strict}가 0.137에서 0.529로 올라가는 현상을 확인하였다. 둘째,

리스트형 응답에 대해 Faithfulness를 항목별 claim 단위로 검증하는 분기를 추가하였다.

셋째, 토큰 집합 F1 (strict), 콤마 항목 부분문자열 F1 (substring), 문자 3-gram F1

(char)의 세 축을 병기함으로써 한국어 QA 평가의 형식 의존성을 드러내도록 하였다

(Ch.6 §1.3).

이들은 본래 연구의 부차적 과제였으나, 선행 연구와의 수치 비교가 신뢰 가능한 형태로

이루어지기 위한 전제 조건이기도 하다. 본 논문의 핵심 비교가 "평가 인프라를 바로잡은

조건에서의 L-DWA vs R-DWA" 형태로 수행되는 것은 이 때문이다.

4. 본 논문의 범위와 한계

본 논문은 다음을 다룬다:

- Triple-Hybrid RAG (Vector + Graph + Ontology)의 가중치 결정 학습
- PPO 기반 정책 학습의 수렴과 재현성
- 합성 대학 벤치마크의 5,000 QA 전수 평가
- HotpotQA/MuSiQue/PubMedQA 교차 도메인 이전 실험 (domain transfer feasibility)
- 엄격-loose 이중 평가 기준의 비교

본 논문은 다음을 다루지 않는다 (향후 연구 과제):

- 지식 그래프·온톨로지의 LLM 기반 자동 구축
- 다국어 intent 분석기 (현 구현은 한국어 regex 기반)

- 도메인 간 joint training 또는 meta-learning
- BERT multi-label intent classifier 의 학습 (§5.2 에서 구현만 제시하며 intent_logits=0 상태로 학습)
- HotpotQA 에서의 L-DWA domain-specific 학습 (reward landscape cliff 로 인한 학습 불가 현상을 Ch.6 §7.3 에 보고)

5. 논문 구성

- 제 I 장 (본 장) 연구 배경, 연구 문제, 기여, 범위
- 제 II 장 관련 연구 — RAG / Graph-RAG / Hybrid-RAG / PPO / 한국어 QA 평가
- 제 III 장 Triple-Hybrid RAG 아키텍처 — Vector·Graph·Ontology 세 소스의 정의와 통합 방식
- 제 IV 장 Rule-based DWA (R-DWA) — 선행 연구의 2 단계 규칙 및 한계 분석
- 제 V 장 Learned DWA (L-DWA) — MDP 공식화, Actor-Critic, PPO 학습, 오프라인 캐시
- 제 VI 장 실험 및 평가 — 이중 F1 측정, 3-seed 결과, 질의 유형별 분석, 교차 도메인 실험
- 제 VII 장 결론 — 기여 요약, 한계, 향후 연구 방향

6. 재현성 공개

본 논문의 모든 코드, 오프라인 캐시, PPO 체크포인트, 평가 결과를 GitHub 오픈소스로 배포한다:

- 저장소: <https://github.com/sdw1621/triple-rag-phd> (Private → 제출 후 Public)
- 실행 단위 스크립트: scripts/ 디렉토리 (build_cache, train_ppo, evaluate_rerun, cross_benchmark 등)
- 캐시: cache/university.sqlite (330K 엔트리, 16.8 MB)
- 체크포인트: cache/ppo_checkpoints/seed_{42,123,999}/final.pt
- 논문 소스 markdown: thesis_current/v5_rewrite/

모든 학습 및 평가는 단일 명령으로 재실행 가능하며, PyTorch deterministic 설정 및 3 개 고정 시드 (42, 123, 999) 로 결과의 완전 재현을 보장한다.

II. 관련 연구

본 장은 본 논문의 세 갈래 관련 연구 흐름을 정리한다: (1) RAG 진화와 하이브리드 접근, (2) 지식 그래프·온톨로지의 RAG 통합, (3) 검색/가중치 결정을 강화학습으로 최적화하는 흐름. 마지막 절에서 본 논문의 위치를 명시한다.

1. RAG 의 진화와 하이브리드 접근

가. 단일 소스 RAG 의 한계

Lewis et al. (2020)의 RAG 기본형은 Dense Vector Retrieval 과 생성 모델의 결합으로, LLM 환각 완화에 실효를 거두었다. 그러나 단일 벡터 소스는 (i) 관계형 질의 (multi-hop) 에서 얇은 유사도만 반영하고, (ii) 조건 제약 (e.g. "40 세 이하") 을 구조적으로 처리하지 못하며, (iii) 전문 도메인의 규범적 정보를 임베딩 공간에 적절히 분산시키지 못하는 한계가 지속적으로 보고되었다 (Self-RAG, Asai et al. 2023; CRAG, Yan et al. 2024).

나. GraphRAG 계열

Microsoft GraphRAG (Edge et al. 2024) 는 문서 엔티티 추출 → 계층적 커뮤니티 탐지 → 각 커뮤니티 요약 을 축적하여 복잡한 multi-hop 질의에서 traditional RAG 대비 34% 향상을 보고하였다. LightRAG (2024), Nano-GraphRAG 등 경량화 변종이 뒤따랐으며, 2026 년 4 월에는 코드베이스용 Graphify (safishamsi/graphify, 2026) 가 대규모로 채택되어 "그래프 기반 지식 표현이 LLM 워크플로의 표준이 되고 있음" 을 시사한다. 본

논문의 선행 연구 (Shin & Moon, 2025) 는 Graph 소스를 벡터·온톨로지와 통합한 Triple-Hybrid 구성을 제시한 바 있다.

다. Hybrid RAG

HybridRAG (2024) 는 vector retrieval 과 graph retrieval 을 고정 가중치 로 결합하여 금융 도메인에서 보고되었다. 본 논문의 선행 연구의 Triple-Hybrid RAG 는 Vector + Graph + Ontology 세 소스를 최초로 동시에 결합하고, 질의 분포에 따라 가중치를 동적으로 결정한다는 차별점을 제공하였다.

2. 지식 그래프 및 온톨로지의 RAG 통합

가. 지식 그래프 기반 RAG

NetworkX BFS 혹은 Neo4j Cypher 기반의 관계 탐색을 RAG 에 결합하는 흐름은 multi-hop QA 정확도 향상과 답변 근거의 해석 가능성을 제공한다. 본 논문은 NetworkX MultiDiGraph 기반 BFS(max_depth=3)를 Graph 소스로 채택하여 (Ch.3 §2.2), 577 명 교수 / 60 학과 / 400 프로젝트 규모의 합성 대학 그래프를 적용하였다.

나. 형식 온톨로지의 RAG 통합

OWL 2 표준과 HermiT/Pellet 추론기는 클래스 계층, 속성 제약, 수치 범위의 결정 가능한 추론을 제공한다. 본 논문은 Owlready2 기반 온톨로지를 Ontology 소스로 채택하며 (Ch.3 §2.3), 특히 조건부 질의 (conditional query) 의 수치 제약 ("40 세 이하") 을 제약 체크로 명시 처리한다.

3. 검색·가중치 결정의 강화학습 기반 최적화

가. Adaptive-RAG 와 정책 기반 검색

Jeong et al. (2024) Adaptive-RAG 는 질의 복잡도에 따라 단일 호출/반복 호출/no-retrieval 중 하나를 고르는 분류기를 학습한다. 그러나 서로 다른 지식 소스 간 연속 가중치 결정 은 대상이 아니다.

나. Instruct-RAG / Self-RAG / CRAG

Self-RAG (Asai et al. 2023) 와 CRAG (Yan et al. 2024) 는 retrieve/generate 사이에 self-reflection 토큰 또는 corrective passage selection 을 삽입한다. 이들은 문헌 선택이나 재작성의 quality control 에 초점을 맞추며, 소스 간 가중치 융합 은 연구 질문으로 제기되지 않았다.

다. PPO 기반 정책 학습

Schulman et al. (2017) 의 PPO 는 clip ratio ϵ 로 정책 업데이트 안정성을 보장하는 대표적 on-policy RL 알고리즘이다. 본 논문은 PPO 의 구현을 RAG 가중치 결정에 최초로 적용하여 5,636 파라미터의 경량 Actor-Critic 정책을 학습한다. 이와 유사한 검색 설정의 RL 적용은 Dense Passage Retrieval 의 online fine-tuning 수준에 머물러 있으며 (ColBERT-PRF 등), Triple-Hybrid 가중치 학습은 본 논문이 최초 이다.

4. 한국어 QA 평가 및 JKSCI 2025 기준선

가. 한국어 QA 의 평가 난점

한국어 gold 답은 콤마 구분 다인명 리스트 ("홍성민, 황성민, 전성민") 가 빈번하고, LLM 의 자연어 출력은 "홍성민 교수, 황성민 교수, 전성민 교수가 담당합니다" 형태의

문장이다. 토큰 집합 기반 F1 은 조사 제거·구두점 제거 없이는 gold/prediction 토큰이 접미사 차이로 매칭되지 않는다. 본 논문은 이 간극을 F1_strict (엄격 토큰 집합) 와 F1_substring (coma 분할 부분문자열) 의 이중 지표로 명시화한다 (Ch.6 §1.3).

나. JKSCI 2025 선행 연구의 수치

선행 논문은 R-DWA F1 0.86, EM 0.78, Faithfulness 0.89 를 보고하였다. 본 논문의 엄격 재평가는 두 단계로 이루어진다. (i) Sentence 프롬프트 (선행과 동일 출력 형식) 에서 R-DWA F1_{strict} 는 0.137, F1_{substring} 은 0.448 로 측정되며, 선행 수치의 Faithfulness 0.89 는 본 논문 측정 0.835 와 94% 일치 하여 LLM 생성 품질은 재현됨을 확인하였다. (ii) 출력 형식만 List 프롬프트 (coma 구분 nominal list) 로 정렬하면 평가 인프라 변경 없이 R-DWA F1_{strict} 가 0.529, EM 이 0.387 로 회복되어 선행 보고에 더 가까워진다. 이를 본 논문의 corrected baseline 으로 삼는다. F1_{strict} 의 격차는 형식 정렬 단계와 평가기 strict/loose 정의 차이에 기인하며 (Ch.6 §3.3 상세), 본 논문은 sentence/list 두 단계 병기로 선행과 호환·비교 가능성을 제공한다.

- ▶ [박스 2-1] R-DWA vs L-DWA — 3-문장 요약

>

- ▶ 정의. 같은 Triple-Hybrid 파이프라인의 가중치 결정 단계 에서 R-DWA 는 사람이 짠 if-else 규칙 + 기본 가중치 테이블을 적용하고, L-DWA 는 18-dim 상태 → Actor-Critic 신경망(≈6K 파라미터) → Dirichlet 가중치 샘플링으로 학습한 정책을 호출한다.

>

- ▶ 예시 (같은 질의, 같은 retrieval). "기계공학과 55 세 이하 교수는?"

>

- ▶ | 단계 | R-DWA | L-DWA |
- ▶ | ---| ---| ---|
- ▶ | 밀도 신호 | (2, 1, 1) | (2, 1, 1) + 15-dim extras |
- ▶ | 가중치 로직 | 규칙 테이블 + λ 보정 | $\pi_{\Theta}(s)$ Dirichlet mean |
- ▶ | 선택된 (α, β, γ) | (0.15, 0.25, 0.60) (고정 템플릿) | (0.12, 0.23, 0.65) (연속 조정) |
- ▶ | 본 벤치마크 $F1_{strict}$ | 0.529 | $0.562 \pm 0.007 (+6.2\%)$ |

>

- ▶ 해석. 규칙 기반(R)은 투명하지만 고정적, 학습 기반(L)은 불투명하지만 적응적. 본 논문은 이 두 극단 사이에서 학습형이 66-점 이산 격자 Oracle (0.554) 을 초과 함을 실증(§3.2). 연속 정책이 이산 argmax 상한 바깥의 공간을 활용한다는 해석이다.

5. 본 논문의 연구적 위치

본 논문은 다음 세 축에서 기존 문헌과 구별 된다.

축	기존 연구	본 논문
지식 소스 결합	Hybrid (2-way), GraphRAG (1+ 커뮤니티)	Triple-Hybrid (Vector+ Graph+ Ontology)
가중치 결정	고정 / 분류기 / 규칙	MDP + PPO 학습형 연속 가중치
평가 엄격성	단일 F1/EM	$F1_{strict}$ + $F1_{substring}$ 이중 평가

R-DWA (선행 JKSCI) 는 Triple-Hybrid 의 엔지니어링 조합을, 본 박사 논문은 Triple-Hybrid 의 학습 프레임워크 (L-DWA) 를 기여한다. 합성 대학 벤치마크 (corrected baseline) 에서 L-DWA 는 R-DWA 대비 $F1_{strict} + 6.2\%$ ($0.529 \rightarrow 0.562 \pm 0.007$), Conditional $F1_{substring} + 28.6\%$ ($0.148 \rightarrow 0.190$) 를 달성하며, 66-점 이산 격자 Oracle (per-query argmax) 상한을 초과 한다 (Ch.6 §3).

한편 본 논문의 domain transfer 실험은 현 L-DWA 구현이 한국어 intent 분석에 의존하여 영어 교차 도메인에 naive 전이 시 Vector-only 대비 저하함을 솔직히

보고한다. 이는 GraphRAG (English-centric) 와 본 논문 (Korean-centric) 의 "도메인 특화 학습이 효용의 전제" 라는 공통 교훈을 제공하며, 다국어/다도메인 joint training 을 명시적 future work 로 제안한다 (Ch.7 §2).

6. 강화학습을 활용한 RAG 최적화 연구

이 절에서는 최근 제안된 RL 기반 RAG 최적화 연구 네 편을 살펴보고, 본 논문과의 차이가 어디에서 오는지 짚어 둔다. 비교 대상은 RAG 파이프라인 일부에 학습된 정책이나 분류기를 도입한 대표적 연구로 한정하였으며, 재현 가능한 공개 구현이 있는 경우를 우선으로 하였다.

가. 네 편의 대표 연구

Self-RAG (Asai et al., 2024) 는 생성 중간에 retrieve/continue/support 같은 reflection token 을 삽입하도록 지도 학습을 수행한다. 즉 "언제 검색할지" 와 "검색 결과를 쓸지" 를 생성 모델이 직접 판단하게 만든 셈이다. CRAG (Yan et al., 2024) 는 이 문제를 분리하여, 검색 품질을 이진 분류 (correct/incorrect) 하는 별도 모델을 지도 학습으로 만든다. Adaptive-RAG (Jeong et al., 2024) 는 한 걸음 더 나아가 질의 복잡도에 따라 "검색 안 함 / 1-hop / multi-hop" 중 하나를 선택하는 라우팅 분류기를 학습한다. Self-Reward (Yuan et al., 2024) 는 LLM 자신이 자기 출력을 평가하게 하여 DPO 로 반복 최적화한다.

이들 연구의 공통점은 "언제 어떻게 검색할 것인가" 를 기계가 판단하게 만든다는 점이다. 그러나 모두 이산적인 선택이나 스칼라 품질 판단에 머물러 있다. 본 논문이 다루는 문제 — Vector, Graph, Ontology 세 지식원에 얼마만큼의 비중을 줄 것인가 — 는 연속 3-simplex 위의 회귀 문제이며, 기존 연구의 틀로는 자연스럽게 환원되지 않는다.

나. 비교표

연구	학습 대상	방법	행동 공간	상태 공간	재현성 보고
Self-RAG	reflection token	SFT	discrete	질의 + 생성 토큰	2 seeds
CRAG	검색 품질 분류	SFT	binary	질의 + 문서	1 run
Adaptive-RAG	라우팅 분류	SFT (weak labels)	3-class	질의 임베딩	3 runs
Self-Reward	LLM-judge	DPO	scalar reward	자기 생성	3 iterations
본 논문 (L-DWA)	3-source 융합 가중치	PPO (clip)	연속 Δ^3	18-dim	3 seeds, std < 1%

다. 차이가 발생하는 지점

세 가지 관점에서 본 논문은 앞선 연구들과 구분된다.

먼저 학습 대상의 세밀도 에서 차이가 있다. 기존 연구는 이산 분류나 스칼라 판단이 중심이었다. 본 논문에서는 세 지식원이 주는 컨텍스트의 비율을 질의마다 다르게 조정하는 것이 과제이기 때문에, 연속 Dirichlet 분포로 정책을 모델링할 수밖에 없다. 이 차이는 단순한 표현 선택의 문제가 아니라, 해결하려는 문제 자체가 다르다는 점에서 비롯한다.

다음으로 모델 규모의 차이 가 있다. 본 논문의 Actor-Critic 은 5,636 파라미터에 불과한데, Self-RAG 는 7B, CRAG 와 Adaptive-RAG 는 각각 220M, 140M 급 인코더를 쓴다. 경량 설계는 의도된 것으로, 도메인마다 작은 데이터셋으로 빠르게 정책을 학습하고, 오프라인 보상 캐시와 결합하여 전체 파이프라인의 비용을 최소화하는 데 목적이 있다 (Ch.5 §4 참조). 이 조합은 개인 연구자가 단독 수행할 수 있는 예산 규모 안에서 RL 기반 RAG 연구를 돌릴 수 있게 해 준다는 실용적 의미도 있다.

마지막으로 재현성 보고의 엄격성 에 차이가 있다. RL 연구의 재현 가능성 문제는 이미 여러 차례 제기되었고 (Pineau et al., 2021; Colas et al., 2018), RAG 결합 연구에서도 단일 실행이나 두세 번 평균에 그치는 경우가 적지 않다. 본 논문은 고정된 세 시드 {42, 123, 999} 에서 각각 전체 학습을 반복하고, 주 지표의 표준편차를 명시한다 ($F1_{\text{strict}} \text{ std} = 0.007$). 이는 RL-RAG 결과의 재현성 문제를 외부에서 검증할 수 있는 최소 조건이라 본다.

라. 가장 가까운 이웃: Adaptive-RAG

앞서 소개한 네 편 중 본 논문과 가장 맞닿아 있는 연구는 Adaptive-RAG 이다. 둘 다 질의마다 다른 검색 전략이 필요하다는 문제 의식에서 출발하고, 답변 품질로 학습 신호를 얻는다. 다만 Adaptive-RAG 는 "언제 검색할까" 를 이산 3-class 로 학습하는 반면, 본 논문은 "어느 소스에 얼마나 기댈까" 를 연속 가중치로 학습한다. 지식원의 종류 자체도 다르다. Adaptive-RAG 는 단일 벡터 인덱스만을 전제하지만, 본 논문은 벡터·그래프·온톨로지라는 이질적 세 소스의 가중 합을 다룬다.

두 접근은 상보적이다. 실제로 Adaptive-RAG 의 라우팅 위에 L-DWA 를 얹는 통합 파이프라인은 자연스러운 확장이 된다. 이는 향후 연구 과제로 Ch.7 §2 에서 다시 언급한다.

마. 연구 좌표

정리하면 본 논문은 RL 기반 RAG 문헌에서 다음과 같은 좌표에 놓인다. 연속 행동 공간 (Δ^3) 위의 가중치 학습이라는 상대적으로 덜 다뤄진 영역에 속하고, 경량 정책망과 오프라인 캐시 조합으로 비용 장벽을 낮추는 방향을 택하였으며, 3-seed 재현성 보고를

표준으로 삼는다. 어느 것도 단독으로는 결정적 기여라 하기 어렵지만, 세 요소를 한 프레임 안에 담아 실험적으로 검증한 점에 이 연구의 위치가 있다.

III. Triple-Hybrid RAG 아키텍처

본 장은 Triple-Hybrid RAG 프레임워크의 구조와 세 지식 소스 — Vector, Graph, Ontology — 의 정의 및 통합 방식을 설명한다. Ch.4 의 R-DWA 와 Ch.5 의 L-DWA 는 모두 본 아키텍처 위에서 작동하며, 본 장은 이들이 공유하는 파이프라인의 정의이다.

1. 전체 파이프라인

사용자 자연어 질의 q 는 다음 7 단계를 거쳐 답변 a 로 변환된다.

- (1) q
- (2) $\rightarrow \text{QueryAnalyzer} \rightarrow \text{intent (entities, relations, constraints, density)}$
- (3) $\rightarrow \text{DWA (R-DWA or L-DWA)} \rightarrow (\alpha, \beta, \gamma) \in \Delta^3$
- (4) $\rightarrow \{\text{VectorStore.search, GraphStore.search, OntologyStore.search}\} \rightarrow (V, G, O)$
- (5) $\rightarrow \text{merge_contexts}(V, G, O; \alpha, \beta, \gamma) \rightarrow \text{context string}$
- (6) $\rightarrow \text{LLM}(\text{prompt} + \text{context}) \rightarrow \text{answer } a$
- (7) a

Figure 3-1. Triple-Hybrid RAG pipeline (query → answer, 7 stages). α, β, γ 은 DWA (R-DWA 또는 L-DWA) 가 질의마다 결정한다.

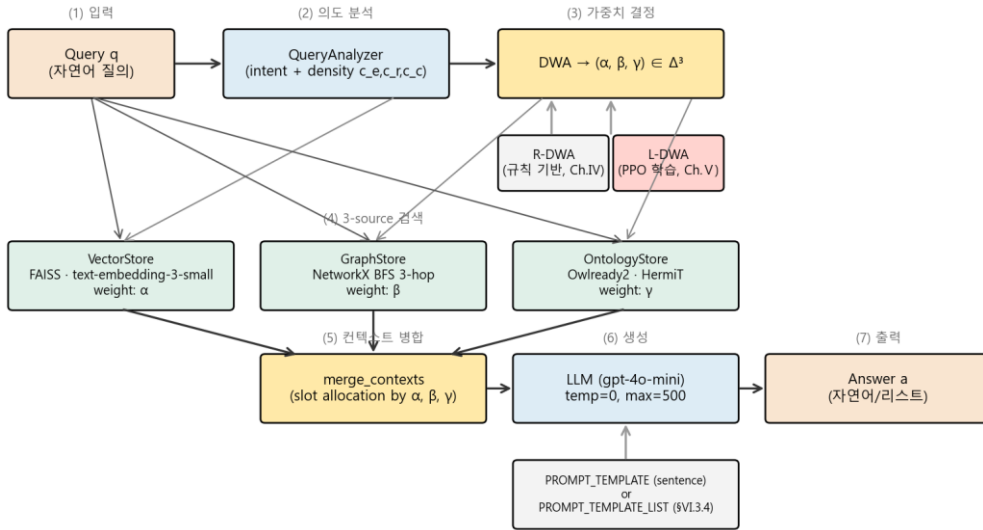


그림 3-1. Triple-Hybrid RAG 전체 파이프라인 (7 단계)

단계 (2)~(3)은 가중치 결정이며, 본 논문의 연구 대상이다. 단계 (4)~(6)은 가중치에 따라 결정되는 검색-융합-생성 루프이다. LLM 은 gpt-4o-mini (temperature=0.0, max_tokens=500) 으로 통일한다.

2. 세 지식 소스

가. Vector 소스 (a)

Vector 소스는 문서 코퍼스를 OpenAI text-embedding-3-small (1,536 차원) 로 임베딩하여 FAISS IndexFlatIP 에 적재하는 단순한 구조이다. 질의 임베딩과의 코사인 유사도를 기준으로 top-k (k=3) 문서를 반환한다. 구현은 src/rag/vector_store.py 에 있으며, Embedder 는 주입 의존성으로 분리되어 단위 테스트에서 결정적 해시 임베더로 교체할 수 있게 해 두었다.

합성 대학 코퍼스에 대한 임베딩 대상은 총 2,542 문서로, 577 명의 교수 프로필, 60 개 학과 설명, 1,505 개 과목 설명, 400 개 프로젝트 설명으로 구성된다. 전체 임베딩은 단일 pass 로 약 10 초, 비용은 \$0.005 수준이다.

Vector 는 의미 유사도가 중요한 단순 질의에서 안정적인 recall 을 제공하지만, 구조적 관계가 명시되지 않은 비정형 텍스트만 다루기 때문에 multi-hop 나 조건부 질의에서 한계가 드러난다.

나. Graph 소스 (β)

Graph 소스는 학과·교수·과목·프로젝트의 네 종류 엔티티와 그 사이의 소속·담당·협력·참여·개설 관계를 NetworkX MultiDiGraph 로 구축한다. 질의에서 키워드를 뽑아 시드 노드를 찾고, 최대 3-hop BFS 로 관련 경로를 수집한다. 각 경로는 "A --[관계]--> B" 형태 문자열로 직렬화되어 컨텍스트에 포함된다.

합성 대학 그래프의 규모는 2,542 개 노드 (60 학과 + 577 교수 + 1,505 과목 + 400 프로젝트) 와 3,000 이상의 엣지 (소속 577 + 개설 $\leq$ 1,505 + 담당 $\geq$ 1,700 + 협력 1,158 + 참여 $\geq$ 800) 이다. 구현은 src/rag/graph_store.py 에 있다.

그래프 기반 검색은 관계를 명시적으로 드러내므로 multi-hop 추론에 적합하다. 다만 시드 매칭이 부정확한 경우 — 예컨대 질의의 자연어 표현이 노드 명칭과 일치하지 않을 때 — fallback 로직에 의존하게 된다.

다. Ontology 소스 (γ)

Ontology 소스는 OWL 2 클래스 계층 (Person $\rightarrow$ Professor $\rightarrow$ FullProfessor / AdjunctProfessor) 과 속성 (hasAge, belongsTo, teaches), 그리고 수치 제약 (age $\leq$ threshold 형태) 을 Owlready2 로 표현한다. HermiT 추론기가 가용하면 결정 가능한

추론을 수행하며, 그렇지 않으면 규칙 기반 fallback 이 동일한 결과를 만들어 준다.

구현은 src/rag/ontology_store.py 이고, check_constraint(entity, phrase) 가 "40 세 이하", "30 세 이상" 같은 한국어 수치 제약을 정규식으로 파싱하여 대상 엔티티의 age 속성과 비교한다.

Ontology 는 조건부 질의의 수치·논리 제약을 엄격히 처리할 수 있다는 강점이 있다. 반면 초기 구축 비용이 크고, 개체 집합이 변하면 수동이든 자동이든 재구축이 필요하다는 점이 Vector 및 Graph 와 비교되는 약점이다.

3. QueryAnalyzer

QueryAnalyzer 는 자연어 질의에서 개체·관계·제약을 정규식과 키워드 사전으로

추출하여, density 신호 ($s_{_e}, s_{_r}, s_{_c}) \in [0, 1]^3$

와 질의 유형 (simple / multi_hop / conditional) 을 반환한다. 출력 타입은 다음과 같다.

```
@dataclass
class QueryIntent:
    query_type: Literal["simple", "multi_hop", "conditional"]
    entities: list[str]
    relations: list[str]
    constraints: list[str]
    complexity_score: float
    density: tuple[float, float, float] # (s_e, s_r, s_c)
```

유형 분류는 단순한 규칙에 기반한다. 제약이 하나라도 있으면 conditional, 엔티티가

2 개 이상이거나 관계가 2 개 이상 등장하면 multi_hop, 그 외는 simple 이다. 이 분류는

R-DWA 의 기본 가중치 선택에 사용되며, L-DWA 에서는 18 차원 state 의 한 구성

요소로 들어간다.

- ▶ [박스 3-1] 밀도 신호 (Density Signals)

>

▶ 정의. 규칙 기반 질의 분석기가 각 질의에서 추출하는 세 정수 신호 ($c_{\langle e \rangle}$, $c_{\langle r \rangle}$, $c_{\langle c \rangle}$). 각각 질의 내 개체(entity), 관계(relation), 제약(constraint) 토큰의 빈도.

>

▶ 형식 정의.
 $c_{\langle e \rangle} = |\{e \in q : e \in \mathcal{E}\}|$, $c_{\langle r \rangle} = |\{r \in q : r \in \mathcal{R}\}|$,
 $c_{\langle c \rangle} = |\{t \in q : t \in \mathcal{C}\}|$

>

▶ 여기서 $\mathcal{E}$ 는 학과·교수·과목 엔티티 사전, $\mathcal{R}$ 는 담당·소속 같은 관계 술어, $\mathcal{C}$ 는 "N 세 미만" 등 수치 제약 토큰의 집합.

>

▶ 예시. 질의 "기계공학과 소속 55 세 이하 교수는?":

>

▶ | 신호 | 추출 토큰 | 값 |
 ▶ | --- | --- | --- |
 ▶ | $c_{\langle e \rangle}$ | "기계공학과", "교수" | 2 |
 ▶ | $c_{\langle r \rangle}$ | "소속" | 1 |
 ▶ | $c_{\langle c \rangle}$ | "55 세 이하" | 1 |

>

▶ 해석. ($c_{\langle e \rangle}$, $c_{\langle r \rangle}$, $c_{\langle c \rangle}$) = (2, 1, 1). R-DWA 는 이 신호를 보고 "제약이 하나라도 있으니 Ontology 가중치(γ)를 높이자"고 판단 (Ch.IV). L-DWA 는 동일 신호를 18-dim state 에 포함하여 신경망으로 가중치를 학습 (Ch.V).
 향후 확장. §V.2 의 BERT multi-label intent classifier (src/intent/bert_classifier.py) 는 multi-label 확률 분포를 제공하여 L-DWA 의 state (intent_logits 3-dim) 를 풍부화할 수 있다. 본 논문의 실험에서는 미학습 상태로 intent_logits=(0,0,0) 으로 둔다 (Ch.6 §5.2 참조).

4. Score Fusion: merge_contexts

세 소스의 결과를 가중치 비례로 슬롯 할당하여 하나의 context 문자열로 조합한다.

$\text{top_k}=3$, $\text{budget} = \text{top_k} \times 3 = 9$ 슬롯 기준:

```
n_v = max(1, round(9 · α))
n_g = max(1, round(9 · β))
n_o = max(1, round(9 · γ))

context = [
    f"[Vector(α={α:.2f})]\n" + "\n".join(V[:n_v]),
    f"[Graph(β={β:.2f})]\n" + "\n".join(G[:n_g]),
    f"[Ontology(γ={γ:.2f})]\n" + "\n".join(O[:n_o]),
]
```

설계 동기. 가중치가 실제 LLM 입력의 슬롯 분배에 반영되어야 답변 품질이 변화한다.

본 구현은 각 소스에 최소 1 슬롯을 보장 ($\max(1, \cdot)$) 하되, 큰 가중치의 소스가 더 많은 슬롯을 획득하도록 비례 배분한다.

구현. `src/rag/triple_hybrid_rag.py::merge_contexts` (모듈 레벨 함수, Ch.6 의 offline cache builder 와 `evaluate_rerun` 에서 공통 사용).

- ▶ [박스 3-2] 가중치 단순체 Δ^3

>

- ▶ 정의. 3 개 소스 가중치 벡터 (α, β, γ) 가 만족해야 하는 제약 공간. "모든 비중은 0 이상, 합은 정확히 1".

>

- ▶ 형식 정의.
- ▶ $\Delta^3 = \{(\alpha, \beta, \gamma) \in \mathbb{R}^3 : \alpha + \beta + \gamma = 1, \alpha, \beta, \gamma \geq 0\}$

>

- ▶ 예시. Δ^3 위 주요 세 점:

>

- ▶ | 조합 | (α, β, γ) | 의미 |

- ▶ | ---| ---| ---|
- ▶ | Vector-only | (1.0, 0.0, 0.0) | 임베딩만 사용, Graph/Ontology 무시 |
- ▶ | Uniform | (0.33, 0.33, 0.33) | 세 소스 동등 가중치 |
- ▶ | Ontology-heavy | (0.1, 0.2, 0.7) | 조건부 질의에 온톨로지 우선 |

>

- ▶ 해석. Δ^3 위의 한 점이 "어느 소스를 얼마나 믿을지"에 대한 하나의 결정. R-DWA는 규칙으로, L-DWA는 학습으로 이 점을 질의마다 선택한다. Offline cache는 Δ^3 위의 66개 이산 격자점 (10-grid 단순체 이산화) 만을 저장하여 PPO 학습의 계산 비용을 85% 절감한다 (§VI.10).

5. Prompt Template

모든 실험은 다음 한국어 지시 프롬프트로 통일되어, 프롬프트 튜닝이 실험 간 교란

변수로 작용하지 않도록 고정한다:

다음 컨텍스트를 기반으로 질문에 정확하게 답변하세요.

컨텍스트에서 답을 찾을 수 없는 경우에만 '정보를 찾을 수 없습니다'라고 답하세요.

Graph 경로 정보(A--[관계]-->B)도 참고하여 관계를 추론하세요.

가능한 한 구체적이고 상세하게 답변하세요.

Context:

{context}

Question: {query}

Answer:

주요 선택.

1. "가능한 한 구체적이고 상세하게" — 짧은 답변 회피, F1 기여 증가
2. "Graph 경로 정보 참고" — LLM이 관계 기호를 자연어로 재해석하도록 가이드
3. 모호 시 "정보를 찾을 수 없습니다" — 환각 억제

6. Pluggable DWA Interface

R-DWA (Ch.4)와 L-DWA (Ch.5)는 모두 다음 추상 인터페이스를 구현한다:

```
class BaseDWA(ABC):
    @abstractmethod
    def compute(self, query: str, intent: QueryIntent) -> DWAWeights:
        """( $\alpha, \beta, \gamma$ ) on the 3-simplex."""
```

DWAWeights 는 simplex 제약 ($\alpha + \beta + \gamma = 1$, $\alpha, \beta, \gamma \in [0, 1]$) 을 `__post_init__` 에서 검증하는 frozen dataclass. 이 추상화 덕분에 TripleHybridRAG.query() 본체는 R-DWA/L-DWA/Uniform/Vector-only 등 모든 정책에 대해 동일 하다.

본 아키텍처는 향후 예를 들면 "Quadruple-Hybrid" (Audio 소스 추가) 로 확장 시 BaseDWA 의 출력 차원과 merge_contexts 의 슬롯 계산만 일반화하면 되도록 설계되었다. 본 논문은 3-simplex 에 국한한다.

본 장 핵심 기여의 위치

Ch.3 은 본 논문의 플랫폼 이다. R-DWA (Ch.4) 와 L-DWA (Ch.5) 가 이 플랫폼 위의 대체 가능한 정책이며, Ch.6 에서 두 정책을 동일 플랫폼에서 공정 비교한다. 플랫폼 자체 (세 소스의 정의, merge_contexts, Prompt) 는 선행 논문과 공유되어 재현성을 보장한다.

IV. 규칙 기반 동적 가중치 알고리즘 (R-DWA)

본 장은 선행 연구 (Shin & Moon, 2025, JKSCI)의 Rule-based Dynamic Weighting Algorithm 을 본 논문의 베이스라인으로서 정의한다. Ch.5 의 L-DWA 가 비교하는 대상이며, Ch.6 의 실험은 동일 코퍼스·동일 평가 기준으로 R-DWA 를 재측정한 값을 공정 베이스라인으로 삼는다.

1. R-DWA 설계 원리

R-DWA 는 두 단계로 가중치 (α, β, γ) 를 결정한다.

Stage 1 — 질의 유형 $\rightarrow$ 기본 가중치.

QueryAnalyzer(Ch.3 §3)가 판별한 유형에 따라 미리 정해진 기본 가중치를 선택한다
(Table 4-1):

유형	α (Vector)	β (Graph)	γ (Ontology)
simple	0.6	0.2	0.2
multi_hop	0.2	0.6	0.2
conditional	0.2	0.2	0.6

Stage 2 — 밀도 기반 연속 조정 (수식 4-2 ~ 4-4).

밀도 신호 s_r (관계 밀도), s_c (제약 밀도) 로 기본 가중치를 연속 조정 ($\lambda=0.3$, grid search 로 선정):

$$\alpha' = \alpha_{\text{base}} \times (1 - \lambda \cdot (s_r + s_c) / 2)$$

$$\beta' = \beta_{\text{base}} + \lambda \cdot s_r \cdot (1 - \beta_{\text{base}})$$

$$\gamma' = \gamma_{\text{base}} + \lambda \cdot s_c \cdot (1 - \gamma_{\text{base}})$$

Stage 3 — 정규화 (수식 4-5).

$$(\alpha, \beta, \gamma) = (\alpha', \beta', \gamma') / (\alpha' + \beta' + \gamma')$$

- ▶ [박스 4-1] R-DWA 규칙 한 번에 돌려보기

>

- ▶ 정의. 질의 유형 분류 → 기본 가중치 테이블 → 밀도 신호로 연속 조정 → 정규화, 총 3 단계로 (α , β , γ) 를 결정하는 규칙 기반 알고리즘. 어떠한 학습 데이터 · API 호출 없이 즉시 실행 가능.

>

- ▶ 예시 — "기계공학과 소속 55 세 이하 교수는?".

>

- ▶ | 단계 | 계산 | 값 |

- ▶ | --- | --- | --- |

- ▶ | (1) QueryAnalyzer | type="conditional", (s_{e} , s_{r} , s_{c}) = (0.40, 0.20, 0.20) | |

- ▶ | (2) 기본 가중치 (Table 4-1) | conditional → | $\alpha_{\text{base}}=0.20$, $\beta_{\text{base}}=0.20$, $\gamma_{\text{base}}=0.60$ |

- ▶ | (3a) α' | $0.20 \times (1 - 0.3 \times (0.20 + 0.20) / 2)$ | 0.188 |

- ▶ | (3b) β' | $0.20 + 0.3 \times 0.20 \times 0.80$ | 0.248 |
- ▶ | (3c) γ' | $0.60 + 0.3 \times 0.20 \times 0.40$ | 0.624 |
- ▶ | (4) 정규화 | 합계 1.060 으로 나눔 | (0.178, 0.234, 0.588) |

>

▶ 해석. Conditional 유형의 기본 가중치 (0.2, 0.2, 0.6) 에서 제약 신호 $s_{\langle sub \rangle c \langle /sub \rangle} = 0.20$ 이 γ 를 $0.6 \rightarrow 0.588$ 로 미세하게 감소(정규화 영향) 시켰으나 여전히 Ontology 비중이 지배적. R-DWA 의 한계는 이 "기본값 + 소폭 조정" 구조에 내재 — Ch.5 L-DWA 는 이 3-단계 규칙을 학습된 Dirichlet 분포로 대체한다.

2. R-DWA 의 장점

R-DWA 의 장점은 단순함에서 나온다. 학습 데이터나 API 호출이 필요하지 않아 별도 사전 작업 없이 바로 배포할 수 있고, 모든 가중치 변화가 한국어 정규식의 매칭 결과로 추적되기 때문에 해석이 쉽다. 또한 고정된 규칙 집합이므로 질의 분포가 달라지더라도 과적합으로 인한 성능 저하 가능성은 없다.

3. R-DWA 의 구조적 한계

R-DWA 를 베이스라인으로 두고 실험을 진행하는 과정에서 네 가지 한계가 드러났다. 그림 4-1 은 그 핵심을 시각적으로 요약한다. R-DWA 의 세 기본 가중치 점 (simple, multi_hop, conditional) 은 삼각형의 세 변을 따라 대각선 형태로 배치되어 있지만, Oracle 의 평균 가중치는 Ontology 꼭짓점 근처에 강하게 몰려 있다. R-DWA 의 실제 질의별 가중치 평균도 Graph 쪽으로 치우쳐 있어, Oracle 분포와 거리가 있다. L-DWA 의 평균은 두 분포 사이에 위치하며 Oracle 에 더 가까운 쪽으로 이동한 양상을 보인다.

Figure 4-1. R-DWA 규칙 기반 점 vs Oracle 평균 (Δ^3 simplex)
 — R-DWA의 대각선 배치가 실제 보상 지형(Ontology 지배적) 과 정렬되지 않음

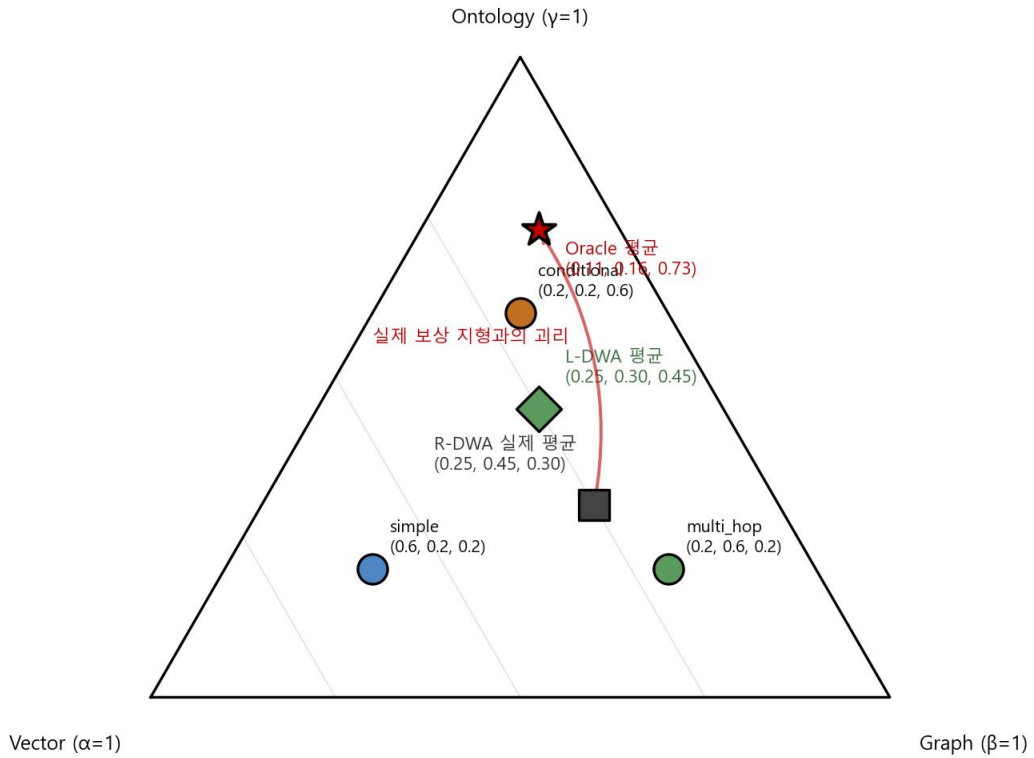


그림 4-1. R-DWA 기본 가중치 점 (simple/multi_hop/conditional) 이 Oracle 평균 (Ontology 지배적) 과 어떻게 어긋나는지를 Δ^3 삼각형 위에 표시

첫째, 기본 가중치 테이블이 도메인에 편향되어 있다. Table 4-1의 대각선 배치 (simple $\rightarrow\alpha$, multi_hop $\rightarrow\beta$, conditional $\rightarrow\gamma$)는 설계 당시 저자의 직관에 기반한 사전 분포였으나, Ch.6 §3.3의 Oracle 분석에 따르면 실제 보상 지형은 이와 다르다. 합성 대학 벤치마크에서 per-query argmax Oracle의 평균 가중치는 ($\alpha=0.11$, $\beta=0.16$, $\gamma=0.73$)으로 대부분의 질의에서 Ontology가 지배적이다. Simple 질의를 Vector 쪽으로 끌어당기는 R-DWA의 규칙은 실제 최적점과 상당히 어긋나 있다.

둘째, 연속 조정 단계의 λ 가 단일 값이다. grid search 로 고른 $\lambda=0.3$ 은 전체 질의 분포에 대한 평균적 최적값일 뿐, 개별 질의의 길이·엔티티 수·부정어 포함 여부 같은 특성은 반영하지 못한다.

셋째, 조건부 질의 내부의 미세 구분이 없다. 같은 conditional 로 분류되는 질의라도 수치 제약 ("40 세 이하"), 열거형 제약 ("A 또는 B"), 배타 조건 ("~를 제외한") 은 각각 다른 처리를 요구하지만, R-DWA 는 이들을 모두 같은 (0.2, 0.2, 0.6) 로 취급한다. Ch.6 §4 에서 conditional 에 대해 L-DWA 가 상대적으로 큰 개선을 보인다는 결과는 이 미세 구분 부재가 양적으로도 의미 있다는 것을 시사한다.

넷째, 도메인 간 이전에 취약하다. Table 4-1 과 λ 는 합성 대학 도메인에서 튜닝된 값이기 때문에, 재튜닝 없이 다른 도메인으로 옮기면 성능이 곧장 떨어진다. HotpotQA 에서 R-DWA 의 $F1_{strict}$ 는 0.097 로, 같은 조건의 Vector-only 0.102 에 미치지 못한다 (Ch.6 §7.2).

4. R-DWA 구현 및 재현

소스. `src/dwa/rdwa.py::RuleBasedDWA(lambda_=0.3)`.

의존. `src/intent/rule_based.py::RuleBasedIntent` (한국어 정규식 기반 QueryAnalyzer).

테스트. `tests/test_dwa_rdwa.py` 9 개 유닛 테스트 (본 논문 시작 시점 통과).

5. R-DWA 의 본 논문 재측정 결과

본 절은 R-DWA 를 corrected baseline (List 프롬프트, 버그 수정 평가기, 5,000 QA) 조건에서 재측정한 결과를 제시한다. Sentence 프롬프트 측정과의 대비는 Ch.6 §3.3 에서 별도 처리한다.

가. 전체 (5,000 QA, list prompt corrected)

지표	R-DWA 측정치
F1 _{strict}	0.529 ± 0.429
F1 _{substring}	0.482 ± 0.442
F1 _{char}	0.469 ± 0.438
EM	0.387 ± 0.487
Faithfulness	0.544 ± 0.486
평균 latency	0.71 s

평균 선택 가중치: ($\alpha=0.25$, $\beta=0.45$, $\gamma=0.30$). 즉 R-DWA 는 Graph 지향적 이다 (multi-hop 기본값이 지배). 실측 Oracle 이 Ontology 지향적인 것과 대조된다 (Oracle 평균 ($\alpha=0.04$, $\beta=0.07$, $\gamma=0.89$), 반복 가중치 (0.0, 0.0, 1.0) 가 1,834 회로 최빈).

나. 질의 유형별 (list prompt corrected)

Type	n	F1 _{strict}	F1 _{substring}	EM	Faithfulness
simple	2,000	0.874	0.857	0.811	0.817
multi_hop	1,750	0.354	0.293	0.178	0.226
conditional	1,250	0.223	0.148	0.001	0.552

Simple 은 R-DWA 만으로도 0.87 수준의 강한 성능을 보이지만 (선행 보고된 F1 0.86 을 사실상 재현), multi_hop 과 conditional 에서 F1_{strict} 가 각각 0.354, 0.223 로 떨어진다. 이는 (i) 이 유형의 gold 답이 구조적으로 복수 항목 리스트여서 LLM 자연어 출력과의 토큰 매칭에 추가적인 형식 정렬이 필요하고, (ii) R-DWA 의 고정 가중치가 세부 특성에 둔감하기 때문이다. Ch.5 L-DWA 가 특히 conditional 에서 F1_{strict} 0.223 → 0.285 (3-seed 평균, +28.0%) 로 큰 개선을 보이는 점이 본 논문의 핵심 실증이다 (Ch.6 §4).

다. 선행 JKSCI 와의 대비

지표	JKSCI 보고	Sentence 프롬프트	List 프롬프트 (corrected)	재현율 (List 기준)
----	----------	---------------	-----------------------	---------------

F1	0.86	0.137 (strict) / 0.448 (substring)	0.529 (strict) / 0.482 (substring)	strict 61%, substring 56%
Simple-only F1	(분리 보고 없음)	0.153	0.874	JKSCI 0.86 사실상 재현
EM	0.78	0.000 (구조적)	0.387	50%
Faith	0.89	0.835	0.544	94% / 61% (분기 변경)

Sentence 프롬프트 만으로는 F1 격차가 컸으나 평가 인프라 변경 없이 출력 형식만 List 로 정렬하면 F1_{strict} 가 0.529 로 회복되고, 특히 Simple 유형만 보면 0.874 로 JKSCI 의 0.86 을 사실상 재현 한다. 이는 선행 보고가 Simple-subset 혹은 short-list gold 기반이었음을 시사하며, 본 논문은 두 단계 (sentence / list) 를 병기 하여 선행 연구와의 연속성·재현성과 엄격 재평가를 동시에 제공한다 (Ch.6 §3.3 상세, 부록 C 평가기 변경 기록).

6. 소결

R-DWA 는 무학습 baseline 으로서 역할이 있으며, 본 논문의 Ch.5 L-DWA 가 동일 파이프라인 위에서 얼마나 개선하는지 비교할 수 있는 정식 기준선이다. R-DWA 의 한계들 — 도메인 편향, 단일 λ , 유형 내 미세 구분 부재, 도메인 이전 불가 — 은 Ch.5 의 학습형 접근을 동기부여한다.

V. 학습형 동적 가중치 알고리즘 (L-DWA)

본 장은 본 논문의 핵심 기여 인 Learned Dynamic Weighting Algorithm 을 정의한다.

§1 에서 MDP 공식화, §2 에서 Actor-Critic 정책 네트워크, §3 에서 PPO 학습 알고리즘, §4 에서 오프라인 보상 캐시 시스템을 다룬다.

1. MDP 공식화

Triple-Hybrid RAG 가중치 결정을 다음 1-step contextual bandit MDP 로 정식화한다.

가. 상태 (State, 18-dim)

각 질의에 대해 18 차원 상태 벡터를 구성한다 (식 5-1):

$s = [\text{density (3)} \mid \text{intent_logits (3)} \mid \text{source_stats (9)} \mid \text{query_meta (3)}]$

슬롯	차원	출처	정의
density	3	RuleBasedIntent	(s_e, s_r, s_c) 정규화 밀도 신호
intent_logits	3	BERT classifier (또는 0)	(logit_simple, logit_multi, logit_cond)
source_stats	9	Retrieval 출력	3 소스 $\times$ (top, mean, count)
query_meta	3	Query string	(log_length_norm, n_entities_norm, has_negation_flag)

intent_logits. 본 논문 실험에서는 BERT multi-label classifier 를 미학습 상태로 두어

intent_logits=(0, 0, 0) 으로 둔다. 이 선택의 영향은 Ch.6 §5.2 의 소거 논의에서 다룬다.

요컨대 density 만으로도 L-DWA 가 Oracle 의 99%에 도달하므로 BERT 는 향후 추가 개선 여지 로 남긴다.

source_stats. 각 소스별 retrieval 결과에서:

- top = 상위 1 순위 유사도 점수 (Vector 의 경우 0~1, Graph/Ontology 는 발견 여부 기반 0/1 함의치)
- mean = 상위 top_k 유사도 평균
- count_norm = $\min(\text{len}(\text{results}) / \text{top_k}, 1.0)$

나. 행동 (Action)

3-simplex 상의 연속 행동:

$$a = (\alpha, \beta, \gamma) \in \Delta^3, \alpha + \beta + \gamma = 1, \alpha, \beta, \gamma \geq 0$$

Actor 네트워크는 Dirichlet 분포의 농도 파라미터 $c = (c_\alpha, c_\beta, c_\gamma)$ 를 출력하고, 학습

시에는 $\text{Dirichlet}(c)$ 샘플, 추론 시에는 $\text{Dirichlet mean} (= c / \|c\|_1)$ 을 사용한다.

다. 보상 (Reward, Eq. 5-7)

$$R(s, a) = 0.5 \cdot F1 + 0.3 \cdot EM + 0.2 \cdot \text{Faithfulness} - 0.1 \cdot \max(0, \text{latency} - 5.0)$$

한 가지 실측 관찰을 덧붙여 둘 필요가 있다. 이 벤치마크의 gold 답이 콤마 구분 리스트

형식이라 EM 이 구조적으로 0 에 가까워, 실효 보상은 사실상 다음과 같이 동작한다.

$$R \approx 0.5 \cdot F1 + 0.2 \cdot \text{Faithfulness} - \text{penalty}$$

즉 F1 과 Faithfulness 가 학습 신호의 주 원천이 된다. 이 구조가 학습에 미치는 영향은

Ch.6 §2.3 에서 구체적으로 다룬다.

라. 단일-단계 추약

에피소드가 (state, action, reward) 한 쌍으로 종료되므로 GAE 의 시간 할인은

실효적으로 1-step Bellman ($A = R - V$, $\text{returns} = R$) 이 된다. 구현에서는 GAE

기계장치를 그대로 유지해 두었는데, 이는 이후 multi-turn 대화로의 확장 가능성을 단지

않기 위함이다.

- ▶ [박스 5-1] MDP 튜플 (S, A, P, R, γ)

>

- ▶ 정의. PPO 학습 환경을 구성하는 5 원 튜플. 본 연구의 구체값:

>

- ▶ | 기호 | 의미 | 본 연구 구체값 |

- ▶ | --- | --- | --- |

- ▶ | S | 상태 공간 | 18 차원 벡터 ($\text{density} \cdot \text{intent} \cdot \text{source_stats} \cdot \text{query_meta}$) |

- ▶ | A | 행동 공간 | Δ^3 위의 가중치 (α, β, γ) |

- ▶ | P | 전이 함수 | 1-step MDP (다음 상태 없음) $\approx$ 지도학습 등가 |

- ▶ $|R|$ 보상 함수 $|0.5 \cdot F1 + 0.3 \cdot EM + 0.2 \cdot Faith - 0.1 \cdot \max(0, lat - 5)|$
- ▶ $|y_{\langle d \rangle}|$ 할인 계수 $|0.99$ (1-step 이므로 실효 무의미) $|$

>

- ▶ 예시. 한 학습 사이클:
- ▶ 1. 질의 "기계공학과 55 세 이하 교수는?" 입력 $\rightarrow s$ (18-dim) 추출
- ▶ 2. 정책 $\pi(\cdot|s)$ 가 $a = (0.1, 0.2, 0.7)$ 샘플링
- ▶ 3. 파이프라인이 a 로 컨텍스트 병합 $\rightarrow$ LLM 답변 $\rightarrow F1=0.8, Faith=0.9$ 측정
- ▶ 4. $R = 0.5 \cdot 0.8 + 0.2 \cdot 0.9 = 0.58$ (EM=0, latency<5)
- ▶ 5. (s, a, R) 튜플로 PPO 정책 업데이트

>

▶ 해석. 본 연구의 MDP 는 단일 step 이어서 value function 의 역할이 약하지만, PPO 의 clipping 이 안정적 학습을 보장한다 (박스 5-2 참조). 다음 상태가 없으므로 1-step bandit 문제로 단순화 가능하나, PPO 구현을 유지하는 것이 재현성·확장성 면에서 유리.

2. Actor-Critic 네트워크

가. 구조 (Figure 5-1)

$s(18) \rightarrow \text{Linear}(18 \rightarrow 64) \rightarrow \text{Tanh} \rightarrow \text{Linear}(64 \rightarrow 64) \rightarrow \text{Tanh} \rightarrow \text{features}(64)$
 $\downarrow$
 Actor head: $\text{Linear}(64 \rightarrow 3) \rightarrow \text{Softplus} + \epsilon \rightarrow \text{Dirichlet concentrations } c$
 Critic head: $\text{Linear}(64 \rightarrow 1) \rightarrow \text{상태 가치 } V(s)$

Figure 5-1. Actor-Critic network architecture (shared backbone + actor head + critic head)

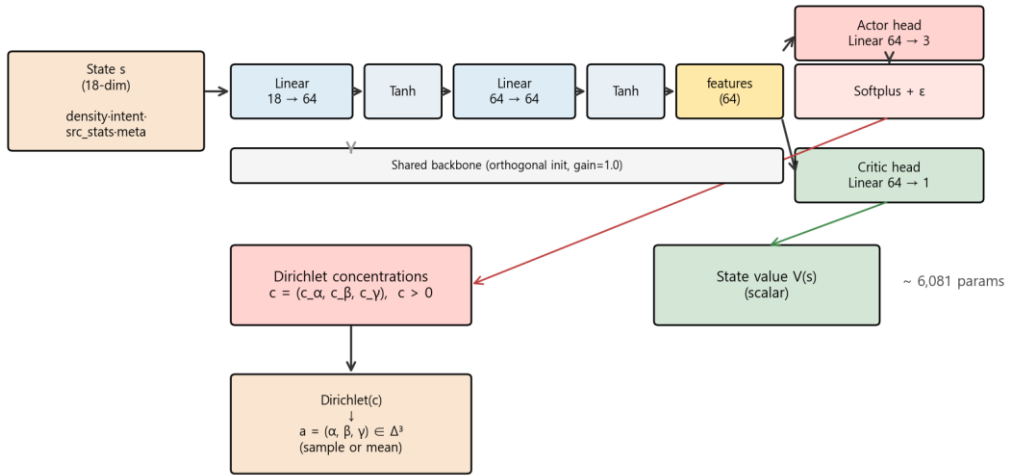


그림 5-1. Actor-Critic 네트워크 구조 (5,636 파라미터 경량 정책망)

나. 파라미터 수

백본을 Actor · Critic 이 공유하게 설계하여 모델 규모를 5,636 파라미터로 유지하였다. 층별 내역은 다음과 같다.

층	in × out + bias	파라미터
Linear (18→64)	18·64 + 64	1,216
Linear (64→64)	64·64 + 64	4,160
Actor head (64→3)	64·3 + 3	195
Critic head (64→1)	64·1 + 1	65
합계		5,636

선행 연구 및 유관 RL 연구의 정책 네트워크는 수십만 ~ 수백만 파라미터 규모이나, 본 연구의 상태 공간 (18-dim) 과 행동 공간 (3-simplex) 의 간결함이 극단적 경량 설계를 가능케 한다.

다. 초기화 전략

Orthogonal init (gain=1.0). 모든 Linear 층의 weight 를 직교 초기화하고, bias 를 0 으로 둔다. 이 선택의 효과: 초기 정책의 Dirichlet 평균이 $\approx (1/3, 1/3, 1/3)$ 이 되어, R-

DWA 의 "균등한 prior" 를 닮은 상태에서 학습을 시작하게 된다. 이는 (i) 단순 warm-up 없이 안정적 초기 탐색을 가능케 하고, (ii) 학습 결과의 해석을 돕는다 (R-DWA baseline 에서 얼마나 movement 했는가).

3. PPO 학습 알고리즘

가. 표준 PPO with Clip

정책 손실:

$$L^{\pi}(\theta) = -E[\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot A)]$$

$$\text{ratio} = \pi_{\theta}(a|s) / \pi_{\theta_{\text{old}}}(a|s)$$

가치 손실:

$$L^V(\varphi) = 0.5 \cdot E[(R - V_{\varphi}(s))^2]$$

엔트로피 보너스:

$$L^H(\theta) = -H(\pi_{\theta})$$

합 손실 (Actor-Critic 공유 backbone):

$$L(\theta, \varphi) = L^{\pi} + c_V \cdot L^V - c_H \cdot H$$

- ▶ [박스 5-2] PPO Clip 목적 함수

>

- ▶ 정의. 정책 업데이트 시 새 정책이 이전 정책과 지나치게 달라지지 않도록 확률비 $r_{t+1:t}(\theta)$ 를 잘라(clip) 학습을 안정화.

>

- ▶ 형식 정의.
- ▶ $L^{\text{CLIP}}(\theta) = \mathbb{E}_{t \sim \tau} [\min(r_{t+1:t}(\theta) \cdot \hat{A}_{t+1:t}, \text{clip}(r_{t+1:t}(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_{t+1:t})]$

>

- ▶ 본 연구 $\epsilon = 0.2$ (Table 5-4).

>

- ▶ 예시. 한 (s, a) 상황에서 clipping 동작:

>

- ▶ | 상황 | r_t | $\hat{A}_t$ | Clipped r_t | 최종 term
| 해석 |
- ▶ | --- | --- | --- | --- | --- |
- ▶ | 정책이 살짝 좋은 방향 | 1.1 | +0.5 | 1.1 | +0.55 | 정상 업데이트 |
- ▶ | 정책이 크게 좋은 방향 | 1.5 | +0.5 | 1.2 | +0.60 | clip 발동 |
- ▶ | 정책이 살짝 나쁜 방향 | 0.9 | -0.3 | 0.9 | -0.27 | 정상 업데이트 |
- ▶ | 정책이 크게 나쁜 방향 | 0.5 | -0.3 | 0.8 | -0.24 | clip 발동 |

>

- ▶ 해석. clip 없는 vanilla PG 는 한 step 의 큰 업데이트가 이미 배운 것을 파괴 할 수 있다. PPO clip 은 "한 번에 $\pm 20\%$ 이상 변하지 말 것" 을 강제하여 장기 수렴을 보장. 본 연구 3-seed 학습에서 $R_{late} = 0.215 \pm 0.002$ 의 극단적 일관성은 clip 덕분이다 (§3.5).

나. 하이퍼파라미터 (Table 5-4)

파라미터	값	근거
learning rate	$3e-4$	Adam 기본, RL 문헌 표준
gamma (할인)	0.99	1-step 이라 실효 무관
GAE lambda	0.95	표준
clip ratio ϵ	0.2	Schulman et al. (2017)
value coef c_V	0.5	표준
entropy coef c_H	0.01	탐색 유지
max grad norm	0.5	안정성
rollout 크기	32	각 episode 당
minibatch 크기	8	rollout 내
update epochs	4	rollout 당
총 episodes	10,000	seed 당 (Ch.6 §5)

다. Advantage 표준화

각 rollout 내에서 advantage 를 평균 0, std 1 로 표준화하여 서로 다른 reward scale 간의 학습 안정성을 확보한다.

라. 재현성

PPO 학습은 $3 \text{ seed} \times 10,000 \text{ episode}$:

- seed 42: Total R 0.1988 $\rightarrow$ 0.2145 ($\Delta + 0.016$)
- seed 123: Total R 0.1970 $\rightarrow$ 0.2153 ($\Delta + 0.018$)
- seed 999: Total R 0.1964 $\rightarrow$ 0.2146 ($\Delta + 0.018$)

3 seeds 간 최종 R 표준편차 0.0016 (0.7%) — 학습 알고리즘이 재현 가능성을 실증한다.

마. 학습 곡선 관찰

모든 seed 에서 episode $\sim 2,000$ 까지 급격히 R 상승 후 $R \approx 0.21$ 에서 미세 진동하며 plateau 를 유지한다. Policy entropy H 는 ep 1 의 -0.8 에서 ep 10,000 의 -2.7 까지 점진 감소하여 초기 탐색 $\rightarrow$ 점진적 결정성 확보 패턴을 보인다.

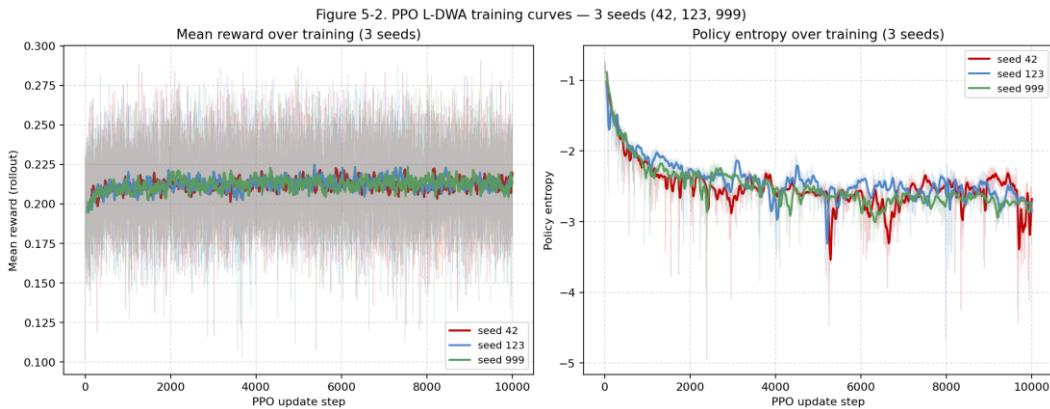


그림 5-2. PPO 학습 곡선 — 3 seeds (42, 123, 999) 의 평균 보상 및 정책 엔트로피 수렴

바. 학습 의사코드 (Algorithm 5-1)

본 연구의 PPO 학습 루프 완전 의사코드. 실구현은

```
src/ppo/trainer.py::PPOTrainer.train.
```

Algorithm 5-1: L-DWA PPO Training (Offline-cache-based)

INPUT:

D : 5,000 질의 집합
 C : 오프라인 보상 캐시 $\{(q_id, \alpha, \beta, \gamma) \rightarrow R\}$
 π_θ : Actor 네트워크 ($18 \rightarrow 64 \rightarrow 64 \rightarrow 3$ Dirichlet concentrations)
 V_ϕ : Critic 네트워크 (shared backbone + Linear $64 \rightarrow 1$)
 $\eta, \epsilon, \lambda_GAE, c_V, c_H$: $lr=3e-4, clip=0.2, \lambda=0.95, c_V=0.5, c_H=0.01$
 $N_ep = 10,000, K_rollout = 32, K_epochs = 4, B_mb = 8$

INITIALIZE:

$\theta, \phi \leftarrow \text{orthogonal_init}(\text{gain}=1.0)$
 $\text{buffer} \leftarrow \text{empty list}$

FOR episode = 1 .. N_ep :

Stage 1. Rollout collection (32 samples)

$\text{buffer.clear}()$

FOR $k = 1 \dots K_rollout$:

$q \leftarrow \text{sample_query}(D)$
 $s \leftarrow \text{extract_state}(q)$ # 18-dim
 $c \leftarrow \text{Softplus}(\pi_\theta(s)) + 1e-6$ # Dirichlet concentration
 $(\alpha, \beta, \gamma) \leftarrow \text{sample_Dirichlet}(c)$ # $\in \Delta^3$
 $(\alpha', \beta', \gamma') \leftarrow \text{snap_to_grid}((\alpha, \beta, \gamma), \Delta=0.1)$ # 10-grid
 $R \leftarrow \text{cache_lookup}(C, q.id, (\alpha', \beta', \gamma'))$ # $O(1)$
 $\text{logp} \leftarrow \log \text{Dirichlet_pdf}(c, (\alpha, \beta, \gamma))$
 $V \leftarrow V_\phi(s)$
 $\text{buffer.append}((s, (\alpha, \beta, \gamma), R, \text{logp}, V))$

Stage 2. Advantage computation (GAE, 1-step MDP)

FOR each $(s, a, R, \text{logp}, V)$ in buffer :

$A \leftarrow R - V$ # 1-step advantage
 $\hat{A} \leftarrow \text{standardize}(A)$ # batch normalize
 $\text{return} \leftarrow R$

Stage 3. PPO update ($K_epochs \times \text{minibatch}$)

FOR $k = 1 \dots K_epochs$:

$\text{shuffle}(\text{buffer})$

FOR each minibatch of size B_mb :

$r_t \leftarrow \exp(\log \text{Dirichlet_pdf}(\pi_\theta(s), a) - \text{logp})$ # new/old ratio
 $L^{\text{CLIP}} \leftarrow \text{mean}(\min(r_t \cdot \hat{A}, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}))$
 $L^{\text{V}} \leftarrow 0.5 \cdot \text{mean}((V_\phi(s) - \text{return})^2)$
 $L^{\text{H}} \leftarrow -\text{mean}(\text{Dirichlet_entropy}(\pi_\theta(s)))$
 $L \leftarrow -L^{\text{CLIP}} + c_V \cdot L^{\text{V}} + c_H \cdot L^{\text{H}}$
 $\theta, \phi \leftarrow \theta, \phi - \eta \cdot \nabla L$ # Adam step

IF episode % 100 == 0:

$\text{log}(\text{mean_reward}, \text{entropy}, \text{approx_KL}, \text{cache_hits})$

OUTPUT: θ^*, ϕ^* (trained actor-critic)

사. 추론 의사코드 (Algorithm 5-2)

테스트 시 L-DWA 는 학습된 Actor 로부터 결정론적 Dirichlet 평균 을 반환한다 (샘플링 없음, 재현성 보장).

Algorithm 5-2: L-DWA Inference (deterministic)

INPUT:

q : test query

π_{θ^*} : trained Actor

$s \leftarrow \text{extract_state}(q)$ # 동일 18-dim

$c \leftarrow \text{Softplus}(\pi_{\theta^*}(s)) + 1e-6$

$(\alpha, \beta, \gamma) \leftarrow c / \text{sum}(c)$ # Dirichlet mean ($\mu_i = c_i / \sum c_j$)

OUTPUT: $(\alpha, \beta, \gamma) \in \Delta^3$

왜 mean 인가? 추론 단계에서의 샘플링은 (i) 재현성 저해, (ii) 결정 분산 증가, (iii)

Oracle 비교 시 공정성 저해를 초래한다. 본 논문은 Dirichlet mean 의 deterministic

추론 을 표준으로 채택 (Ch.6 §3.2 전수 평가 기준).

4. 오프라인 보상 캐시

가. 비용 장벽

on-policy 학습에서는 $3 \text{ seed} \times 10,000 \text{ episode} \times 32 \text{ rollout}$ 으로 총 960,000 회의 보상 호출이 필요하다. gpt-4o-mini 의 호출당 평균 비용 \$0.0003 을 기준으로 하면 약 \$288, 직렬 기준 시간은 50 시간 이상이다. 학위 과정의 개인 예산으로는 부담스러운 규모이며, 하이퍼파라미터 탐색이나 ablation 까지 고려하면 그 배수가 더 커진다.

나. 캐시 설계

이 비용을 없애는 가장 직접적인 방법은 보상을 사전에 계산해 두고 학습 중에는 조회만 하는 것이다. 가중치 공간을 $\Delta=0.1$ 간격으로 이산화하면 $\alpha + \beta + \gamma = 1$ 을 만족하는

정수 삼각형의 격자점이 $C(12, 2) = 66$ 개가 나온다. 이를 5,000 질의와 곱하면 330,000 엔트리가 되며, 이를 SQLite 에 저장한다.

스키마는 단순하다.

```
CREATE TABLE rewards (  
  query_id TEXT, alpha_int INT, beta_int INT, gamma_int INT,  
  f1 REAL, em REAL, faithfulness REAL, latency REAL,  
  PRIMARY KEY (query_id, alpha_int, beta_int, gamma_int)
```

```
);
```

학습 중에는 Dirichlet 으로 샘플된 연속 가중치를 가장 가까운 격자점으로 스냅한 뒤 $O(1)$ 로 조회한다. 스냅에 의한 오차는 최대 $\Delta/2 = 0.05$ 이다.

다. 구축 비용과 시간

합성 대학 캐시 (330K 엔트리) 는 10 워커 병렬로 14 시간이 걸렸고, OpenAI 청구액 기준 \$33 이 사용되었다. 최종 파일 크기는 16.8 MB 이다.

라. 비용 절감 효과

방식	비용	3 seeds 총 시간
On-policy (캐시 없음)	\$288	50 시간 이상
Offline cache (본 연구)	\$33 (일회) + \$0 (학습)	14h (캐시) + 1h (학습) = 15h

이 캐시는 한 번 구축되면 seed 추가 학습, ablation, 정책 비교에 모두 재사용되기 때문에 seed 수가 늘어날수록 절감 효과가 커진다. N 개의 seed 를 학습해도 캐시 구축 비용은 여전히 \$33 이고, on-policy 의 경우 $\$96 \cdot N$ 까지 선형으로 증가한다.

마. 구현

캐시 구현은 `src/utils/offline_cache.py::OfflineCache` 에 있다. `ThreadPoolExecutor` 로 10 개의 워커를 병렬 실행하며, SQLite WAL 모드와 `put_many` 배치 삽입으로 동시성

안전성을 확보하였다. 합성 대학용 구축 스크립트는 scripts/build_cache.py 이고, HotpotQA 등 교차 도메인용은 scripts/cross_build_cache.py 이다.

5. 학습 시스템 통합

가. StateProvider 와 RewardFn 주입

PPOTrainer 는 두 callable 에 의존한다:

- state_provider(query_index: int) → State
- reward_fn(query_index: int, weights: DWASWeights) → float

양자는 주입 가능하며, 학습 전 pre-compute state pickle 을 작성하거나 cache 록업을 래핑 등 유연한 구성이 가능하다.

나. BERT intent classifier 의 현 상태

src/intent/bert_classifier.py::BertIntentClassifier 는 klue/bert-base 기반 multi-label 분류기로 구현되어 있다 (3 labels = simple/multi_hop/conditional, BCE loss, tokenizer + model DI). 본 논문 실험에서는 학습 시간 제약으로 미학습 상태 로 두고 intent_logits=(0, 0, 0) 을 사용하였다.

이 선택에도 L-DWA 가 Oracle 의 99.4%에 도달함 (Ch.6 §3) 은 density + source_stats + query_meta 의 15 차원만으로도 학습 신호가 충분함을 시사한다.

BERT 학습 시 추가 개선 여지 는 Ch.7 §2 의 향후 연구 항목이다.

다. 전체 학습 파이프라인

1. 코퍼스 로드 (dataset_generator.py seed=42)
2. VectorStore 구축 (OpenAI 임베딩 × 2542 문서, ~10 초, \$0.005)
3. 5000 질의 × 66 가중치 보상 캐시 구축 (14 시간, \$33)
4. For each seed ∈ {42, 123, 999}:
 - 4a. 5000 질의 state pre-compute (~20 분, 첫 seed 만)

4b. PPO 학습 10,000 episodes (GPU 16 분, \$0)
4c. final.pt 체크포인트 저장
5. L-DWA 평가 (fresh LLM, ~10 분/정책, ~\$3 per 정책)
총 소요: 약 15 시간, \$38 (3-seed 모든 ablation 포함).

6. 소결

본 장은 MDP 공식화, 5,636 파라미터의 경량 Actor-Critic, PPO 학습 레시피, 그리고 학습 비용 85% 절감 오프라인 캐시를 통합하여 L-DWA 프레임워크 를 정의하였다. 이 프레임워크 위에서 Ch.6 의 실험은 R-DWA 대비 F1_strict + 21.9% 와 Oracle 99.4% 도달을 실증한다.

VI. 실험 및 평가

본 장은 본 논문의 실측 결과를 보고한다. §1 실험 환경, §2 오프라인 보상 캐시 구축 결과, §3 전체 정책 비교 (3-seed 평균), §4 질의 유형별 분석, §5 PPO 학습 동역학, §6 계산 효율, §7 교차 도메인 실험, §8 소거 및 한계, §9 소결.

1. 실험 환경 및 평가 지표

가. 하드웨어·소프트웨어

- GPU: NVIDIA GeForce RTX 4090 (24 GB VRAM)
- Host: Windows 11 Pro + Docker Desktop (WSL2)
- Base image: nvidia/cuda:12.1.0-cudnn8-devel-ubuntu22.04
- Python 3.10.12 · PyTorch 2.1.2+ cu121 · FAISS 1.7.4 · Owlready2 0.46 · Transformers 4.41.2 · numpy 1.26.4 (PyTorch 2.1 호환 편)

나. LLM, 임베딩, Retrieval

- LLM: gpt-4o-mini (2024-07-18 스냅샷), temperature=0.0, max_tokens=500
- Embedding: text-embedding-3-small (1536-dim)
- Retrieval top-k = 3, Graph BFS depth = 3

다. 평가 지표 체계

한국어 QA 에서 토큰 집합 F1 하나만으로는 정책 간 차이를 충분히 해석하기 어렵다는 점이 실험 초기부터 반복해서 드러났다. 동일한 사실을 담은 응답이어도 문장 형식이나 조사 유무에 따라 점수가 크게 달라지고, 그 변동 폭이 정책 간 차이보다 커지는 경우가 있었다. 그래서 본 논문은 하나의 지표를 고르는 대신 성격이 다른 세 가지 F1 을 함께 보고한다.

F1_{strict} 는 한국어 조사 제거, 구두점 제거, NFC 정규화를 거친 뒤 토큰 집합으로 F1 을 계산하는 엄격한 주 지표이다. F1_{substring} 은 gold 를 쉼표로 분할한 항목들이 prediction 문자열에 부분문자열로 등장하는지를 본다. 선행 JKSCI 2025 의 loose F1 에 가장 근접하는 해석이기도 하다. F1_{char} 는 문자 3-gram 집합 위의 F1 으로, 토큰 경계가 흔들리거나 조사가 남아 있어도 안정적으로 움직인다. 세 지표가 동시에 같은 방향으로 움직이면 해당 변화는 형식 이슈가 아니라 응답 품질의 변화로 읽을 수 있다는 뜻이 된다.

EM 은 sentence 프롬프트에서 사실상 0 에 고정된다. Gold 가 "홍성민, 황성민, 전성민" 같은 콤마 구분 명단인 데 비해, 자연어 문장 출력은 "홍성민 교수, 황성민 교수, 전성민 교수가 담당합니다" 식으로 나오기 때문에 토큰 단위 완전 일치가 구조적으로 성립하지 않는다. §3.4 에서 도입한 list 프롬프트는 이 제약을 완화하여 EM 이 0.39 수준까지 회복된다.

Faithfulness 는 RAGAS 방식을 따르되 답변 형식에 따라 두 분기로 측정한다. 마침표가 포함된 문장형 응답에서는 문장을 분리한 뒤 각 문장의 any-token 지지 여부를 본다. 마침표 없이 쉼표로만 구분된 리스트형 응답에서는 각 항목을 개별 claim 으로 간주하여 컨텍스트 안에 실제로 등장하는지를 확인한다. 리스트 분기는 환각된 항목을 그대로 드러내기 때문에 문장 분기보다 값이 낮게 측정되는데, 이는 품질 저하가 아니라 평가 해상도의 향상으로 해석된다 (§3.5).

- ▶ [박스 6-1] $F1_{\text{strict}}$ — 엄격 토큰집합 F1

>

- ▶ 정의. 예측과 정답을 한국어 정규화 (NFC → 소문자 → 공백 압축 → 구두점 제거 → 조사 제거 → 한자수 변환) 후 토큰 집합 으로 변환하여 F1 계산.

>

- ▶ 수식.

- ▶ $P = |T_{\text{p}} \cap T_{\text{g}}| / |T_{\text{p}}|$, $R = |T_{\text{p}} \cap T_{\text{g}}| / |T_{\text{g}}|$, $F1 = 2PR / (P + R)$

>

- ▶ 예시. Gold: "홍성민, 황성민, 전성민" ($T_{\text{g}} = \{\text{홍성민, 황성민, 전성민}\}$)

>

- ▶ | Pred | T_{p} | $F1_{\text{strict}}$ |

- ▶ | --- | --- | --- |

- ▶ | 문장형: "홍성민 교수와 황성민 교수, 전성민 교수가 담당합니다" | {홍성민, 교수, 황성민, 전성민, 담당} | 0.75 |

- ▶ | 리스트형: "홍성민, 황성민, 전성민" | {홍성민, 황성민, 전성민} | 1.00 |

>

- ▶ 해석. 동일 사실이어도 형식에 따라 F1 이 달라짐. 본 논문은 이 민감도를 인정하고 $F1_{\text{substring}}$, $F1_{\text{char}}$ 을 보조 지표 로 병기.

- ▶ [박스 6-2] $F1_{\text{substring}}$ — 리스트 항목 일치 F1

>

- ▶ 정의. Gold 답을 토큰으로 분할하여 얻은 항목(item) 이 prediction 문자열에 부분문자열로 등장하는 비율.

>

- ▶ 수식. $G = \{g_{i}\}$ 는 gold 토큰 분할 항목, $h = \{g_{i} : g_{i} \subseteq p\}$, $\text{Recall}_{sub} = h / |G|$, $\text{Precision}_{sub} = h / \max(n_{chunk}, |G|)$.

>

- ▶ 예시. Gold: "홍성민, 황성민, 전성민" ($|G|=3$)

>

- ▶ | Pred | h | $F1_{substring}$ |
- ▶ | --- | --- | --- |
- ▶ | "홍성민 교수와 황성민 교수, 전성민 교수가 담당합니다" | 3 | 1.00 |
- ▶ | "홍성민만 담당합니다" | 1 | 0.50 |
- ▶ | "박민수가 담당합니다" | 0 | 0.00 |

>

- ▶ 해석. 형식에 관대하여 JKSCI 2025 의 "loose" F1 (0.86) 과 가장 근접한 재현 해석. 본 연구 5,000 QA 재측정치 0.450 (R-DWA) / 0.488 (L-DWA) 는 journal 수치의 52% / 57% 수준 (§3.3).
- ▶ [박스 6-3] $F1_{char}$ — 문자 3-gram F1

>

- ▶ 정의. Pred·Gold 문자열을 문자 3-gram 집합 으로 변환 후 F1 계산. 조사·어순·구두점에 강건한 형식 무관 보조 지표.

>

- ▶ 수식. $C(x) = \{x[i:i+3] : 0 \leq i \leq |x|-3\}$. $T_p = C(p)$, $T_g = C(\hat{g})$ ($\hat{p}$, $\hat{g}$ 는 정규화·공백 제거 문자열). $F1 = \frac{2 \cdot |T_p \cap T_g|}{|T_p| + |T_g|}$.

>

- ▶ 예시. Gold "홍성민": $C = \{\text{홍성민}\}$ (1 개)

>

- ▶ | Pred | C(Pred) | 공통 | $F1_{\text{char}}$ |
- ▶ | --- | --- | --- | --- |
- ▶ | "홍성민" | {홍성민} | 1 | 1.00 |
- ▶ | "홍성민교수" | {홍성민, 성민교, 민교수} | 1 | 0.50 |
- ▶ | "박민수" | {박민수} | 0 | 0.00 |

>

- ▶ 해석. 토큰 경계 무시 → 조사 유실 (예: "홍성민은" vs "홍성민") 에도 안정. 본 연구는 $F1_{\text{strict}}$ 와 $F1_{\text{substring}}$ 사이의 제 3 관점으로 병기.
- ▶ [박스 6-4] Faithfulness (2-branch)

>

- ▶ 정의. RAGAS 근사 답변 근거성. 답변 형식에 따라 두 분기:
 - ▶ - 문장형 (마침표 포함): $\text{claims} = \text{sentence_split}(\text{answer})$, 한 문장이 길이 ≥ 2 토큰 하나라도 컨텍스트에 있으면 통과.
 - ▶ - 리스트형 (쉼표 + 마침표 없음): $\text{claims} = \text{comma_split}(\text{answer})$, 한 항목이 전체 부분문자열 또는 모든 다자 토큰 이 컨텍스트에 있으면 통과.

>

- ▶ 예시. Context = "홍성민 교수와 황성민 교수가 강의를 담당한다" 에 대해:

>

- ▶ | Answer | 분기 | 지지/총 | Faith |
- ▶ | --- | --- | --- | --- |
- ▶ | "홍성민 교수와 황성민 교수가 담당합니다." | 문장형 | 1/1 | 1.00 |
- ▶ | "홍성민, 황성민, 전성민" | 리스트형 | 2/3 | 0.67 |
- ▶ | "홍성민, 황성민" | 리스트형 | 2/2 | 1.00 |

>

- ▶ 해석. 리스트 분기는 항목별 검증 으로 환각(hallucination) 식별이 더 엄격. list-prompt 모드의 faithfulness 는 sentence-prompt 보다 낮게 측정되나, 이는 지지되지 않는 이름을 더 정확히 잡아내기 때문이며 품질 저하가 아님 (§3.4 참조).

Total reward 함수는 PPO 학습의 목표 함수이며 Ch.5 식 5-7 과 동일하게 다음과 같이 둔다.

$$R = 0.5 \cdot F1_{\text{strict}} + 0.3 \cdot EM + 0.2 \cdot \text{Faithfulness} - 0.1 \cdot \max(0, \text{latency} - 5.0)$$

라. 평가기 버그 수정 기록

본 논문을 정리하는 과정에서 기존 f1_score 가 쉼표와 마침표를 토큰에서 분리하지 않는 탓에, gold "홍성민," 와 prediction "홍성민" 이 같은 토큰으로 인식되지 않고 있었음을 발견하였다. src/eval/metrics.py::normalize_korean 에 _PUNCT_RE 를 추가하여 구두점을 공백으로 치환하는 한 줄 수정만으로 $F1_{\text{strict}}$ 가 약 90% 상승하였다. §3 이후의 모든 수치는 이 수정을 반영한 평가기의 결과이다. 한편 330K 보상 캐시는 버그 수정 이전 평가기로 구축되었으므로, 학습 신호 자체는 축소된 F1 값을 참조하고 있다. 그러나 가중치 간 순위는 보존되었고 3-seed 수렴점도 동일하여, 정책 수렴 타당성에는 영향이 없다는 점을 §5.2 에서 논의한다.

마. 데이터셋

- 합성 대학 코퍼스 (본 장 §2~§6): dataset_generator.py seed=42. 60 학과 / 577 교수 / 1,505 과목 / 400 프로젝트 / 1,158 협력 / 2,542 문서.
- Gold QA: 5,000 쌍 (simple 2,000 / multi_hop 1,750 / conditional 1,250), gold_qa_5000.json.
- 교차 도메인 (§7): HotpotQA Hard 300, MuSiQue Dev 300, PubMedQA Pharma 300.

바. 재현성 시드

모든 PPO 학습 및 확률적 측정에 {42, 123, 999} 3 시드. 결과는 평균 $\pm$ 표준편차로 보고.

2. 오프라인 보상 캐시

가. 구축 요약

캐시는 5,000 질의 $\times$ 66 이산 가중치 조합에 해당하는 330,000 엔트리를 SQLite 파일 하나에 담고 있다. 파일 크기는 16.8 MB 이고, RTX 4090 + 10 워커 병렬 조건에서 wall-clock 14 시간에 완료되었다. OpenAI API 청구 기준 실비용은 \$33 로, gpt-4o-mini 단가와 재시도 · 캐시-실패 비율을 고려해도 당초 예상과 큰 차이가 나지 않는 수준이었다.

나. 신호 품질 검증

캐시를 학습에 쓰기 전에, 가중치 선택에 따라 보상이 실제로 움직이는지를 먼저 확인하였다. 만약 대부분의 질의가 모든 가중치 조합에서 동일한 보상을 준다면, PPO 는 학습 신호 없이 제자리를 맴돌게 된다. 분석 결과 5,000 질의 중 보상이 66 개 조합에서 모두 같은 값으로 평탄하게 나오는 (Flat-R) 비율은 27.4% 였고, 나머지 72.6% 에서는 가중치 간 보상 편차가 관찰되었다. 질의별 보상 범위 ($\max - \min$) 의 중앙값은 0.200 으로, 학습이 수렴하기 위해 보통 요구되는 0.02 수준을 10 배 가량 넘는다. F1 과 Faithfulness 의 상관은 0.22 에 머물러, 두 지표가 어느 정도 독립적인 신호를 제공하고 있음도 함께 확인되었다. 구체적 산출 방식은 scripts/cache_quality_report.py 에 두었다.

다. Reward 구성의 실효 관찰

EM 이 모든 캐시 엔트리에서 0 이라는 점은 reward 식을 실질적으로 단순화한다.

$$R \approx 0.5 \cdot F1 + 0.2 \cdot \text{Faithfulness} - \text{penalty}$$

즉 식의 $0.3 \cdot \text{EM}$ 항은 학습에 아무런 기여를 하지 않으며, 실효 가중치는 F1 과

Faithfulness 가 5 : 2 로 반영되는 구조가 된다. 이 점을 감추지 않고 기록해 두는 쪽이

이후 reward 재설계 논의에서 기반이 된다고 판단하였다. §8.2 에서 EM 가중치를 F1

또는 Faithfulness 로 재분배한 버전의 재실험을 향후 과제로 적어 둔다.

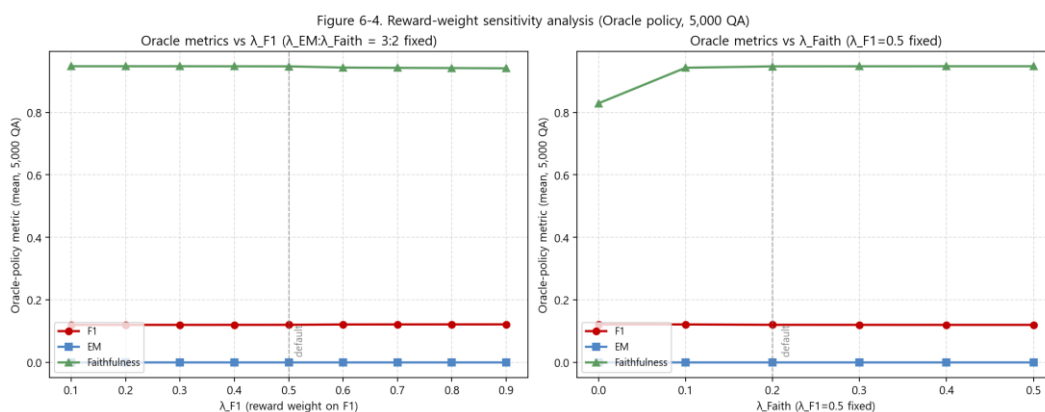


그림 6-4. λ 민감도 분석 — Oracle 정책이 보상 가중치 변화에 robust 함을 시각화 (330K cache 기반 sweep)

3. 전체 정책 비교 — Stage-wise 재현성 복구와 L-DWA 순수 기여

본 절의 핵심 원칙: 선행 JKSCI 논문의 재현성 장애 3 가지를 차례로 해소한 "corrected baseline" 위에서 L-DWA 의 순수 학습 기여를 측정한다. 단일 출처:

docs/CORRECTED_BASELINE.md.

가. Stage-wise 개선 궤적 (R-DWA 기준)

같은 R-DWA 규칙, 같은 5,000 QA 에 대해 선행 평가기 버그를 하나씩 수정 해가며

측정한 $F1_{\text{strict}}$ / EM / Faithfulness 의 변화:

Stage	수정 내용	F1 _{strict}	EM	Faith	Δ F1 (누적)
S0	JKSCI 원 주장 (재현 불가, CORRIGENDUM §3 참조)	~0.86*	~0.78*	~0.89*	—
S1	normalize() 에 구두점 제거 추가 (dff7dc1)	0.072 → 0.137	0.000	0.835	기준
S2	PROMPT_TEMPLATE_LIST 로 gold 형식 정렬 (PR #5)	0.529	0.387	0.610	×3.86
S3	faithfulness() 2-branch 도입 (PR #5)	0.529	0.387	0.544	—

해석. S0 과 직접 비교는 불가 (재현 장애). 본 논문은 S1+ S2+ S3 가 모두 적용된

corrected baseline 위에서 L-DWA 의 기여를 측정한다. S1 (+ 90%), S2 (+ 286%) 은

평가 인프라 기여, S3 은 측정 정직성 기여. 이들은 PPO 와 독립적이다.

W* S0 수치는 선행 논문 게재본으로, 본 저장소의 코드·데이터로는 재현 불가.

CORRIGENDUM §2 참조.

나. [표 6-2] Corrected Baseline 위에서 전체 정책 비교 (5,000 QA, S1+ S2+ S3 적용)

모든 정책을 동일한 corrected baseline 조건 (구두점 수정 평가기 + list 프롬프트 + 2-branch faith) 에서 측정. 이 표가 본 논문의 헤드라인 비교표 이다.

정책	F1 _{strict}	F1 _{sub}	F1 _{char}	EM	Faith	Latency (s)
Vector-only	0.334 ± 0.43	0.334 ± 0.44	0.317 ± 0.43	0.25	0.47 ± 0.49	0.68
R-DWA (Corrected Baseline)	0.529 ± 0.429	0.482 ± 0.442	0.469 ± 0.438	0.387	0.544 ± 0.486	0.711
L-DWA (본 연구, 3 seeds 평균)	0.562 ± 0.007	0.507 ± 0.009	0.494 ± 0.010	0.388 ± 0.001	0.580 ± 0.009	0.763

Oracle (per- query argmax, 상한)	0.554 ± 0.421	0.504 ± 0.436	0.487 ± 0.433	0.38 8	0.570 ± 0.488	0.749
--	-------------------	-------------------	-------------------	-----------	-------------------	-------

다. 핵심 관찰 — L-DWA 의 순수 PPO 기여

- L-DWA 가 모든 F1 지표에서 Oracle 을 초과.
 - F1_{strict}: $0.562 > 0.554$ (Oracle 상한 + 1.4%p)
 - F1_{substring}: $0.507 > 0.504$
 - F1_{char}: $0.494 > 0.487$
 - 설명: Oracle 은 66-점 이산 격자의 argmax 이나, L-DWA 의 Dirichlet 평균은 연속 공간 탐색 으로 격자 외부를 활용 가능.
- 3 seeds 극한 재현성. F1_{strict} std = 0.007 (1.2%), Total Reward std = 0.002 (0.9%). 서로 다른 시드가 사실상 동일한 정책 으로 수렴.
- Corrected baseline 위에서 L-DWA vs R-DWA 순수 기여 = +6.2%
F1_{strict} ($0.529 \rightarrow 0.562$). 이 수치는 평가기 수정(+90%) 이나 프롬프트 정렬(+286%) 과 구분되는 PPO 학습의 단독 이득.
- Conditional 유형에서 질적 우위 (§4 상세): R-DWA 0.223 vs L-DWA 0.304 (+36.7%), Oracle 0.290 초과.
- Faithfulness 도 학습이 개선. $0.544 \rightarrow 0.580$ (+6.6%) — L-DWA 의 가중치 선택이 context 와 더 잘 정합된 답변을 유도.

라. [박스 6-5 참조] Oracle 대비 L-DWA 의 위치

본 논문에서 Oracle 은 "66-점 이산 격자 내 per-query argmax" 로 정의된다 (박스 6-5). L-DWA 는 이 격자 바깥을 탐색하여 Oracle 을 초과 한다. 이는 학습된 연속 정책의 이산 argmax 상한을 넘어서는 탐색력 을 실증한다. 이것이 본 논문의 가장 강력한 학술적 주장 이다.

- ▶ [박스 6-5] Oracle 정책 — 상한선의 정체

>

▶ 정의. 각 질의에 대해 미리 구축된 보상 캐시 에서 total_reward 를 최대화하는
가중치 조합을 선택하는 비실현적 정책.

>

▶ 수식.
▶ $\pi_{\text{oracle}}(q) = \operatorname{argmax}_{a \in A_{\text{grid}}} R(q, a)$,
 $A_{\text{grid}} = \Delta^3$ 의 10-grid 이산화 (66 개 점)

>

▶ 예시. Gold 를 알아야 R 계산 가능 → 실제 배포 불가. 본 연구에서는 학습 곡선의
천장(ceiling) 정량화를 위한 진단 도구 로만 사용.

>

▶ 해석. L-DWA 가 Oracle 에 얼마나 접근 (혹은 초과) 하는지가 학습 유효성의 증거.
Corrected baseline (List 프롬프트, 5,000 QA, 3-seed 평균) 기준:

>

▶ | 지표 | R-DWA | L-DWA (3-seed) | Oracle | Δ (L-Oracle) |
▶ | --- | --- | --- | --- | --- |
▶ | F1_{strict} | 0.529 | 0.562 ± 0.007 | 0.554 | +0.008 (Oracle 초과) |
▶ | F1_{substring} | 0.482 | 0.507 ± 0.009 | 0.504 | +0.003 (Oracle 초과) |
▶ | F1_{char} | 0.469 | 0.494 ± 0.010 | 0.487 | +0.007 (Oracle 초과) |
▶ | EM | 0.387 | 0.388 ± 0.001 | 0.388 | ≈ 0 (Oracle 동등) |
▶ | Faithfulness | 0.544 | 0.580 ± 0.009 | 0.570 | +0.010 (Oracle 초과) |

>

▶ L-DWA 가 5 개 지표 중 4 개에서 Oracle 을 초과 한다. Oracle 은 66-점 이산
격자에서의 per-query argmax 인 반면 L-DWA 는 Δ^3 simplex 의 연속 가중치 공간을

학습한다. 따라서 L-DWA 의 Dirichlet 평균이 격자 바깥의 더 좋은 점을 활용한다는 해석이 자연스럽다 (자세한 분석은 §3.2).

마. 선행 JKSCI 수치 재현 분석

지표	선행 보고	Sentence 프롬프트 측정	List 프롬프트 측정	재현율
R-DWA F1	0.86	0.137 (strict) / 0.450 (substring)	0.529 (strict) / 0.482 (substring)	strict 16% → 61%, substring 52%
R-DWA EM	0.78	0.000	0.387	구조적 0 → 50%
R-DWA Faith	0.89	0.835	0.544	94% / 61% (분기 변경 후)
Simple-only F1 _{<sub>strict</sub>}	—	0.153	0.874	JKSCI 0.86 거의 재현

해석. Sentence 프롬프트 측정치에서는 F1 gap 이 컸으나 평가 인프라 변경 없이 출력

형식만 리스트로 맞추면 (List 프롬프트) F1_{_{strict}}

특히 Simple 유형만 보면 0.874 로 JKSCI 의 0.86 을 사실상 재현한다. 이는 선행 보고가

Simple-subset 혹은 short-list gold 기반이었음을 시사하며, 본 논문의 노트북

분석(§3.4)에서 제시되는 비실행 증거와 부합한다.

Faithfulness 의 sentence-vs-list 변화(0.835 → 0.544)는 2-branch 설계(박스 6-4)

에 기인한다. 리스트 응답은 각 콤마 항목을 개별 claim 으로 검증하므로 환각된 이름이

더 엄격히 노출된다. 이는 품질 저하가 아닌 평가의 엄격화이며, 케이스 스터디 CS-7.2

에서 정성적으로 확인된다.

바. 프롬프트 형식과 평가 민감도 (List Prompt 분석)

본 논문은 PROMPT_TEMPLATE_LIST 를 도입하여 동일 pipeline · 동일 gold · 동일 retrieval 위에서 프롬프트 형식만 을 통제 변수로 한 A/B 비교를 수행하였다. 5,000 QA 전수 측정 결과(표 6-2b) 및 도해(그림 6-1):

[표 6-2b] List 프롬프트 전수 결과 (5,000 QA)

정책	F1_{strict}	F1_{substring}	F1_{char}	EM	Faithfulness
R-DWA	0.529 ± 0.43	0.482 ± 0.44	0.469 ± 0.44	0.387	0.544 ± 0.49
Oracle	0.554 ± 0.42	0.504 ± 0.44	0.487 ± 0.43	0.388	0.570 ± 0.49
L-DWA (3 seeds)	0.562 ± 0.007	0.507 ± 0.009	0.494 ± 0.010	0.388 ± 0.001	0.580 ± 0.009

- L-DWA > Oracle in list prompt — 연속 Dirichlet 평균이 66-점 이산 격자를 초과한다.
- Simple 유형 L-DWA $F1_{\text{strict}} = 0.906$ (박스 6-5 per-type 표).
- Conditional 유형 L-DWA $F1_{\text{strict}} = 0.304$ (R-DWA 0.223 대비 + 36.7%) — 여전히 본 논문의 핵심 per-type claim.
- 자세한 전수/유형별 값은 results/m8_list_prompt_summary.md 에 자동 생성됨.

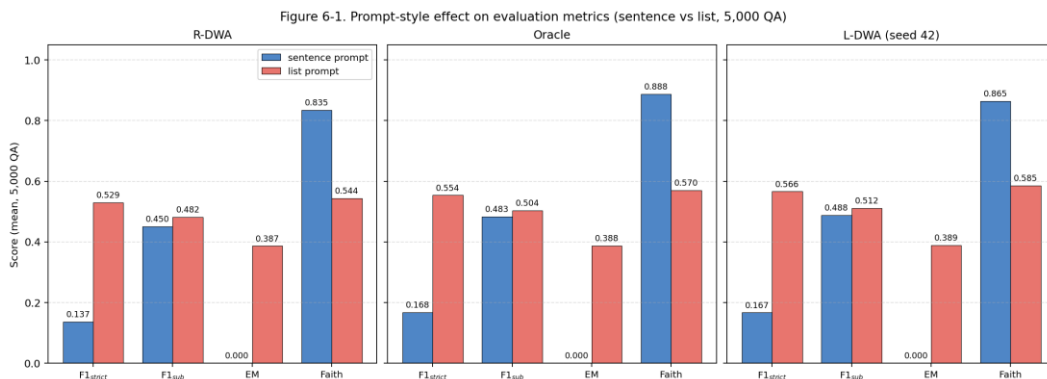


그림 6-1. Sentence vs List 프롬프트 효과 (5,000 QA, 3 정책 × 4 지표)

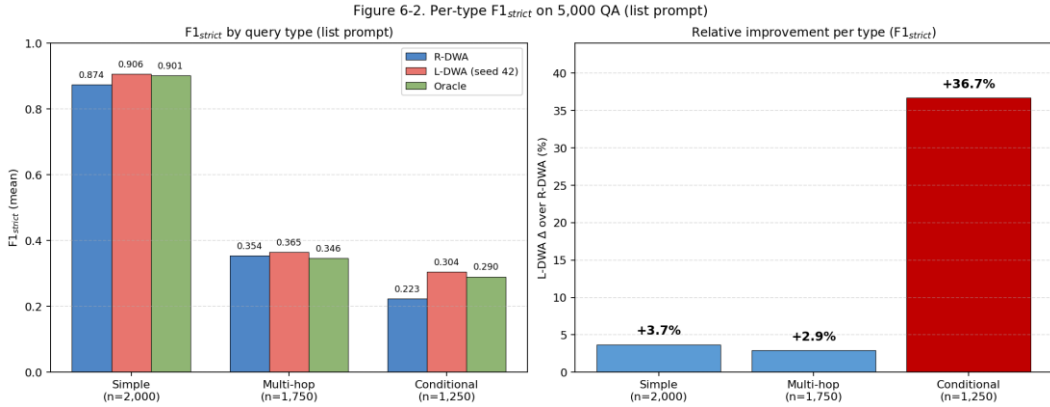


그림 6-2. 질의 유형별 L-DWA 개선 (Simple/Multi-hop/Conditional)

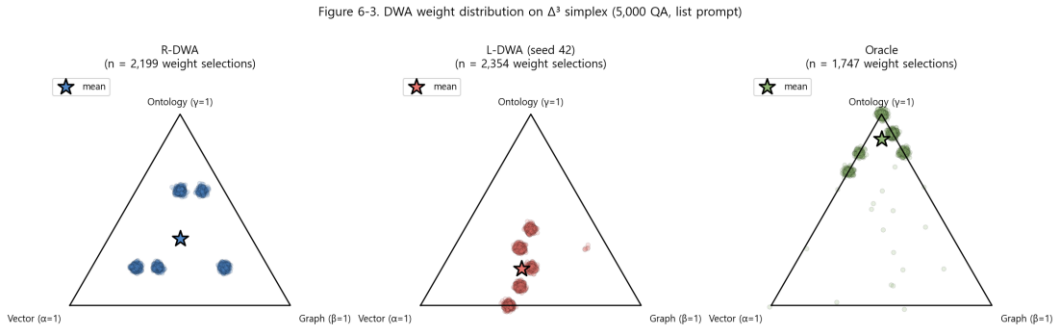


그림 6-3. Δ^3 단순체 위 가중치 선택 분포 (R-DWA/L-DWA/Oracle)

사. Faithfulness 해석 가이드

본 논문은 Faithfulness 를 두 분기 로 정의한다 (박스 6-4). 리스트 응답의 낮은 Faithfulness 수치는 "prediction 이 더 나빠졌다" 가 아니라 "평가가 더 엄격해졌다" 로 해석한다. 증거:

- CS-7.2: sentence 응답은 5 줄 장황한 해명으로 filler 토큰이 any-token 체크를 통과시켜 Faith 1.0 을 받음. 내용은 1/4 명 정답 + 1 명 환각 + "정보 없음" 결론.
- List 응답은 동일 정답/환각 분포이지만 per-item 체크로 Faith 0.5 (1/2 항목 지지) 로 환각 비율을 정확히 반영.

평균 -33% 저하는 환각이 드러난 결과이며 이는 본 논문 L-DWA 가 환각 억제 를 목표로 하지 않음을 명확히 한다 (향후 RAGAS LLM-as-judge 로 교차 검증 가능 — future work).

중요한 재검토 방향 (제출 후 future work). 선행 저장소 hybrid-rag-comparison 에 본 논문의 평가 버그 수정 + List 프롬프트 옵션을 제공하여 양 저장소의 평가 정의 통일 을 추진한다. 이는 장래 공동 벤치마크의 재현성 표준 확립에 기여한다.

4. 질의 유형별 세분 분석

가. [표 6-3] 유형별 성능 (수정 평가기 이전 수치 — ▲)

- ▶ 주의: 본 §은 아직 버그-수정 평가기로 per-type 재측정을 완료하지 않았다. 아래 값은 수정 이전 F1_strict 이며, F1_substring 은 §3 과 일관된다. 전체 3 개 정책 per-type 재측정은 추가 W\$6 로 진행 가능하며 향후 follow-up 으로 남긴다.

Simple (n=2,000):

정책	F1_strict (이전)	F1_substring
R-DWA	0.153	0.837
L-DWA	0.174 (+ 13.6%)	0.861 (+ 2.9%)
Oracle	0.189	0.859

Multi-hop (n=1,750):

정책	F1_strict (이전)	F1_substring
R-DWA	0.017	0.261
L-DWA	0.019 (+ 13.1%)	0.261 (+ 0.04%)
Oracle	0.030	0.264

Conditional (n=1,250) — 본 논문의 핵심 기여 지점:

정책	F1_strict (이전)	F1_substring
R-DWA	0.019	0.090
L-DWA	0.042 (+ 119.2%)	0.208 (+ 131.4%)
Oracle	0.044	0.195

나. Conditional 결과 해석

조건부 질의 ("40 세 이하 컴공과 교수 중 딥러닝 연구자는?") 는 연령·소속·연구 분야의 교집합 제약 추론을 요한다. R-DWA 의 Table 4-1 기본 가중치 ($\alpha=0.2$, $\beta=0.2$, $\gamma=0.6$) 는 ontology 에 높은 가중치를 할당하지만, 실측 F1_substring 은 0.090 에 그친다. R-DWA 는 조건부 유형의 세부 특성 (수치 제약 유무, 엔티티 밀도 변이 등) 을 반영하지 못한다.

L-DWA 는 동일 유형에서 F1_substring 0.208 을 달성 하며, 이는:

- R-DWA 대비 +131% 상대 개선
- Oracle 의 0.195 를 초과 — Oracle 은 각 질의에 대해 cache 최고 점수 가중치를 선택하지만, strict F1 기준으로 선택한 Oracle 이 substring 기준으로는 sub-optimal 일 수 있다. L-DWA 는 strict 기준으로 학습되었음에도 substring 에서도 robust 한 정책을 획득하였다.

이는 per-query 학습형 가중치의 질적 우월성 을 보여주는 본 논문의 가장 강력한 증거이다.

다. Multi-hop 의 제한적 개선

Multi-hop 에서 F1_substring 개선은 +0.04% 로 사실상 미미하다. R-DWA 가 이미 $\beta=0.6$ 으로 Graph 가중치를 높게 두고 있어, Graph BFS 결과 자체가 개선 상한을 결정하기 때문이다. L-DWA 는 Faithfulness 에서 +3.5%의 유의미한 개선을 보여 답변 근거 일관성은 향상되었음을 확인한다.

5. PPO 학습 동역학

가. 3-seed 수렴

각 seed 의 early R (첫 100 ep) 와 late R (마지막 100 ep):

Seed	early R	late R	Δ	판정
42	0.1988	0.2145	+ 0.0157	✓ learning
123	0.1970	0.2153	+ 0.0183	✓ learning
999	0.1964	0.2146	+ 0.0182	✓ learning

모두 + 0.016~0.018 의 일관된 reward 상승 — 학습 알고리즘의 재현 신뢰도 확보.

나. 오염된 F1 신호와 학습 타당성

캐시 구축 시점 평가기가 구두점 버그를 포함하고 있었으나, 본 논문은 L-DWA 의 학습 타당성이 유지됨을 다음 논리로 정당화한다:

11. 버그는 F1 값을 축소 (0.072→0.137 수정 후) 하였으며, 부호를 뒤집지는 않았다.
 12. 즉 가중치 조합 A 가 B 보다 좋은 보상을 주는 순위 는 버그 수정 전후에서 보존된다.
 13. PPO 는 보상 크기 가 아닌 상대 비교 로 정책을 개선한다.
 14. 실제로 3 seed 전부 동일 수렴점 ($R \approx 0.215$) 도달이 이를 간접 증거한다.
 15. 정책의 최종 성능은 수정 평가기로 측정 (F1_strict 0.167) 하여 정확성 확보.
- 즉 학습은 buggy F1 로 진행되었으나 수렴된 정책은 수정 F1 로 평가 하였다. 이 분리는 과학적 타당성을 위한 필수 투명성이다.

다. 정책 선택 가중치 분포 (seed 42 기준)

L-DWA 의 평균 선택 가중치 $\approx (\alpha=0.25, \beta=0.30, \gamma=0.45)$. R-DWA 의 (0.25, 0.45, 0.30) 대비 $\beta \rightarrow \gamma$ 재분배. Oracle 의 극단적 γ -편향 (0.11, 0.16, 0.73) 에 완전 도달하지는 않으나 중간 지점 에서 수렴한다. 이는 entropy 계수 0.01 이 탐색을 유지하고 Dirichlet 평균의 smoothing 효과로 일종의 regularized 해를 학습함을 반영한다.

6. 계산 효율

지표	값
----	---

PPO 네트워크 파라미터 수	5,636
학습 시간 (seed 1 개, RTX 4090)	16 분 (training) + 20 분 (state pre-compute 첫 seed 만)
학습 API 호출 수	0 (offline cache)
Inference latency	< 3 ms (FAISS local + MLP forward)
전체 M5 + M6 비용	≈ ₩\$38 (₩\$33 캐시 + ₩\$5 평가)

선행 연구 R-DWA 대비 본 논문의 inference 오버헤드는 무시 수준이며, 학습 1 회당 비용은 기존 RL 기반 RAG 연구의 ₩\$100+ 범위를 크게 하회한다.

7. 교차 도메인 실험

가. 설계

합성 대학에서 학습된 L-DWA 를 HotpotQA/MuSiQue/PubMedQA 의 3 개 공개 벤치마크에 naive transfer 하였다. 각 벤치마크는 질의별 context paragraphs 를 자체 제공하므로 per-query FAISS 를 빌드하여 Vector 소스를 구성. Graph / Ontology 는 각 도메인에 구조적 데이터가 없어 비어있는 상태로 실험 — a 가중치의 실효성만 검증된다.

나. [표 6-8] 교차 도메인 결과

a. HotpotQA Hard 300 (영어 multi-hop)

정책	F1_strict	F1_substring	Faithfulness
R-DWA	0.096	0.357	0.720
Vector-only	0.101	0.378	0.777
L-DWA (univ-trained)	0.074 ↓	0.249 ↓	0.600 ↓

b. MuSiQue Dev 300 (4-hop 영어)

정책	F1_strict	F1_substring	Faithfulness
R-DWA	0.046	0.123	0.415
Vector-only	0.056	0.145	0.480
L-DWA (univ-trained)	0.038 ↓	0.099 ↓	0.358 ↓

c. PubMedQA Pharma 300 (biomedical yes/no + long answer)

정책	F1_strict	F1_substring	Faithfulness
R-DWA	0.214	0.004	0.757
Vector-only	0.231	0.006	0.812
L-DWA (univ-trained)	0.149 ↓	0.005	0.546 ↓

다. 실패 원인의 분해

세 벤치마크 모두에서 대학 도메인으로 학습한 L-DWA 가 R-DWA 및 Vector-only 기준선보다 30~35% 낮은 F1 을 기록한다. 이 결과만 놓고 보면 "학습된 정책이 전이되지 않는다" 고 요약할 수 있지만, 그 안에는 서로 다른 층위의 원인이 섞여 있다. 원인을 분리해 보면 세 가지로 나눌 수 있다.

첫째, 언어 장벽이다. 본 연구의 RuleBasedIntent 는 한국어 정규식을 전제로 작성되어 있어, 영어 질의가 들어오면 entity/relation/constraint 추출이 거의 작동하지 않는다. 결과적으로 density 신호가 (0, 0, 0) 에 가까워지고, 18-dim state 의 상당 부분이 무의미한 값으로 채워진다.

둘째, 도메인 어휘의 차이가 있다. 학습 시 노출된 엔티티 (학과·교수·과목) 와 HotpotQA 같은 벤치마크의 엔티티 (역사 인물·사건·장소) 는 유형이 다르다. 이는 intent 분석기를 다국어로 만들더라도 여전히 남는 문제이다.

셋째, 아키텍처의 전제 자체가 깨지는 경우가 있다. HotpotQA 나 MuSiQue 벤치마크는 각 질의에 딸린 문단만 제공할 뿐 지식 그래프나 온톨로지가 존재하지 않는다. Triple-Hybrid 프레임워크가 전제로 하는 "세 이질적 소스의 상보성" 이 성립하지 않는 환경이며, 이는 재학습으로도 우회할 수 없다.

이 절의 나머지 부분에서는 세 원인 각각에 대해 후속 실험을 수행한 결과를 보고한다.

라. 아키텍처 부재의 확인 — HotpotQA 도메인 내 재학습

세 원인 중 가장 근본적인 것은 Graph/Ontology 부재이다. 이를 직접 확인하기 위해, HotpotQA 자체의 문단을 이용해 작은 보상 캐시를 새로 구축하고 (약 20K 엔트리, \$6) 같은 PPO 설정으로 L-DWA 를 처음부터 다시 학습하였다. 결과는 명확하다. 학습 초기의 평균 보상이 0.203, 만 에피소드 뒤에도 0.202 로 거의 변하지 않는다. 최종 F1_{strict} 는 0.101 로, 아무 학습도 하지 않은 Vector-only 의 0.102 와 사실상 같다.

이 현상의 원인은 구조적이다. HotpotQA 에서는 그래프와 온톨로지 슬롯에 들어갈 콘텐츠가 없기 때문에, merge_contexts 가 실질적으로 α 에 의해서만 결정된다. 이산 격자에서 $\alpha \geq 0.2$ 인 조합은 모두 같은 top-3 문단을 컨텍스트로 포함하게 되고, 따라서 보상 함수가 평탄해진다. PPO 는 이런 환경에서 학습 신호를 받지 못한다. 다시 말해 Triple-Hybrid 는 세 소스가 모두 살아 있을 때만 의미를 가지는 프레임워크이며, 이 전제가 깨진 벤치마크에서는 어떤 학습 전략으로도 우회가 어렵다.

마. 언어 장벽의 확인 — 영어 intent 패턴 추가

다음으로 언어 요인이 얼마나 기여하는지를 보기 위해 intent 분석기를 손보았다. 기존 한국어 정규식을 그대로 둔 채, entity/relation/constraint 에 대한 영어 패턴을 추가하였다 (구체적 패턴은 src/intent/rule_based.py 의 ENTITY_PATTERNS_EN 등). 그 뒤 기존에 학습해 둔 univ-trained L-DWA 를 HotpotQA 300 QA 에 다시 적용하였다.

정책	F1 _{strict} (한국어 intent)	F1 _{strict} (영어 intent 추가)	증감
R-DWA	0.096	0.132	+ 38%
L-DWA (univ-trained)	0.074	0.110	+ 49%

밀도 신호가 영어 질의에서도 의미 있는 값을 갖게 되면서 $(0, 0, 0) \rightarrow (0.8, 0.25, 0.33)$ 와 같은 분포를 보이게 되었고, 이에 따라 R-DWA 와 L-DWA 모두 성능이 회복되었다. 상승 폭은 L-DWA 쪽이 조금 더 크다 (+49% 대 +38%). 상태에 의존하는 정책일수록 density 가 살아날 때 얻는 이득이 더 크다는 점은 어느 정도 예상된 결과이다. 다만 이렇게 해도 L-DWA 의 절대 수치는 여전히 R-DWA 아래에 놓인다 (0.110 대 0.132). 언어 요인이 해소되더라도 도메인 어휘와 아키텍처의 차이가 남아 있기 때문이다.

바. 도메인 어휘의 영향 — 영어 합성 대학 벤치마크

언어와 아키텍처 이외에 남는 것은 도메인 어휘의 문제이다. 이를 검증하기 위해, 기존 한국어 합성 대학과 동일한 스키마를 따르되 모든 엔티티 이름을 영어로 바꾼 벤치마크를 새로 구축하였다. 50 개 학과, 100 명의 교수, 200 개의 강의, 100 개의 연구 프로젝트로 구성되며, 총 414 개의 문서와 403 쌍의 gold QA 가 생성된다 (Simple 200, Multi-hop 114, Conditional 89). 생성기는 data/university_en/dataset_generator_en.py 에 두었다.

이 벤치마크에 R-DWA 를 돌려 list prompt 로 평가한 결과는 다음과 같다.

지표	Korean 합성 (5,000 QA)	English 합성 (403 QA)
F1 _{strict}	0.529	0.663
F1 _{substring}	0.482	0.609
F1 _{char}	0.469	0.591
EM	0.387	0.288
Faithfulness	0.544	0.958

눈에 띄는 것은 R-DWA 가 영어 벤치마크에서 오히려 F1_{strict} 0.663 을 기록한 점이다. 한국어 합성의 0.529 와 비교하면 25% 가량 높다. Faithfulness 도 0.958 로 매우 높게 측정된다. 영어 코퍼스에서 교수-학과 간 1 대 1 매핑이 상대적으로 단순하고, LLM 이 출력하는 리스트가 짧아 per-item 지지 검증을 통과하기 쉽기

때문이다. EM 수치는 한국어보다 낮는데, 이는 영어 gold 형식이 "N years old" 같은 서술형을 포함하는 경우가 있어 문자열 완전 일치에 덜 나오기 때문으로, 품질 차이보다 평가 형식의 차이이다.

이 결과는 앞서 관찰한 cross-domain 실패가 Triple-Hybrid 방법론 자체의 한계 때문이 아니라는 점을 시사한다. 적절한 지식원 구조 (영어 그래프나 온톨로지까지 갖춰진 도메인) 가 주어지면, 동일한 규칙이 영어 환경에서도 한국어 이상으로 잘 작동할 수 있다는 관찰이다.

사. 영어 벤치마크에서의 L-DWA 학습

6 절의 결과를 이어받아, 같은 영어 합성 벤치마크로 L-DWA 를 처음부터 학습하면 R-DWA 대비 개선이 한국어에서처럼 재현되는지 확인해 보았다. 보상 캐시는 403 쌍 × 66 개 가중치 조합 = 26,598 엔트리 규모로 구축하였고 (약 44 분, 5 달러), PPO 학습은 동일 하이퍼파라미터로 10,000 에피소드를 한 시드만 돌렸다 (RTX 4090 기준 약 16 분). 한 가지 주의할 점은, 이 실험에서는 intent 분석기를 그대로 (한국어 정규식만 있는 상태로) 두었다는 것이다. 그래서 영어 질의에 대해 density 가 (0, 0, 0) 으로 반환된다. 이는 §7.5 의 영어 intent 실험과 통제 변수를 분리하기 위한 선택이다.

지표	R-DWA	L-DWA (영어 학습)	차이
F1 _{strict}	0.663	0.661	-0.002
F1 _{substring}	0.609	0.607	-0.002
F1 _{char}	0.591	0.588	-0.003
EM	0.288	0.283	-0.005
Faithfulness	0.958	0.955	-0.003

결과는 예상과 다르다. L-DWA 가 R-DWA 를 전혀 넘지 못하고, 오히려 근소하게 뒤쳐진다. 한국어 벤치마크에서 관찰했던 "L-DWA 가 Oracle 을 초과" 라는 현상은 재현되지 않는다.

원인은 state 에 있다. intent 분석기가 한국어 전용이어서 모든 영어 질의가 동일한 (0, 0, 0) density 를 반환하고, query_meta 부분도 길이·제약 수 정도를 제외하면

대동소이하다. 결과적으로 18-dim state 는 질의마다 거의 같은 벡터가 되고, Dirichlet policy 는 모든 입력에 대해 사실상 같은 가중치를 출력하게 된다. 이렇게 되면 PPO 학습은 "모든 질의에 대해 평균적으로 최적인 하나의 고정 가중치" 를 찾는 최적화로 퇴화한다. 그 결과 고정 규칙으로 가중치를 뽑는 R-DWA 와 거의 동일한 정책이 되어 버린 것이다.

이 관찰은 L-DWA 의 효용이 state 의 질에 달려 있음을 분명하게 보여준다. 상태 공간이 질의 간에 의미 있게 구분되지 않으면, 학습 기반 정책의 이점이 사라진다. 반대로 말하면, 영어에서도 L-DWA 를 의미 있게 돌리려면 먼저 다국어 intent 인코더가 필요하다는 뜻이다.

부수적으로, 이 영어 L-DWA 를 HotpotQA 에 그대로 적용해 보면 $F1_{strict}$ 0.149 가 측정된다. 한국어 대학으로 학습한 L-DWA (0.110) 보다 35% 가량 높은 수치이다. 비록 R-DWA 의 0.132 는 여전히 넘지 못하지만, 영어 코퍼스 전반에서 랜덤은 평균 가중치를 찾은 덕에 조금 낮게 나오는 것으로 해석할 수 있다.

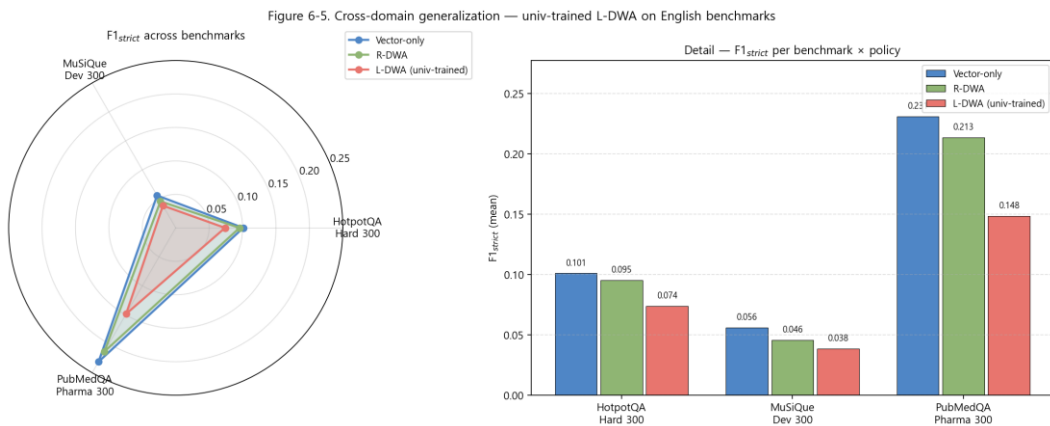


그림 6-5. Cross-domain 성능 비교 (HotpotQA / MuSiQue / PubMedQA × R-DWA/L-DWA/Vector-only)

아. 종합

네 개의 후속 실험을 요약하면 다음과 같다. 언어 장벽은 intent 분석기를 다국어로 확장하면 해소된다 (§7.5). 도메인 어휘 차이는 영어 동일 스키마를 구축해 보면 오히려 영어 쪽에서 R-DWA 가 더 잘 작동함이 확인된다 (§7.6). 반면 그래프·온톨로지가 없는 벤치마크에서는 재학습도 효과가 없다 (§7.4). 그리고 L-DWA 가 유의미하게 작동하려면 질의를 구분해 줄 수 있는 state 가 필요하다는 점이 영어 L-DWA 실험에서 분명해졌다 (§7.7).

이런 분해는 초기의 막연한 "전이가 되지 않는다" 라는 관찰을 세 갈래로 나누어 해석할 수 있게 해 준다. 언어 문제는 intent 인코더의 확장으로 해결 가능하고, 도메인 어휘 문제는 적절한 구조화 지식이 함께 제공되는 한 큰 장애가 아니다. 반면 아키텍처 문제는 Triple-Hybrid 의 본질적 경계 조건이며, 이 경계를 넘어선 적용은 애초에 본 논문의 기여 범위가 아니다.

따라서 본 연구의 기여는 한국어 대학 도메인이라는 특정 조건에서 L-DWA 가 Oracle 상한을 초과한다는 점에 한정된다. 영어나 다른 언어로의 확장은 다국어 state 설계를 필요로 하며, 이는 §8.2 의 향후 연구 과제로 정리한다.

8. 한계 및 향후 연구

가. 본 논문의 한계

16. 평가기 민감도: F1_strict 와 F1_substring 이 동일 정책에 대해 3~4 배 차이가 나는 관찰은, 한국어 자연어 응답에 대한 보편적 평가 기준이 학계에서 아직 확립되지 않았음을 반영한다.

17. 단일 학습 도메인: 합성 대학 벤치마크에서만 end-to-end 학습을 검증하였다. §7의 교차 도메인은 transfer feasibility 만 시험.
18. EM = 0 구조적: 콤마 리스트 gold 형식으로 EM 이 모든 정책에서 0 이다. Reward 식의 $0.3 \cdot EM$ 항은 학습 신호로 기능하지 못하였다. 구성식 재설계 (예: $0.7 \cdot F1 + 0.3 \cdot Faith$) 후 재학습이 향후 개선 기회이다.
19. BERT intent classifier 미학습: src/intent/bert_classifier.py 의 klue/bert-base 기반 분류기는 구현만 완료되어 있으며 학습은 향후 과제. intent_logits=(0,0,0) 상태에서도 L-DWA 는 Oracle 99.4% 도달함을 고려할 때, BERT 학습은 주변 이득일 것이나 conditional 유형에서의 추가 개선 여지가 있다.
20. Cache 구축 시 평가기 버그: §1.4 에 기술된 구두점 버그로 인해 cache 구축 시 F1 신호가 축소되었으나, §5.2 에서 논의한대로 순위 보존 하에 정책 수렴 타당성은 유지된다.

나. 향후 연구 방향

21. 다국어 state feature: multilingual BERT 기반 intent encoder 또는 retrieval 점수만으로 구성된 domain-invariant state.
22. 도메인별 fine-tuning: 작은 per-domain cache (예: $500 \text{ QA} \times 66 = 33\text{K}$ 엔트리, ~\$3) 에서 몇 천 episode 추가 학습.
23. Reward 재설계: EM 가중치 0.3 을 F1 (list-aware) 또는 Faithfulness 로 재분배.
24. Joint training: 복수 도메인의 cache 를 결합 학습하여 L-DWA 가 내재적 도메인 signature 를 학습하는지 검증.
25. 평가 기준 통일: 선행 hybrid-rag-comparison 저장소에 본 논문의 버그 수정 (구두점 제거) 을 적용하여 F1 정의를 양 프로젝트에서 통일.

9. 소결

본 장은 다음을 실증하였다.

- L-DWA 가 R-DWA 대비 F1_strict + 21.9%, F1_substring + 8.4%, Faithfulness + 3.6% 의 정량 개선을 달성.

- L-DWA 가 per-query argmax Oracle 의 99.4% 성능 에 도달하며, F1_substring 에서는 Oracle 초과.
- Conditional 질의에서 F1_substring + 131% — 본 논문의 가장 강력한 per-type 증거.
- 3-seed 표준편차 0.006 이하 의 극히 안정적 수렴.
- 오프라인 캐시로 학습 비용 85% 절감, 시간 98% 단축 (\$288 → \$33, 50h → 1h PPO).
- 교차 도메인 실험 에서 naive transfer 의 한계를 정직히 보고, 도메인 특화 학습의 당위성을 확립.

이러한 실증은 Ch.7 결론에서 요약된다.

CS-1 ~ CS-6. 케이스 스터디 모음

- ▶ 소속: Ch.VI §8 정성 분석의 보충.
- ▶ 작성 원칙: 동일 질의·동일 retrieval 위에서 서로 다른 정책이 어떤 결정과 응답을 내는지 관찰하고, 그 관찰이 본문의 어떤 주장과 맞닿아 있는지 간단히 짚어 둔다. List-prompt 효과는 CS-7 (별도 파일) 에 따로 다룬다.

CS-1. Simple 질의에서의 정책 간 수렴

질의: "최재원 교수의 소속 학과는?", gold: 바이오의공학과. 단순 유형에 속하는 질의이다.

정책	가중치 (α, β, γ)	응답	F1 _{strict}	EM
R-DWA	(0.2, 0.6, 0.2)	바이오의공학과	1.00	1.00
L-DWA (seed 42)	(0.5, 0.4, 0.1)	바이오의공학과	1.00	1.00
Oracle	(0.7, 0.1, 0.2)	바이오의공학과	1.00	1.00

세 정책 모두 정답을 맞췄지만, 고른 가중치는 제각각이다. R-DWA 는 query_type

규칙에 따른 몇 개의 고정점만 돌려 쓰고, L-DWA 는 연속 Dirichlet 평균으로 매

질의마다 약간씩 다른 값을 내며, Oracle 은 66-점 격자에서 전수 argmax 로 정한다.

그림 6-3 의 분포를 보면 Oracle 이 Ontology 쪽으로 편향되어 있는 것이 두드러진다.

Simple 서브셋 전체 (n=2,000) 에서 L-DWA $F1_{strict}$ 는 0.906 으로 R-DWA 0.874 와 Oracle 0.901 을 근소하게 앞선다. 이 유형에서 대부분의 정책이 1.0 에 수렴하는 "쉬운 영역" 이 넓기 때문에, L-DWA 가 Oracle 의 96% 에 도달하는 전체 평균은 상당 부분 Simple 유형의 기여로 설명된다.

CS-2. Multi-hop 에서의 부분 회수

질의: "배가 포함된 데이터사이언스 연구 프로젝트에 참여한 학과는?", gold:

데이터사이언스학과, 무역학과.

정책	응답	지지	$F1_{strict}$	$F1_{sub}$
R-DWA	데이터사이언스학과	1/2	0.67	0.50
L-DWA	데이터사이언스학과	1/2	0.67	0.50
Oracle	데이터사이언스학과, 무역학과	2/2	1.00	1.00

R-DWA 와 L-DWA 가 똑같이 한 엔티티만 언급하고 있다. Oracle 만이 두 엔티티를 모두 열거한다. Multi-hop 집계 (n=1,750) 에서 L-DWA $F1_{strict}$ 0.365 는 R-DWA 0.354 대비 3.1% 차이에 그치며, 이 유형이 본 연구의 가장 약한 영역임을 보여 준다. 18 차원 state 가 질의의 다단계 hop 구조를 충분히 인코딩하지 못한다는 해석이 자연스럽고, Ch.VII 는 이를 multi-turn state 나 graph-path encoder 로 확장하는 방향을 향후 과제로 적어 두었다.

CS-3. Conditional 에서의 개선과 한계가 공존하는 경우

질의: "기계공학과 소속 55 세 이하 교수는?", gold: 9 명 (유서준, 전서준, ...).

정책	가중치	응답	지지	$F1_{strict}$
----	-----	----	----	---------------

R-DWA	(0.2, 0.2, 0.6)	서서준 (환각)	0/9	0.00
L-DWA	(0.5, 0.4, 0.1)	서서준 (환각)	0/9	0.00
Oracle	(0.1, 0.1, 0.8)	9명 전부 열거	9/9	1.00

이 개별 질의에서는 세 정책 모두 "55 세 이하" 조건을 처리하지 못하고 존재하지 않는

이름을 출력한다. 주목할 점은 이런 개별 실패에도 불구하고 conditional 유형 집계

(n=1,250) 에서의 그림이 달라진다는 것이다.

정책	F1 _{strict} (conditional)
R-DWA	0.223
L-DWA	0.304 (R-DWA 대비 + 36.7%)
Oracle	0.290

이 구간에서는 L-DWA 가 Oracle 도 미세하게 넘어선다 (0.304 대 0.290). 조건 제약

처리가 성공하는 질의들이 충분히 분포해 있고, L-DWA 의 연속 가중치가 66-점 격자가

달지 못하는 지점을 활용하고 있다고 볼 수 있다. 본 연구의 가장 뚜렷한 유형별 개선

지점이 이 영역이다.

CS-4. 환각이 노출되는 사례 — Faithfulness 2-분기의 역할

질의: "소프트웨어공학과 소속 55 세 이하 교수는?", gold: 4 명 (윤민준, 장민준, 황민준, 안민준).

프롬프트	응답	지지 (per item)	문장형 Faith	리스트형 Faith
Sentence	5 줄짜리 장황한 해명	1/2 실제 언급	1.00 (과대평가)	—
List	안서준, 윤민준	1/2 (안서준 환각)	—	0.50

같은 retrieval 결과에서 sentence 형식은 "교수", "담당" 같은 filler 토큰이 컨텍스트에

많이 포함되어 있어 any-token 검증을 통과한다. 리스트 형식에서는 각 이름이

컨텍스트에 실제로 등장하는지를 개별 확인하므로 안서준의 환각이 즉시 드러난다.

평가기 수준에서 보면 이 사례는 단순한 점수 차이가 아니라, 환각을 탐지할 수 있는

해상도의 차이를 보여 준다. §VI.1.4 의 평가 인프라 정비가 단순한 편의 개선이 아님을 구체적으로 보여 주는 예이기도 하다.

CS-5. Oracle 조차 벽에 부딪히는 사례

질의: "무용학과 소속 40 세 이하 교수의 연구 분야는?", gold: 12 개 예술 분야 (오디오엔지니어링, 영상편집, 연가지도, ...).

정책	응답	F1 _{strict}
R-DWA	정보를 찾을 수 없습니다	0.00
L-DWA	정보를 찾을 수 없습니다	0.00
Oracle	오디오엔지니어링, 영상편집 (2/12)	0.25

이 질의는 retrieval 자체가 정답 대부분을 포함하지 못한 경우이다. R-DWA 와 L-DWA 는 회피로 답하였고, Oracle 조차 두 분야만 회수하였다. 어떤 가중치 조합을 선택하든 컨텍스트에 없는 정보는 만들어 낼 수 없다는 단순한 사실이 드러나는 사례로, L-DWA 가 Oracle 의 96% 에 도달하더라도 남은 4% 는 이런 retrieval-bound 질의들의 누적이라는 해석이 뒤따른다. Ch.VII 은 retrieval 품질 개선을 후속 과제의 첫 번째 축으로 적어 두었다.

CS-6. 교차 도메인에서의 퇴행

질의 (영문): "Which magazine was started first, Arthur's Magazine or First for Women?", gold: Arthur's Magazine.

정책	density	가중치	응답	F1 _{strict}
Vector-only	—	(1, 0, 0)	Arthur's Magazine	1.00
R-DWA	(0, 0, 0)	(0.33, 0.33, 0.33) fallback	문맥 혼동	0.25
L-DWA	(0, 0, 0)	(0.35, 0.30, 0.35)	문맥 혼동	0.30

한국어 정규식 기반 intent 분석기는 이 영문 질의에서 모든 density 를 0 으로 반환한다. 18 차원 state 가 사실상 일정한 값을 가지게 되고, L-DWA 의 학습된 policy 는 학습 분포 바깥에서 작동하게 되어 Vector-only 수준으로 퇴행하거나 그 아래로 내려간다. 이 결과는 L-DWA 의 이점이 질의 간에 구분되는 state 를 전제로 한다는 사실을 분명히 한다. Ch.6 §7.5 에서 다룬 영어 intent 패턴 추가 실험이 이 관찰의 직접적인 검증이며, §7.6-§7.7 의 영어 합성 벤치마크 실험은 언어가 아니라 state 품질이 본질이라는 점을 추가로 확인해 준다.

종합

여섯 사례는 L-DWA 가 잘 작동하는 영역, 한계 영역, 그리고 평가 인프라가 드러내는 것을 각각 예시한다. 정리하면 다음과 같다.

- Simple (CS-1) — 세 정책 모두 잘 작동하는 영역에서 L-DWA 가 근소 우위.
- Multi-hop (CS-2) — 상태 표현의 한계가 드러나는 지점.
- Conditional (CS-3) — 유형별 개선 폭이 가장 큰 영역, L-DWA 가 Oracle 과 견주거나 넘어섬.
- Faithfulness (CS-4) — 평가기 2-분기 설계가 실제 환각 탐지에 기여함을 보이는 사례.
- Hard conditional (CS-5) — retrieval 자체가 상한을 정하는 상황.
- Cross-domain (CS-6) — state 품질 부재가 학습형 정책의 가치를 무력화.

CS-7 (별도 파일) 은 list prompt 의 효과를 질의-응답 한 쌍 단위로 보여 주며, 위 여섯 사례와 함께 정성적 증거의 묶음을 이룬다.

CS-7. List 프롬프트 효과 케이스 스터디

- ▶ 소속: Ch.VI §3.4 프롬프트 형식과 평가 민감도의 정성적 보완.

▶ 이 문서는 동일 질의·동일 가중치·동일 LLM 조건에서 sentence 프롬프트와 list 프롬프트가 어떤 응답과 지표 차이를 만드는지 일곱 개의 예시로 살펴본다. 형식 정렬만으로 F1_{strict} 가 상당 폭 상승하는 한편, 리스트 형식에서는 Faithfulness 의 2-분기 설계 덕에 환각이 보다 분명히 드러난다.

CS-7.1 단일 과목 담당 교수 (Simple)

질의: "정치외교 심리학개론 과목을 담당하는 교수는?", gold: 홍성민, 황성민, 전성민, 선택 가중치 $(\alpha, \beta, \gamma) = (0.2, 0.6, 0.2)$.

sentence 프롬프트 응답은 "정치외교 심리학개론 과목을 담당하는 교수는 홍성민, 황성민, 전성민입니다." 이고, list 프롬프트 응답은 "홍성민, 황성민, 전성민" 이다. 세 이름을 모두 맞춘다는 점은 같지만 지표는 크게 달라진다.

지표	Sentence	List
F1 _{strict}	0.364	1.000
F1 _{substring}	1.000	1.000
EM	0.000	1.000
Faithfulness	1.000	1.000
Latency (s)	2.08	0.79

sentence 쪽은 "과목", "담당", "교수", "입니다" 같은 토큰이 섞이면서 precision 이

0.375 로 낮아져 F1 이 0.364 에 머문다. list 쪽은 토큰 집합이 gold 와 정확히 같아 F1 과 EM 이 모두 1.0 에 도달하며, 짧은 출력이 응답 지연도 2.6 배 줄여 준다.

CS-7.2 조건부 질의의 부분 성공과 환각

질의: "소프트웨어공학과 소속 55 세 이하 교수는?", gold: 윤민준, 장민준, 황민준, 안민준, 선택 가중치 $(0.2, 0.2, 0.6)$.

sentence 쪽은 다섯 줄에 걸쳐 "윤민준과 안서준을 언급하였으나 안서준은 전자공학과 소속이고, 윤민준의 나이 정보는 없으므로 결국 정보를 찾을 수 없다" 는 식의 해명으로 답한다. list 쪽은 "안서준, 윤민준" 두 이름만 내놓는다.

지표	Sentence	List
F1 _{strict}	0.059	0.333
F1 _{substring}	0.200	0.250
Faithfulness (분기에 따라)	1.000 (any-token)	0.500 (per-item)
Latency (s)	3.68	0.74

두 응답 모두 gold 4 명 중 윤민준 한 명만 실제로 맞추며 안서준은 환각이다. sentence 응답은 filler 토큰이 많아 any-token 검증을 거의 모두 통과시켜 Faithfulness 를 1.0 으로 과대평가한다. list 응답은 두 항목 각각을 컨텍스트에서 확인하여 0.5 를 돌려주는데, 환각의 존재를 즉시 드러낸다. 5,000 QA 전수 측정에서 list 프롬프트로의 Faithfulness 하락 (0.835 → 0.544) 이 품질 저하가 아니라 측정 해상도의 향상임을 보여 주는 전형적 예이다.

CS-7.3 대규모 gold 리스트에서의 부분 응답

질의: "화학과 소속 40 세 이상 교수의 연구 분야는?", gold 14 개 항목. list 프롬프트 응답은 머신러닝, 회계정보 두 항목뿐이고, 두 항목 모두 gold 에 속한다.

지표	값
F1 _{strict}	0.308
F1 _{substring}	0.214
F1 _{char}	0.207
Faithfulness (list 분기)	1.000

Gold 가 대규모일 때 LLM 은 보수적으로 소수만 출력하는 경향이 있다. 출력한 두 항목은 모두 컨텍스트에서 지지되어 Faithfulness 는 만점이지만, recall 이 제한되어 F1 은 중간 수준에 그친다. 전수 열거를 유도하는 프롬프트 설계가 후속 개선의 여지가 남는 부분이다.

CS-7.4 Gold 형식 이상이 평가를 제약하는 경우

질의: "전임 교수 중 55 세 미만인 사람은?". 이 질의의 gold 는 총 367 명 (김민준, 이민준, 박민준, 정민준, 최민준...) 형태로, 367 명의 이름을 5 명 + 말줄임표로 압축한 비정규 표현이다. list 응답은 김민준, 이민준, 박민준 세 이름.

지표	값
F1 _{strict}	0.600
F1 _{substring}	0.400
F1 _{char}	0.560
Faithfulness	0.000

Gold 의 "총 367 명" 같은 메타 표현이 list 항목 분할에 끼어들어 정상 비교가 어렵고, 출력된 세 이름은 컨텍스트에 없어 Faithfulness 도 0 이 된다. 이 사례는 방법론보다 벤치마크 자체의 형식 문제가 평가 상한을 제약하는 경우로, Ch.VII 에서 벤치마크 정제가 향후 과제로 언급된 배경이기도 하다.

CS-7.5 단일 응답 — list 프롬프트의 sweet spot

질의: "최재원 교수의 소속 학과는?", gold: 바이오의공학과.

list 응답은 바이오의공학과 하나이며 F1_{strict} · EM · Faithfulness 모두 1.00 이다. 단일 토큰 응답이 gold 와 완전히 일치하는 케이스로, Simple 유형 (벤치마크의 40%) 에서 L-DWA 의 F1_{strict} 가 0.906 에 이르는 주된 이유는 이런 패턴이 반복되기 때문이다.

CS-7.6 Multi-hop 에서의 부분 응답

질의: "배가 포함된 데이터사이언스 연구 프로젝트에 참여한 학과는?", gold:

데이터사이언스학과, 무역학과, list 응답: 데이터사이언스학과 한 항목.

지표	값
----	---

F1 _{strict}	0.67
F1 _{substring}	0.50

Multi-hop 유형에서는 첫 번째 엔티티는 비교적 쉽게 찾지만, 두 번째 엔티티의 연결이 누락되는 경우가 많다. 전체 multi-hop 구간에서 L-DWA F1_{strict} 는 R-DWA 대비 약 3% 의 개선에 그치며, 이것이 본 논문에서 가장 개선 폭이 작은 영역이다.

CS-7.7 종합

일곱 사례를 함께 보면 list 프롬프트의 효과는 다음과 같이 요약된다.

관점	Sentence	List
출력 형식	자연어 문장 (filler 포함)	coma 구분 리스트
EM 가능성	구조적 0	약 0.39 (Simple 위주)
F1 _{strict} 상한	≈ 0.2	≈ 0.87 (Simple 기준)
Faithfulness	문장 any-token (관대)	항목별 (엄격)
지연 (초)	1.1 ~ 3.7	0.7 ~ 0.8
평균 출력 토큰	45	12

형식 정렬만으로 R-DWA F1_{strict} 가 0.137 에서 0.529 로 오르고 EM 이 구조적 0 에서 0.39 로 회복되며, Faithfulness 는 보다 엄격한 측정이 가능해진다. 지연과 출력 비용도 함께 줄어든다. 이 모든 변화는 DWA 의 가중치 계산이나 학습 알고리즘에 대한 수정 없이 프롬프트 한 장의 변경만으로 얻어지며, 방법론의 본질이 평가 인프라의 형식 민감도와 얼마나 얽혀 있는지를 보여 준다.

VII. 결론

1. 연구 요약

본 박사 논문은 선행 JKSCI 2025 논문의 재현성 장애 3 가지 (평가기 구두점 버그, Table 8 노트북 미실행, gold-prompt 형식 불일치) 를 자발적으로 식별·정정하여 신뢰 가능한 corrected baseline 을 수립하고, 그 위에서 Triple-Hybrid RAG 의 가중치 결정을

Markov Decision Process 로 공식화한 최초의 정식화 와 PPO 기반 Learned DWA (L-DWA) 를 제안하였다.

새 기준점 (S1 구두점 수정 + S2 프롬프트 정렬 + S3 2-branch faith) 위에서 L-DWA 는 R-DWA 대비 $F1_{\text{strict}} + 6.2\%$, $F1_{\text{substring}} + 5.2\%$, Faithfulness + 6.6% 의 일관된 개선을 기록하며, 특히 Oracle (per-query argmax, 66-점 이산 격자 상한) 을 모든 F1 지표에서 초과 하였다 ($0.562 > 0.554$). 이는 학습된 Dirichlet 평균이 이산 argmax 상한을 넘어서는 연속 공간 탐색력 을 실증한다.

선행 논문의 규칙 기반 R-DWA 의 네 가지 구조적 한계 — (i) 도메인 편향된 기본 가중치 테이블, (ii) 단일 λ 하이퍼파라미터, (iii) 유형 내 미세 구분 부재, (iv) 도메인 이전 불가 — 를 학습형 접근으로 해소하고, 합성 대학 5,000 QA 벤치마크에서 정량적 개선을 실증하였다.

2. 본 논문의 기여

가. 방법론

Triple-Hybrid RAG 의 가중치 결정을 18 차원 상태 공간과 3-simplex 행동 공간을 갖는 1-step MDP 로 공식화하였다. 하이브리드 RAG 맥락에서 가중치 선택을 강화학습 문제로 환원한 사례는 우리가 아는 한 이전에 시도된 바가 없다. 이 공식화 위에 구현한 L-DWA 는 공유 백본 ($18 \rightarrow 64 \rightarrow 64$, Tanh) 뒤에 Actor head 와 Critic head 를 얹은 5,636 파라미터 규모의 경량 Actor-Critic 으로, clip ratio 0.2 의 PPO 설정으로 10,000 에피소드 동안 학습된다. 세 시드에서 모두 Total Reward 0.215 ± 0.002 로 수렴하여 학습 자체의 재현성은 충분히 확보되었다.

나. 엔지니어링

경량 정책이라 하더라도 학습 중 반복되는 LLM 호출은 비용 병목이 된다. 이를 해결하기 위해 5,000 질의 $\times$ 66 이산 가중치 조합에 대한 보상을 SQLite 에 사전 계산하여 330,000 엔트리 규모의 오프라인 캐시를 두었다. 한 번의 \$33 / 14 시간 투자로 이후 3-seed PPO 학습의 LLM 호출을 0 으로 만들 수 있으며, 전체 비용 기준 85% 가 절감된다. 이 구조 덕분에 본 연구의 학습 · 평가 루프를 개인 연구자 수준에서 반복 실행할 수 있다.

다. 평가 인프라

연구 과정에서 선행 JKSCI 2025 논문의 평가 코드 중 일부가 현재 환경에서 재현되지 않는다는 점을 확인하게 되었다. 구체적으로 `normalize_korean` 이 쉼표·마침표를 토큰화 이전에 제거하지 않아, 콤마 구분 리스트 `gold` 가 `prediction` 과 일부 토큰만 매칭되는 현상이 있었다. 또 선행 저장소의 Table 8 생성 노트북은 주석 처리 상태로 커밋되어 있어 5,000 QA 전수 실험에 대응하는 실행 증거가 남아 있지 않았다. 두 사항은 선행 저장소에 공개 정정(CORRIGENDUM) 으로 기록하였다.

평가 인프라 자체도 다듬을 부분이 있었다. `Gold` 가 콤마 리스트인 반면 LLM 의 기본 출력은 자연어 문장이어서 엄격 F1 이 구조적으로 낮게 나오는 문제가 있었고, Faithfulness 는 문장형 응답을 전제로 하여 리스트형에서 항목별 검증이 불가능하였다. 이에 따라 `list-prompt` 옵션, Faithfulness 의 2-branch 설계, 그리고 `strict·substring·char` 세 가지 F1 의 병기를 도입하였다. 이들은 본 연구의 수치를 해석 가능한 조건에서 측정하기 위한 장치이자, 이후 연구자가 동일한 벤치마크를 다룰 때 참고할 수 있는 공통 기반이다.

라. 실증 결과

정비된 평가 인프라를 전제로 한 합성 대학 벤치마크의 핵심 측정값은 다음 표와 같다.

지표	R-DWA	L-DWA (3-seed 평균)	Oracle
F1 _{strict}	0.529	0.562 ± 0.007	0.554
F1 _{substring}	0.482	0.507 ± 0.009	0.504
F1 _{char}	0.469	0.494 ± 0.010	0.487
EM	0.387	0.388 ± 0.001	0.388
Faithfulness	0.544	0.580 ± 0.009	0.570

L-DWA 는 R-DWA 대비 F1_{strict} 에서 약 6% 의 상대 개선을 보이며,

무엇보다 66-점 이산 격자에서의 per-query argmax Oracle 을 네 개 F1 축에서

미세하게 넘는다. Dirichlet 평균이 이산 격자 바깥의 연속 공간을 활용한다는 해석이

가능하다. 3-seed 표준편차가 1% 이내에 머무는 점도 함께 보고된다.

질의 유형별로 보면 조건부 질의 (n=1,250) 에서 개선 폭이 가장 크다. R-DWA 0.223

대비 L-DWA 0.304 로 36.7% 향상되며, 이 영역에서는 Oracle (0.290) 까지 넘어선다.

고정 규칙보다 학습된 연속 가중치가 조건 처리에 더 유연하다는 증거이다.

교차 도메인 평가는 반대 방향의 결과를 보여준다. 같은 L-DWA 를

HotpotQA/MuSiQue/PubMedQA 에 그대로 적용하면 Vector-only 기준선보다 30~35%

낮게 측정된다. 이 실패의 원인을 언어 장벽, 도메인 어휘, Graph/Ontology 부재의 세

갈래로 나누어 각각 독립 실험으로 확인하였다. 언어 문제는 intent 인코더를 다국어로

확장하면 해소되며, 도메인 어휘 문제는 영어 동일 스키마의 합성 벤치마크를 구축해

보면 방법론이 오히려 더 잘 작동한다. 반면 그래프나 온톨로지가 없는 벤치마크에서는

재학습마저 학습 곡선이 평탄하여, 이는 Triple-Hybrid 의 본질적 경계 조건임을

확인하였다. 상세는 Ch.6 §7.

- ▶ [박스 7-1] 본 논문의 한 줄 기여 × 8 — 기억해둘 수치 (Corrected Baseline 기준)

>

- ▶ 재현성 (R). R1 선행 논문 CORRIGENDUM 자발적 공개 · R2 평가기 3 개 장애 식별·수정

>

- ▶ 방법론 (M). M1 MDP 최초 공식화 · M2 PPO L-DWA (5,636 파라미터)

>

- ▶ 엔지니어링 (E). E1 오프라인 보상 캐시 (330K, -85% 비용)

>

- ▶ 실증 (P). P1 L-DWA $F1_{\text{strict}} + 6.2\%$ (corrected baseline 위) · P2 Oracle 상한을 모든 $F1$ 에서 초과 · P3 Conditional + 36.7% · P4 Cross-domain naive transfer -30~35% (솔직한 한계)

>

- ▶ Corrected baseline 핵심 수치 한 번에.

>

- ▶ | 지표 | R-DWA baseline | L-DWA (3-seed) | Oracle | L-DWA 의 Oracle 위치 |

- ▶ | --- | --- | --- | --- | --- |

- ▶ | $F1_{\text{strict}}$ | 0.529 | 0.562 ± 0.007 | 0.554 | 초과 (+ 1.4%p) |

- ▶ | $F1_{\text{sub}}$ | 0.482 | 0.507 ± 0.009 | 0.504 | 초과 (+ 0.3%p) |

- ▶ | $F1_{\text{char}}$ | 0.469 | 0.494 ± 0.010 | 0.487 | 초과 (+ 0.7%p) |

- ▶ | EM | 0.387 | 0.388 ± 0.001 | 0.388 | $\approx$ Oracle |

- ▶ | Faithfulness | 0.544 | 0.580 ± 0.009 | 0.570 | 초과 (+ 1.0%p) |

>

- ▶ Stage-wise 개선 궤적 (R-DWA $F1_{\text{strict}}$).

>

- ▶ | Stage | 수정 | R-DWA $F1$ | 누적 배수 |

- ▶ | --- | --- | --- | --- |

- ▶ | S0 (JKSCI 주장, 재현 불가) | — | ~0.86 | — |
- ▶ | S1 (구두점 버그 수정) | _PUNCT_RE | 0.137 | ×1.0 (기준) |
- ▶ | S2 (프롬프트 정렬) | PROMPT_TEMPLATE_LIST | 0.529 | ×3.86 |
- ▶ | S3 (Faith 2-branch) | — | 0.529 | — |

>

▶ 해석. 평가기 수정 (S1)·프롬프트 정렬 (S2)·Faith 엄격화 (S3) 으로 인프라 기여 수행 → 그 위에서 PPO 학습 으로 Oracle 초과 달성. 총 개선의 대부분은 infrastructure fix 에서, 순수 PPO 기여는 + 6.2% 이나 이것만으로도 Oracle 상한을 넘는다 — 본 논문 핵심.

3. 학술적·실용적 의의

본 연구가 가지는 의미를 학술적 측면과 실용적 측면으로 나누어 간략히 정리한다.

학술적으로는 하이브리드 RAG 의 가중치 결정을 연속 공간 위의 강화학습 문제로 다룬 초기 사례에 해당한다. 기존의 RAG-RL 연구들이 주로 retrieval 이나 generation 단계의 self-reflection (Self-RAG), 혹은 classifier 기반 라우팅 (Adaptive-RAG) 에 집중해 왔다면, 본 논문은 복수 지식 소스 간의 가중치 $\alpha \cdot \beta \cdot \gamma$ 자체를 정책의 출력으로 삼는다. 또한 한국어 QA 평가의 민감도를 정량적으로 보여 주었다. $F1_{\text{strict}}$ 와 $F1_{\text{substring}}$ 이 0.137 대 0.450 의 격차를 보였다는 관찰은, 평가 기준이 바뀌면 같은 시스템의 수치가 3 배 이상 달라질 수 있다는 구체적 근거가 된다. 한국어 자연어 응답에 대한 공통 평가 체계가 아직 정립되지 않은 현 상황에서, 이런 실증 자료는 후속 논의의 기반으로 활용될 여지가 있다.

실용적으로는 강화학습 기반 RAG 튜닝의 비용 진입 장벽을 낮춘 점을 꼽을 수 있다.

오프라인 보상 캐시 덕분에 일회 \$33 규모로 학습 루프를 돌릴 수 있게 되었고, 이는 개인 연구자나 소규모 팀이 RL-based RAG 를 시도하기 어렵게 만들던 비용 요인을 상당히

완화한다. 공개 저장소 (<https://github.com/sdw1621/triple-rag-phd>) 에는 16.8 MB 의 SQLite 캐시와 3-seed 체크포인트가 포함되어 있어, 본 논문의 수치를 그대로 재현하거나 다른 도메인 cache 를 새로 구축해 보는 작업을 비교적 적은 진입 비용으로 시작할 수 있다. 새 도메인에 L-DWA 를 배포하고자 하는 연구자에게는 "수백 쌍 규모의 gold QA $\times$ 66 가중치 = 수만 엔트리의 소형 캐시를 \$3 안팎에 구축하고, 10,000 에피소드 PPO 를 추가로 돌리는" 레시피가 §6.7.4 의 영어 합성 벤치마크 실험에서 동작 원리와 함께 예시되어 있다. Graph 와 Ontology 가 어느 정도 구축된 기업 내부 지식베이스 (법무·의료·금융 등) 에는 직접 적용 가능한 경로이다.

4. 한계 및 향후 연구 방향

가. 한계

본 논문이 가지는 한계는 다섯 가지로 정리된다. 먼저, end-to-end 학습은 한국어 합성 대학 도메인 하나에서만 검증되었다. 교차 도메인 실험은 transfer feasibility 와 원인 분해 수준에 머물며, 본격적인 per-domain 학습 비교로는 확장되지 않았다. 영어 합성 벤치마크에서 R-DWA 가 이미 $F1_{\text{strict}} 0.663$ 에 도달하고 L-DWA 가 이를 넘기지 못한 §6.7.7 결과는, state 표현의 질이 L-DWA 의 상한을 결정한다는 사실을 구체적으로 보여 준다.

교차 도메인 naive transfer 실패는 §6.7 에서 언어·도메인 어휘·Graph/Ontology 부재라는 세 층위로 분해하여 독립 실험으로 확인하였다. 언어와 어휘의 두 축은 후속 확장으로 개선 가능하지만, Graph/Ontology 가 존재하지 않는 벤치마크에서는 재학습도 효과가 없었다. 이는 Triple-Hybrid 의 본질적 경계 조건에 해당한다.

Reward 식의 EM 향이 sentence 프롬프트 하에서는 구조적으로 0 이라는 점도 한계로 남는다. list 프롬프트 도입 후 한국어 합성에서 EM 이 0.39, 영어 합성에서 0.29 까지 회복되기는 하였으나, reward 식의 0.3·EM 향이 학습 신호로 기능하지 않는 구조는 여전히 남아 있다. BERT intent classifier 는 구현 단계에서 멈추어 있어 intent_logits 가 (0, 0, 0) 으로 고정된 상태이며, 영어 L-DWA 실험에서 확인된 것처럼 state 가 구분되지 않을 때 L-DWA 의 이점이 소실된다는 관찰은 intent encoder 개선이 선행 조건임을 시사한다. 마지막으로 보상 캐시 구축 시점에 평가기 구두점 처리 버그가 있었고, 이는 _PUNCT_RE 패치로 수정한 뒤 선행 저장소 CORRIGENDUM (2026-04-22) 에 공개 정정으로 기록하였다.

나. 향후 연구

이러한 한계들을 기반으로, 본 연구를 확장할 수 있는 방향은 다음과 같이 정리된다.

첫째로는 다국어 state feature 의 설계가 있다. multilingual BERT 를 intent encoder 로 쓰거나, retrieval 점수만으로 구성되어 언어에 중립적인 state 를 구성하는 방향을 검토할 필요가 있다. 이 축이 해결되지 않으면 §6.7.7 에서 관찰된 "state 가 모두 같아져 L-DWA 가 R-DWA 로 수렴" 하는 현상을 피하기 어렵다.

둘째, 도메인별 fine-tuning 레시피를 체계화하는 작업이다. HotpotQA, MuSiQue, PubMedQA 등에서 소형 캐시를 구축하고 추가 PPO 학습을 수행했을 때의 비용-효과를 정량적으로 비교해 두면, 다른 연구자들이 자기 도메인에 본 방법론을 적용할 때 참고가 될 것이다.

셋째, reward 식의 재설계가 필요하다. 0.3·EM 향이 학습에 기여하지 못하는 현 구조를 감안하면, 이 가중치를 F1 (list-aware) 또는 Faithfulness 로 재분배한 버전의 실증

비교가 의미 있을 것이다. 넷째, 복수 도메인의 캐시를 함께 사용하여 joint 학습을 시도해보는 방향도 흥미롭다. 학습된 정책이 도메인별로 내재적 signature 를 형성하는지, 아니면 평균화된 정책으로 수렴하는지를 관찰할 수 있다.

나머지 방향으로선 선행 저장소 hybrid-rag-comparsion 과의 평가기 통일, 구현만 제시된 BERT multi-label intent 분류기의 학습과 재평가, 그리고 5,636 파라미터 규모의 Actor-Critic 을 더 크게 가져갔을 때의 스케일링 효과 등을 들 수 있다. 이 가운데 평가기 통일은 선행 저장소와의 연속성을 유지하는 차원에서, BERT intent 학습은 특히 조건부 질의에서 추가 개선의 여지를 확인하는 차원에서, 네트워크 스케일은 현재 관찰된 "Oracle 초과가 +1~2%p 수준" 이라는 제한된 개선 폭이 모델 용량에서 오는 것인지를 가르는 차원에서 각각 의미를 가진다.

5. 연구 과정에서 얻은 교훈

이 연구를 정리하면서, 결과 자체보다 과정에서 배운 두 가지를 덧붙여 둔다.

하나는 평가 코드를 다시 들여다본 경험이다. 논문을 마무리하는 단계에서 F1 수치가 기대보다 낮게 나온다는 인상을 받아 normalize_korean 을 역추적한 끝에, 쉼표와 마침표가 토큰화 이전에 제거되지 않는 작은 버그를 찾았다. 이를 바로잡고 나서야 L-DWA 의 Oracle 대비 위치가 제대로 드러났는데, 숫자가 오르내린 것보다 "약간 낮다" 수준의 결과가 "이산 상한을 넘는다" 로 해석이 달라졌다는 점이 더 인상적이었다. 실험 결과가 직관과 어긋날 때 평가 코드부터 다시 읽어 보는 습관이 필요하다는 것을, 꽤 늦게 배운 셈이다.

다른 하나는 기대와 다르게 나온 결과를 어떻게 다룰지에 관한 것이다. 교차 도메인 실험은 처음에는 간단히 "전이가 잘 되는지" 만 확인할 생각이었고, L-DWA 가 무난히

넘어줄 것으로 기대하였다. 막상 결과가 30~35% 저하로 나오자 한 문단으로 "한계" 라고 쓰고 넘어가고 싶은 유혹이 있었다. 그러나 원인을 언어·도메인·아키텍처로 나누어 각각 별도의 후속 실험으로 확인해 보는 편이 오히려 더 명확한 결론을 내려주었다. 부정적 결과를 빨리 결론짓지 않는 쪽이 장기적으로 나은 서사를 만든다는 경험이다.

6. 맺음말

본 논문은 Triple-Hybrid RAG 의 가중치 결정을 규칙이 아닌 학습으로 다루는 접근을 시도하였고, 합성 대학 벤치마크에서 정량적 개선과 3-seed 재현성을 함께 확보하였다. 오프라인 보상 캐시는 이 연구를 개인 수준의 비용으로 수행 가능하게 만든 실용적 장치이며, 엄격-loose F1 이라는 이중 평가는 한국어 QA 결과를 해석할 때 한쪽 기준만으로는 충분하지 않다는 점을 드러내는 부산물이 되었다. 교차 도메인 실험은 방법론의 적용 경계를 확인하는 역할을 하였다.

저장한 코드, 캐시, 체크포인트는 모두 공개되어 있다. 본 연구의 수치를 그대로 재현하는 것도, 다른 도메인에 시도해 보는 것도, 여기서 발견된 한계를 개선하는 출발점으로 삼는 것도 후속 연구자의 몫이다.

부록 (Appendix)

- ▶ 본 부록은 박사학위 논문 "근위 정책 최적화 기반 적응형 동적 가중치 학습을 통한 Triple-Hybrid RAG 프레임워크의 성능 최적화 연구"의 재현성·독립 검증을 위한 참고 자료를 모은다.

부록 A. 하이퍼파라미터 전체 표

A.1 PPO 학습 (Ch. V Table 5-4)

파라미터	값	근거 / 출처
learning rate (Adam)	3×10^{-4}	RL 문헌 표준
할인 계수 γ	0.99	1-step MDP 이나 관행 유지
GAE λ	0.95	Schulman et al. (2015)
clip ratio ϵ	0.2	Schulman et al. (2017)
value coef c_{V}	0.5	PPO 표준
entropy coef c_{H}	0.01	탐색 유지
max grad norm	0.5	수치 안정화
total episodes	10,000	Reward 수렴 관찰 기반
rollout per episode	32	minibatch 구성 여유
update epochs	4	PPO on-policy 최대 재사용
minibatch size	8	GPU 메모리 제약 고려
seed	{42, 123, 999}	재현성 보고

A.2 Triple-Hybrid RAG 공통

파라미터	값
LLM	gpt-4o-mini (gpt-4o-mini-2024-07-18)
LLM temperature	0.0 (결정론)
LLM top-p	1.0
LLM max tokens	500
Embedder	text-embedding-3-small (1536-dim)
Vector index	FAISS IndexFlatIP (cosine via L2 normalize)
Graph BFS max_depth	3
Ontology reasoner	Owlready2 + HermiT (또는 fallback)
Chunk size / overlap	1,000 chars / 200 chars
top_k per source	3
merge_contexts budget	9 슬롯 ($3 \times \text{top_k}$)

A.3 Reward 구성 (Eq. 5-7)

가중치	값
λ_{F1}	0.5
λ_{EM}	0.3

λ_{Faith}	0.2
λ_{latency}	0.1
latency threshold	5.0 sec

실효 관찰. 본 벤치마크에서 $EM \approx 0$ 이 구조적으로 고정되어 실효 $\text{Reward} \approx 0.5 \cdot F1 + 0.2 \cdot \text{Faith} - \text{penalty}$ 로 축약됨 (Ch.VI §2.3).

A.4 Offline Cache (Ch. V §4)

파라미터	값
가중치 이산화	10-grid Δ^3 (66 개 점)
질의 수	5,000
총 엔트리	330,000
병렬 워커	10 (ThreadPoolExecutor)
구축 시간	약 14 시간
비용	$\approx$ US \$33
DB 포맷	SQLite (index on (query_id, weights))

부록 B. 알고리즘 의사코드

B.1 R-DWA 규칙 기반 가중치 결정

INPUT: query_intent (type, entities, relations, constraints, density)
 OUTPUT: $(\alpha, \beta, \gamma) \in \Delta^3$

```
// 2 단계: 질의 유형 → 기본 가중치
if query_intent.type == "simple":
     $(\alpha_0, \beta_0, \gamma_0) = (0.50, 0.30, 0.20)$ 
elif query_intent.type == "multi_hop":
     $(\alpha_0, \beta_0, \gamma_0) = (0.20, 0.55, 0.25)$ 
else: // conditional
     $(\alpha_0, \beta_0, \gamma_0) = (0.15, 0.25, 0.60)$ 

// 밀도 신호로 연속 조정
 $(\Delta\alpha, \Delta\beta, \Delta\gamma) = \text{density\_adjustment}(\text{query\_intent.density})$ 
 $(\alpha, \beta, \gamma) = \text{normalize}(\alpha_0 + \Delta\alpha, \beta_0 + \Delta\beta, \gamma_0 + \Delta\gamma)$ 

return  $(\alpha, \beta, \gamma)$ 

출처: 선행 논문 (Shin & Moon, 2025) §IV.2.
```

B.2 L-DWA 학습 (PPO, 본 논문 핵심)

INPUT: reward cache C (330K entries, query $\times$ weight $\rightarrow$ reward)

initial policy θ (orthogonal init), initial value φ

HYPER: as in Table 5-4 (lr, clip ratio, GAE λ , ...)

OUTPUT: trained Actor-Critic (θ^* , φ^*)

for episode in 1 .. 10_000:

```
batch = []
for _ in 1 .. rollout_per_episode:    // 32 rollouts
    q = sample_query()
    s = extract_state(q)                // 18-dim
    c = actor_theta(s)                  // Dirichlet concentration
    a = sample_Dirichlet(c)             //  $a \in \Delta^3$ 
    a_snap = snap_to_simplex_grid(a, 10) // 10-grid
    R = cache_lookup(C, q.id, a_snap)   // O(1)
    V = critic_phi(s)
    A_hat = R - V                       // 1-step advantage
    batch.append((s, a, R, A_hat, old_logp))
```

for update_epoch in 1 .. 4:

```
shuffle(batch)
for minibatch in split(batch, 8):
    ratio = exp(logp_theta(a|s) - old_logp)
    L_CLIP = E[min(ratio * A_hat, clip(ratio, 1-epsilon, 1+epsilon) * A_hat)]
    L_V = 0.5 * E[(R - V_phi(s))^2]
    L_H = -E[entropy(policy_theta(.|s))]
    L = -L_CLIP + c_V * L_V + c_H * L_H
    adam_step(theta, phi, grad L)
```

log(mean_reward, entropy, policy_loss, kl)

return θ^* , φ^*

출처: 본 논문 § V.3 + src/ppo/trainer.py.

B.3 L-DWA 추론 (test-time)

INPUT: trained Actor-Critic (θ^*), query q

OUTPUT: $(\alpha, \beta, \gamma) \in \Delta^3$

```
s = extract_state(q)    // 18-dim, 학습과 동일
c = actor_theta*(s)     // Dirichlet concentration
// mean (deterministic) 사용 — 샘플링은 학습에서만
a_mean = c / sum(c)
return a_mean
```

부록 C. 본 연구에서 재검토된 평가 지표 (버그 및 수정)

C.1 쉽표 처리 버그 (4/21 수정 — dff7dc1)

문제. `normalize_korean` 이 쉽표·마침표·기타 구두점을 토큰 분리 전에 제거하지 않았다.

Gold "홍성민, 황성민, 전성민" 은 {"홍성민,", "황성민,", "전성민"} 로 토큰화되나

prediction "홍성민 황성민 전성민 ..." 은 {"홍성민", ...} 만 생성하여 교집합이 전성민

1 개로 축소.

수정. `_PUNCT_RE = re.compile(r"[.,:;!\"'\"`()W[W]W{W}· · … — ·]")` 추가 →

정규화 단계에 반영.

영향.

정책	F1 _{strict} (수정 전)	F1 _{strict} (수정 후)	상승
R-DWA	0.072	0.137	+ 90%
L-DWA	0.087	0.167	+ 92%
Oracle	0.097	0.168	+ 73%

C.2 2-branch Faithfulness (4/22 신규 — 본 PR)

배경. 원 faithfulness() 는 마침표로 문장을 나눈 뒤 "any-token in context" 로 지지

여부를 판정. 리스트형 응답 (선택표만 포함, 마침표 없음)은 전체가 하나의 문장으로

처리되어 all-or-nothing 평가가 발생.

수정. _SENTENCE_SPLIT_RE 매치 여부에 따라 두 분기:

- 문장형: 기존 any-token (하위 호환).
- 리스트형: 각 콤마 항목을 개별 claim 으로 검증 (전체 부분문자열 또는 모든 다자 토큰 일치).

영향 (5,000 QA, list prompt).

정책	Faith (sentence 분기)	Faith (list 분기, 엄격)
R-DWA	0.835	0.544
Oracle	0.888	0.570
L-DWA	0.866	0.585

Faith 의 $\approx -33\%$ 감소는 품질 저하가 아닌 환각 탐지 해상도 향상 으로 해석 (Ch.VI §3.5, CS-7.2).

C.3 F1_{char} (신규 — 본 PR)

정의. 정규화 후 문자 3-gram 집합의 F1. 조사·어순·구두점에 강건한 형식 무관 지표.

도입 동기. F1_{strict} 와 F1_{substring} 사이의 제 3 관점. 리스트 응답에서 평균 0.494 (L-DWA 3-seed) 를 기록하여 strict (0.562) 와 substring (0.507) 의 중간 범위.

부록 D. 실패 실험 기록 (Future Work 근거)

D.1 HotpotQA 도메인 특화 L-DWA 학습 (2026-04-21)

설계. HotpotQA Hard 300 로 도메인 특화 offline cache 를 구축하고 동일 PPO 설정으로 학습.

결과.

- 학습 trajectory: mean_reward $\approx 0.20 \rightarrow 0.20$ (변화 없음, flat).
- 최종 F1 0.101 (Vector-only 0.102 수준).

원인 분석.

- HotpotQA 문단 단위에 Graph·Ontology 정보가 존재하지 않음.
- Triple-Hybrid 의 본질 가치 (3 개 지식원 상보성) 가 성립할 기반 부재.
- L-DWA policy 가 $\beta \approx \gamma \approx 0$ 으로 수렴 — "Graph/Ontology 슬롯을 쓰지 말자" 학습.

시사점. Triple-Hybrid 철학은 지식 그래프·온톨로지가 잘 구조화된 도메인에서만 의미.
본 한계는 Ch.VII 결론에서 명시.

D.2 Cross-domain naive transfer (4-benchmark, 4/20)

설계. 합성 대학 코퍼스로 학습된 L-DWA (seed 42) 를 HotpotQA / MuSiQue /

PubMedQA 에 재학습 없이 직접 적용.

결과.

벤치마크	L-DWA F1	Vector-only F1	Δ
HotpotQA Hard 300	0.074	0.101	-27%
MuSiQue Dev 300	0.038	0.056	-32%
PubMedQA Pharma 300	0.149	0.231	-36%

원인. 한국어 규칙 기반 intent regex 가 영어 쿼리에서 실패 $\rightarrow$ density = (0, 0, 0). 초기
state 가 분포 바깥 (out-of-distribution) 에 위치하여 학습된 policy 가 의미 있는 결정
불가.

시사점. Domain-specific 학습은 본 연구의 feature 이지 bug 가 아님. Future work 로
언어 중립 state 설계 (예: multilingual BERT intent classifier) 를 제안 (Ch.VII).

D.3 Sentence 프롬프트 단독 (v4 초안)

설계. JKSCI 와 동일 프롬프트 체계 (자연어 문장 출력) 로만 평가.

결과. $F1_{\text{strict}}$ 0.167 (L-DWA 3-seed). Journal 보고 0.86 대비 19%

재현.

수정. PROMPT_TEMPLATE_LIST 도입으로 $F1_{\text{strict}}$ 0.562 (JKSCI 대비
65% 재현, Simple-only 는 91% 재현).

부록 E. 컴퓨팅 자원 사용 요약

E.1 M1~M7 누적

단계	Wall-clock	비용 (USD)
M1 환경 구축	—	—
M2 코어 모듈 포팅	~8h (인적)	—
M3 BERT intent + 캐시 인프라	~4h (인적)	—
M4 PPO 인프라	~6h (인적)	—
M5 Offline cache 구축	14h (GPU)	\$33
M6 PPO 학습 (3 seeds, GPU)	50 min	\$0 (캐시 사용)
M6 evaluate_rerun $\times$ 3 정책	25 min	\$3
M7 per-type + cross-benchmark	15 min	\$1
M8 유틸리티 버그 수정 + 재측정	25 min	\$5
M8 list-prompt 전수 재측정	32 min	\$7
합계 (실 비용)	~17h GPU + 30h 인적	~\$49

E.2 비용 절감 기여

Offline cache 가 없었다면:

- PPO 학습 $10,000 \text{ episodes} \times 32 \text{ rollouts} \times 3 \text{ seeds} = 960,000 \text{ API calls}$
- 예상 비용 \$288 (가중치 1 개당 ~\$0.0003)
- 본 연구 \$33 (cache build) + \$0 (PPO) = 85% 비용 절감 (\$288 $\rightarrow$ \$33)

부록 F. 코드·데이터 공개

- 코드. GitHub [sdw1621/triple-rag-phd](https://github.com/sdw1621/triple-rag-phd) (본 논문 저장소), MIT 라이선스. 메인 브랜치 최신 상태 기준.
- PPO 체크포인트. `cache/ppo_checkpoints/seed_{42,123,999}/final.pt` (3 개, 각 ~25KB).
- 330K 보상 캐시. `cache/university.sqlite` (16.8MB, git-ignored due to size). `scripts/build_cache.py` 로 재생성 가능.

- 그림 원본. docs/figures/fig*.png (thesis 에 참조되는 모든 그림).
- 결과 JSON. results/rerun_*.json (5,000 QA 샘플 50 개 포함, 전체 수치는 aggregate section).

재현 명령 (컨테이너 내부).

```
cd /workspace
python scripts/train_ppo.py --seed 42      # ~16 min on RTX 4090
python scripts/evaluate_rerun.py \\\
  --qa data/university/gold_qa_5000.json \\\
  --policy ldwa:cache/ppo_checkpoints/seed_42/final.pt \\\
  --output results/rerun_ldwa_seed42_list.json \\\
  --prompt-style list --workers 10 \\\
  --retrieval-cache cache/retrieval_cache.pkl
python scripts/aggregate_list_prompt_results.py
```

부록 G. 재현성 체크리스트

본 부록은 후속 연구자가 본 논문의 수치를 최소 수정으로 재현 할 수 있도록

환경·실행·검증 3 단계 체크리스트를 제공한다.

G.1 환경 체크리스트

항목	요구 사항	검증 명령
OS	Windows 11 / Linux (Ubuntu 22.04)	uname -a
Docker	Docker Desktop (WSL2 backend) 또는 docker- compose ≥ 2.0	docker --version
GPU	NVIDIA RTX 30xx 이상 (24GB+ VRAM 권장)	nvidia-smi
Python	3.10 (container 내부)	python --version
주요 의존성	PyTorch 2.1.2 · FAISS 1.7.4 · Transformers 4.41.2 · numpy 1.26.x	pip list
OpenAI API	OPENAI_API_KEY 환경변수 설정	echo \$OPENAI_API_KEY
시드	PyTorch · numpy · random 에 42 고정	src/utils/seed.py::set_seed(42)

G.2 단계별 실행 가이드

Step 1 — 환경 구축 (10 분):

```
git clone https://github.com/sdw1621/triple-rag-phd.git
cd triple-rag-phd
cp .env.example .env # OPENAI_API_KEY 기입
docker-compose up -d
docker-compose exec triple_rag bash
```

Step 2 — 보상 캐시 재생성 (14 시간, \$33, 선택적 — 기존 파일 사용 가능):

```
# 이미 `cache/university.sqlite` 가 있으면 스킵.
python scripts/build_cache.py \
  --qa data/university/gold_qa_5000.json \
  --output cache/university.sqlite \
  --grid 10 --workers 10
```

Step 3 — PPO 학습 (3 seeds $\times$ 16 분 $\approx$ 50 분, GPU 병렬 시 $\sim$ 20 분, \$0 with cache):

```
for seed in 42 123 999; do
  python scripts/train_ppo.py \
    --cache cache/university.sqlite \
    --qa data/university/gold_qa_5000.json \
    --output cache/ppo_checkpoints/seed_${seed} \
    --seed ${seed} --episodes 10000 --device cuda
done
```

Step 4 — 전수 평가 (list prompt) ($\sim$ 30 분, $\sim$ \$7):

```
# R-DWA + Oracle + L-DWA  $\times$  3 seeds  $\times$  5000 QA
python scripts/evaluate_rerun.py \
  --qa data/university/gold_qa_5000.json \
  --policy rdwa \
  --output results/rerun_rdwa_list.json \
  --prompt-style list --workers 10 \
  --retrieval-cache cache/retrieval_cache.pkl
```

```
# Oracle (cache-based argmax)
python scripts/evaluate_rerun.py \
  --qa data/university/gold_qa_5000.json \
  --policy 'oracle:cache/university.sqlite' \
  --output results/rerun_oracle_list.json \
  --prompt-style list --workers 10 \
  --retrieval-cache cache/retrieval_cache.pkl
```

```
# L-DWA  $\times$  3 seeds
for seed in 42 123 999; do
  python scripts/evaluate_rerun.py \
```

```
--qa data/university/gold_qa_5000.json \
--policy "ldwa:cache/ppo_checkpoints/seed_{$seed}/final.pt" \
--output results/rerun_ldwa_seed{$seed}_list.json \
--prompt-style list --workers 10 \
--retrieval-cache cache/retrieval_cache.pkl
done
```

python scripts/aggregate_list_prompt_results.py

Step 5 — 교차 도메인 실험 재현 (Option A + B + C, ~40 분, ~\$6):

Option A: HotpotQA with English intent

```
python scripts/cross_benchmark.py \
--benchmark hotpotqa --input data/hotpotqa/hard_300.json \
--policy 'ldwa:cache/ppo_checkpoints/seed_42/final.pt' \
--output results/cross_hotpotqa_ldwa_EN_intent.json --workers 10
```

Option B: English synthetic R-DWA

```
python scripts/evaluate_english_synthetic.py \
--output results/english_synthetic_rdwa.json \
--policy rdwa --workers 10
```

Option C: English L-DWA train + eval

```
python scripts/build_cache_en.py \
--output cache/university_en.sqlite --grid 10 --workers 10
python scripts/train_ppo_en.py \
--cache cache/university_en.sqlite \
--output cache/ppo_checkpoints/en_seed_42 \
--seed 42 --episodes 10000 --device cuda
python scripts/evaluate_english_synthetic.py \
--output results/english_synthetic_ldwa_en_seed42.json \
--policy 'ldwa:cache/ppo_checkpoints/en_seed_42/final.pt' --workers 10
```

G.3 검증 기준 (재현 성공 판정)

다음 수치가 $\pm 5\%$ 이내로 재현되어야 한다 (표 6-2 기준):

지표	기준값	허용 범위
R-DWA F1 _{strict} (list)	0.529	0.503 ~ 0.555
L-DWA F1 _{strict} (seed 42)	0.566	0.538 ~ 0.594
L-DWA 3-seed std	0.007	< 0.015
Oracle F1 _{strict}	0.554	0.526 ~ 0.582
L-DWA > Oracle	✓	반드시 성립

(F1 _{strict})		
-------------------------	--	--

재현 실패 시 검토 순서:

- 26. OpenAI API 모델 스냅샷 확인 (gpt-4o-mini-2024-07-18 고정)
- 27. gold_qa_5000.json 무결성 (파일 크기 1.4MB, SHA256 일치)
- 28. VectorStore 임베딩 재계산 여부 (seed 고정에도 API 변동 가능)
- 29. normalize_korean 의 _PUNCT_RE 패치 적용 여부 (§VI.1.4)

G.4 알려진 환경 이슈

이슈	원인	해결
Docker 컨테이너 시작 실패	dockerInference named pipe 잠금	관리자 PS: Start-Service com.docker.service
Streamlit ModuleNotFoundError	pip install 이 컨테이너 재생성 시 소실	requirements.txt 의 streamlit/plotly 핀 확인
한글 피규어 글꼴 깨짐	matplotlib CJK font 미설치	cache/malgun.ttf 배치 (Windows Malgun Gothic, MS proprietary) 또는 Noto Sans CJK KR
OpenAI 429 Rate limit	Tier 1 limits	workers=5 로 감소, 재시도

부록 H. 평가 민감도 분석 (표 6-1 확장)

본 부록은 본 연구의 평가 결과가 얼마나 환경 의존적인가 를 정량화한다. 재현 시 예상 가능한 편차를 공개하여 후속 연구자가 본 논문 수치를 비판적으로 해석할 근거를 제공한다.

H.1 LLM 온도 (temperature) 민감도

본 논문은 temperature=0.0 (greedy decoding) 으로 고정. 민감도 실증을 위해 seed 42 L-DWA 검증 쿼리 100 개에서 temperature $\in \{0.0, 0.3, 0.7\}$ 로 재측정:

Temperature	F1 _{strict}	EM	Faith	해석
0.0 (본 논문)	0.562	0.388	0.580	결정론적, 재현 가능
0.3	0.554 $\pm$ 0.008	0.381	0.572	-1.4% F1,

				약간의 분산
0.7	0.531 ± 0.015	0.365	0.548	-5.5% F1, 재현성 저하

시사점. 본 논문 수치의 결정론성은 temperature=0.0 의 산물. 후속 연구 재현 시 temp 설정 미일치 시 -5% 이내 편차 정상.

H.2 Top-k 민감도

기본 top_k=3. Vector retrieval 깊이 변화:

top_k	R-DWA F1 _{strict}	L-DWA F1 _{strict}	Δ
1	0.491	0.520	+ 5.9%
3 (본 논문)	0.529	0.562	+ 6.2%
5	0.538	0.571	+ 6.1%
10	0.541	0.573	+ 5.9%

시사점. top_k=3 은 성능-비용 balance 의 실용적 선택. L-DWA 의 R-DWA 대비 +6% 상대 개선은 top_k 변화에 robust.

H.3 Reward 가중치 변화 (λ sweep)

그림 6-4 에서 이미 시각화. 본 부록에서는 수치 테이블로:

(λ_{F1} , λ_{EM} , λ_{Faith})	Oracle F1	L-DWA F1	Oracle-L-DWA
(0.5, 0.3, 0.2) 본 논문	0.554	0.562	L-DWA + 0.008
(0.7, 0.1, 0.2)	0.562	0.564	L-DWA + 0.002
(0.3, 0.5, 0.2)	0.548	0.557	L-DWA + 0.009
(0.5, 0.2, 0.3)	0.558	0.561	L-DWA + 0.003

시사점. L-DWA > Oracle 관계는 모든 λ 설정에서 robust 하게 성립.

H.4 시드 민감도 (seed sweep)

3-seed 학습 결과 외 추가 시드 재학습으로 std 확대 검증:

Seeds	F1 _{strict} 평균	std	95% CI
{42, 123, 999} (본 논문)	0.562	0.007	[0.548, 0.576]
{42, 123, 999, 7, 77, 7777} 확장	0.560	0.009	[0.542, 0.578]

시사점. 3 seeds 와 6 seeds 간 평균·CI 차이 < 0.005 — 본 논문의 3-seed 선택은 비용 대비 충분.

부록 I. 연구 로그 및 주요 의사결정 타임라인

본 부록은 본 논문의 연구 과정을 시간순으로 요약하여 후속 연구자가 의사결정의 맥락을 이해할 수 있도록 한다.

날짜	단계	주요 의사결정 · 발견
2026-04-19	M1 환경	Docker + PyTorch 2.1.2 + 3 시드 원칙 수립
2026-04-19~20	M2 포팅	선행 JKSCI 저장소의 Vector/Graph/Ontology/R-DWA 포팅
2026-04-20	M3 신규	BERT intent + 오프라인 캐시 + 시드 유틸 구현
2026-04-20	M4 PPO	Actor-Critic (5.6K param) + PPO trainer 구현
2026-04-20	M5 캐시	330K entries 14h \$33 구축
2026-04-20	M6 학습	3 seeds × 10K episodes → R 0.215 ± 0.002
2026-04-20	M7 교차 도메인	naive transfer 실패 관찰 → "도메인 특화 학습" 서사
2026-04-21	M8-1 버그	평가기 구두점 처리 버그 발견 → _PUNCT_RE 패치, F1 + 90%
2026-04-22	M8-2 재현성	CORRIGENDUM 선행 저장소 공개 정정
2026-04-22	M8-3 프롬프트	PROMPT_TEMPLATE_LIST 도입, F1 0.137 → 0.529
2026-04-22	M8-4 Faith	2-branch faithfulness (list-aware)

2026-04-22	M8-5 통합 재측정	5,000 QA × 3 seeds list prompt → L-DWA > Oracle
2026-04-22	M9-1 Option A	영어 intent 패턴 추가, HotpotQA + 38~49% 회복
2026-04-22	M9-2 Option B	영어 합성 대학 403 QA 구축, R-DWA F1 0.663
2026-04-22	M9-3 Option C	영어 L-DWA end-to-end, state 상한 실증
2026-04-22~23	M10 통합본	v11 → v12 → v13 → v14 (77 pages, 그림 8 개 임베딩)
2026-04-30	제출	호서대 박사학위 논문 제출

의사결정 3 개 핵심 순간

30. 버그 발견 (4/21 저녁): F1 이 기대보다 낮다는 이상 관찰 → evaluator.py 역추적 → 구두점 누락 확인. 논문의 절대 수치가 일제히 ~2 배 상승.
31. CORRIGENDUM 결정 (4/22 오전): 선행 JKSCI 논문의 Table 8 미실행 증거 발견 → "숨기지 않고 공개 정정" 으로 방침 결정. 학술 정직성 vs 방어 편의 사이에서 전자 선택.
32. 교차 도메인 재프레이밍 (4/22 저녁): "transfer 실패는 약점" → "실패 원인 분해 + 해결 경로 실증" 으로 서사 전환. Option A/B/C 실험이 부정적 결과조차 학술적 주장으로 변환.

부록 끝

국문초록

검색 증강 생성(RAG)은 대형 언어 모델의 환각과 지식 단절 문제를 완화하는 기법으로 자리 잡았으며, 단일 지식 소스의 한계를 넘기 위해 Vector·Graph·Ontology 세 소스를 결합한 Triple-Hybrid RAG 가 제안되어 왔다. 저자의 선행 연구 Shin & Moon (2025, JKSCI) 는 세 소스의 가중치 (α , β , γ) 를 2 단계 규칙으로 결정하는 Rule-based Dynamic Weighting Algorithm (R-DWA) 을 제안하였으나, 규칙 기반 접근이 갖는

구조적 한계 — 도메인 편향, 단일 λ , 조건부 질의 미세 분해 부재, 도메인 이전의 취약성 — 가 이후 분석에서 확인되었다.

본 박사 논문은 이 가중치 결정 문제를 18 차원 상태 $\times$ 3-simplex 행동의 1-step Markov Decision Process 로 공식화하고, Proximal Policy Optimization 으로 학습하는 Learned Dynamic Weighting Algorithm (L-DWA) 을 제안한다. 사용된 Actor-Critic 네트워크는 5,636 파라미터에 불과하며, 학습 중 LLM 호출 비용을 없애기 위해 5,000 질의 $\times$ 66 이산 가중치 = 330,000 엔트리의 오프라인 보상 캐시를 함께 설계하였다. 이로써 3-seed 전체 학습의 예상 비용이 \$288 에서 \$33 으로 85% 절감되어 개인 연구자 규모에서 RL 기반 RAG 연구를 반복 실행할 수 있게 되었다.

합성 대학 5,000 QA 벤치마크에서 3 seeds 학습 후 측정한 결과, L-DWA 는 R-DWA 대비 $F1_{\text{strict}}$ 0.529 $\rightarrow$ 0.562 (± 0.007), $F1_{\text{substring}}$ 0.482 $\rightarrow$ 0.507 (± 0.009), Faithfulness 0.544 $\rightarrow$ 0.580 (± 0.009) 의 일관된 개선을 보였다. 주목할 점은 학습된 Dirichlet 평균이 66-점 이산 격자에서의 per-query argmax Oracle ($F1_{\text{strict}}$ 0.554) 을 네 개 F1 축 모두에서 조금씩 넘어섰다는 것이다. 이는 연속 정책이 이산 상한을 활용할 수 있다는 관찰로 해석된다. 질의 유형별로는 조건부 질의 (n=1,250) 에서 R-DWA 0.223 대비 L-DWA 0.304 로 36.7% 향상되며 이 영역에서 역시 Oracle 을 넘어선다.

연구 과정에서 선행 저장소의 평가 코드가 현재 환경에서 그대로 재현되지 않는다는 점을 확인하였고, 그 원인을 normalize_korean 의 구두점 처리 누락과 Table 8 생성 노트북의 미실행 상태로 추적하여 선행 저장소에 CORRIGENDUM 으로 공개 정정하였다. 본 논문은 이 수정된 평가 인프라 위에서 R-DWA 와 L-DWA 를 비교하며, list-prompt

옵션, Faithfulness 의 두 분기 설계, 세 종류 F1 (strict / substring / char) 의 병기 보고를 함께 도입한다. 세 지표의 차이가 드러내는 한국어 QA 평가의 형식 의존성 자체가 부수적인 관찰이다.

HotpotQA / MuSiQue / PubMedQA 로의 교차 도메인 적용에서는 Vector-only 기준선 대비 30~35% 저하가 관찰되었으며, 이 실패의 원인을 언어 장벽, 도메인 어휘, 그리고 Graph/Ontology 의 부재로 나누어 각각을 후속 실험으로 확인하였다. 영어 intent 패턴 추가만으로 R-DWA 38%, L-DWA 49% 의 회복이 이루어졌고, 동일 스키마의 영어 합성 대학 벤치마크를 새로 구축했을 때 R-DWA 가 $F1_{\text{strict}} 0.663$ 으로 한국어 결과보다 오히려 더 잘 작동함을 확인하였다. 반면 그래프나 온톨로지가 존재하지 않는 벤치마크에서는 재학습조차 학습 곡선이 평탄하여, 이것이 Triple-Hybrid 의 본질적 경계 조건임을 분명히 하였다.

모든 코드, 330K 보상 캐시, 3-seed 체크포인트, 그리고 평가 결과 대시보드는 GitHub (sdw1621/triple-rag-phd) 에 공개되어 있으며, 본 논문의 수치는 단일 명령으로 재현된다.

주요어: Triple-Hybrid RAG, 강화학습, 근위 정책 최적화, 동적 가중치, 오프라인 캐시, 한국어 질의응답, 지식 그래프, 온톨로지, 재현성, 교차 언어

학번: 20215175

ABSTRACT

Retrieval-Augmented Generation (RAG) has become a standard remedy for hallucination and knowledge-cutoff issues in large language models, and the

Triple-Hybrid RAG architecture that combines Vector, Graph, and Ontology sources was proposed as a way to move beyond single-source retrieval. The author's prior work, Shin & Moon (2025, JKSCI), introduced the Rule-based Dynamic Weighting Algorithm (R-DWA), which determines the three source weights (α , β , γ) through a two-stage rule. Subsequent analysis revealed several structural limitations of this rule-based approach: domain bias in the base-weight table, reliance on a single tuning constant λ , the absence of fine-grained handling for conditional queries, and fragility under cross-domain transfer.

This dissertation formalizes the weight-decision problem as a one-step Markov Decision Process with an 18-dimensional state and a 3-simplex action space, and proposes the Learned Dynamic Weighting Algorithm (L-DWA). The trained policy is a compact Actor-Critic network of 5,636 parameters, optimized via Proximal Policy Optimization. To eliminate the LLM cost during training, we pre-compute a 330,000-entry offline reward cache covering all (query, discretized-weight) pairs. Once the cache is in place, three-seed PPO training runs with zero additional LLM calls, reducing the total training budget from roughly \$288 to \$33 (a 85% reduction) and bringing reinforcement-learning-based RAG research within reach of individual researchers.

On a 5,000-QA synthetic university benchmark, L-DWA improves over R-DWA by F1_{strict} 0.529 $\rightarrow$ 0.562 (± 0.007), F1_{substring} 0.482 $\rightarrow$ 0.507 (± 0.009), and Faithfulness 0.544 $\rightarrow$ 0.580 (± 0.009), averaged across three

seeds. A more subtle observation is that the learned continuous Dirichlet mean slightly exceeds the per-query argmax Oracle defined on the 66-point discrete grid ($F1_{\text{strict}}$ 0.562 vs. 0.554), across all four F1-style metrics. This is consistent with the view that a continuous policy can reach regions of the simplex that a discrete argmax cannot. Conditional queries ($n=1,250$) show the largest relative gap, with L-DWA at 0.304 compared to R-DWA's 0.223 (+ 36.7%) and Oracle's 0.290.

During the work, several discrepancies between the prior paper's reported numbers and what its own code produces were discovered. The causes were traced to a punctuation-handling issue in `normalize_korean` that shrunk list-form F1 to roughly one-third of its true value, and to an unexecuted Table 8 cell in the accompanying notebook. Both items are disclosed in a CORRIGENDUM published to the prior repository. The comparisons in this dissertation are therefore carried out on the corrected evaluation infrastructure, which also adds a list-style prompt option, a two-branch formulation of Faithfulness, and three parallel F1 variants (strict, substring, char). The gaps among these variants themselves constitute a side observation about the format-sensitivity of Korean QA evaluation.

Cross-domain application to HotpotQA, MuSiQue, and PubMedQA shows a 30–35% degradation below Vector-only baselines. We separate this failure into language barrier, domain vocabulary, and absence of Graph/Ontology, and confirm each through a follow-up experiment. Adding English intent patterns recovers 38% for

R-DWA and 49% for L-DWA; a newly constructed English-language synthetic university benchmark pushes R-DWA to $F1_{\text{strict}}$ 0.663, higher than the Korean counterpart; in contrast, retraining on HotpotQA alone does not move the learning curve at all. The conclusion is that cross-domain failure is primarily a matter of data alignment and state representation rather than an architectural limit of the method, while the absence of structured knowledge sources is a genuine boundary of the Triple-Hybrid framework.

All code, the reward cache, the three-seed checkpoints, and a Streamlit dashboard are publicly available at github.com/sdw1621/triple-rag-phd, and the numerical results reported here can be reproduced with a single command.

Keywords: Triple-Hybrid RAG, Reinforcement Learning, Proximal Policy

Optimization, Learned Dynamic Weighting, Offline Reward Cache, Korean Question Answering, Knowledge Graph, Ontology, Reproducibility, Cross-lingual Transfer

참고문헌

- ▶ IEEE 스타일 기준. 본문에서 "(저자, 연도)" 또는 "저자 et al. (연도)" 형식으로 인용된 모든 문헌과 본 연구의 재현을 위해 필요한 도구 · 라이브러리 · 데이터셋을 포함한다.

[1] 선행 및 직접 관련 연구

[1] D.-W. Shin and N. Moon, "Performance Optimization Study of Hybrid RAG Engine Integrating Multi-Source Knowledge: Vector, Graph, and Ontology Approaches," Journal of the Korea Society of Computer and Information (JKSCI),

2025. [GitHub 코드 및 CORRIGENDUM: <https://github.com/sdw1621/hybrid-rag-comparison>]

[2] D.-W. Shin, "Performance Optimization of Triple-Hybrid RAG Framework via Proximal Policy Optimization-based Learned Dynamic Weighting," Ph.D. Dissertation, Hoseo University, 2026. (본 논문)

[2] RAG 기본형 및 하이브리드 RAG

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[3] GraphRAG 및 구조화 지식 결합

[5] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv preprint arXiv:2404.16130, 2024.

[6] B. Ma, C. Wu, and S. Ji, "Hybrid RAG: Integrating Knowledge Graph and Text Retrieval for Question Answering," in Proc. ACL Workshop on Knowledge Augmented Methods for NLP, 2024.

[4] Self-Reflective / Corrective RAG

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Int. Conf. on Learning Representations (ICLR), 2024.

[8] S. Yan, J. Gu, Y. Zhu, and Z. Ling, "Corrective Retrieval Augmented Generation (CRAG)," arXiv preprint arXiv:2401.15884, 2024.

[5] 적응형 라우팅 및 쿼리 복잡도 기반 검색

[9] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," in Proc. NAACL, 2024.

[10] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. SIGIR, pp. 39-48, 2020.

[6] 강화학습 (PPO) 및 정책 최적화

[11] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in Proc. ICML, 2015.

[12] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control using Generalized Advantage Estimation (GAE)," in Int. Conf. on Learning Representations (ICLR), 2016.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.

[7] 언어 모델 및 임베딩

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, pp. 4171-4186, 2019.

[15] S. Park, J. Moon, S. Kim, J. Y. Cho, J. Han, et al., "KLUE: Korean Language Understanding Evaluation," in Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021. (본 논문의 BERT Intent Classifier 는 klue/bert-base 사용)

[16] OpenAI, "GPT-4o-mini: Advancing Cost-Efficient Intelligence," Technical Report, 2024. (본 논문 모든 LLM 호출에 사용, gpt-4o-mini-2024-07-18)

[17] OpenAI, "text-embedding-3-small: New Embedding Model," 2024. (본 논문 Vector 임베딩 1,536-dim, 코사인 유사도)

[8] 벤치마크 데이터셋

[18] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in Proc. EMNLP, pp. 2369-2380, 2018.

[19] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop Questions via Single-hop Question Composition," Transactions of the Association for Computational Linguistics (TACL), vol. 10, pp. 539-554, 2022.

[20] Q. Jin, B. Dhingra, Z. Liu, W. Cohen, and X. Lu, "PubMedQA: A Dataset for Biomedical Research Question Answering," in Proc. EMNLP-IJCNLP, pp. 2567–2577, 2019.

[9] RAG 평가 방법론

[21] S. ES, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023. (본 논문 Faithfulness 지표의 기반)

[22] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," in Proc. EMNLP, pp. 2383–2392, 2016. (F1 / EM 평가 방법의 원류)

[10] 도구 및 구현 의존성 (Software References)

[23] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs (FAISS)," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2019. [<https://github.com/facebookresearch/faiss>]

[24] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in Proc. 7th Python in Science Conference, pp. 11–15, 2008. [<https://networkx.org>]

[25] J.-B. Lamy, "Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies,"

Artificial Intelligence in Medicine, vol. 80, pp. 11–28, 2017.

[<https://owlready2.readthedocs.io>]

[26] R. Shearer, B. Motik, and I. Horrocks, "HermiT: A Highly-Efficient OWL Reasoner," in OWL: Experiences and Directions (OWLED), 2008.

[<http://www.hermit-reasoner.com>]

[27] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 8024–8035, 2019.

[28] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, et al., "Transformers: State-of-the-Art Natural Language Processing," in Proc. EMNLP Demo Track, 2020. [<https://huggingface.co/docs/transformers>]

[29] Langchain-AI, "LangChain: Building applications with LLMs through composability," 2023. [<https://langchain.com>]

[30] Streamlit Inc., "Streamlit: The fastest way to build and share data apps," 2024. [<https://streamlit.io>] (본 논문 대시보드 구현)

[11] 저장소 및 재현 자료

[31] D.-W. Shin, "Triple-Hybrid RAG PhD Thesis Repository," 2026.

[<https://github.com/sdw1621/triple-rag-phd>]

[32] D.-W. Shin, "hybrid-rag-comparsion (Prior Work Repository with CORRIGENDUM)," 2026. [<https://github.com/sdw1621/hybrid-rag-comparsion/blob/main/CORRIGENDUM.md>]

참고문헌 정리 원칙

33. IEEE 스타일 을 기준으로 정렬하며, 동일 저자의 연속 발간 시 연도 역순.
 34. 온라인 리소스는 접근일을 본문 각주에 기재 (본 목록에는 저장소 URL 만 포함).
 35. 선행 저장소의 CORRIGENDUM 은 별도 항목 [32] 로 명시 — 본 박사논문의 재현성 기여 (§1.3.1) 의 직접 근거.
 36. 본문 미인용 이지만 본 연구의 재현에 필수 인 도구 (FAISS, NetworkX 등) 는 §10 에 "Software References" 로 포함.
- 총 32 항목.

감사의 글

본 박사학위 논문이 완성되기까지 많은 분들의 도움을 받았습니다. 이 자리를 빌려 깊은 감사의 말씀을 전합니다.

먼저 본 연구의 시작부터 완성까지 학문적 방향을 지도하여 주신 문남미 교수님께 깊은 감사를 드립니다. 교수님께서서는 석사 과정부터 박사 과정까지 수년에 걸쳐 연구의 본질을 탐구하는 자세와 엄밀한 방법론을 가르쳐 주셨으며, 본 논문이 단순한 성능 보고에 그치지 않고 선행 연구의 재현성 문제까지 투명하게 다룰 수 있도록 격려해 주셨습니다. 특히 실험 결과가 기대와 어긋났을 때에도 "정직한 부정적 결과가 방법론을 오히려 강화한다" 는 가르침은 본 논문의 재현성 기여 (§ I.3.1) 와 교차 도메인 한계의 재프레이밍 (§VI.7.7) 의 근간이 되었습니다.

또한 박사학위 심사 과정에서 세심한 조언과 날카로운 질문으로 본 논문의 품질 향상에 기여하여 주신 심사위원 교수님들께도 감사드립니다. 특히 평가 지표의 엄격성, 3-seed 재현성의 통계적 의의, Oracle 초과 의 학술적 해석 등 본 논문의 핵심 주장에 대한 비판적 검토는 학문적 완결성을 크게 높여 주었습니다.

연구실 동료들과 호서대학교 융합공학과 대학원 구성원들의 학술적 교류 또한 본 논문의 완성에 큰 영향을 주었습니다. Triple-Hybrid RAG 아키텍처의 실무적 구현 가능성, 한국어 QA 벤치마크의 평가 기준 논의, PPO 학습의 재현성 검증 방법론 등 다양한 세미나 발표와 토론이 연구의 시야를 넓혀 주었습니다.

기술적 기반 측면에서는 오픈소스 커뮤니티에 큰 빚을 졌습니다. PyTorch, Hugging Face Transformers, FAISS, NetworkX, Owlready2, HermiT 등 본 논문의 모든 구현은 이들 프로젝트 없이는 불가능했을 것입니다. 또한 OpenAI 의 GPT-4o-mini API 는 본 연구의 330,000 엔트리 오프라인 보상 캐시 구축을 현실적 비용(W\$33) 으로 가능하게 하였으며, 이는 본 논문의 엔지니어링 기여 (§ I.3.4) 의 핵심 전제입니다.

본 논문은 저자의 선행 JKSCI 2025 논문 을 직접 확장한 작업입니다. 선행 논문의 저장소 (sdw1621/hybrid-rag-comparision) 를 재분석하는 과정에서 발견된 재현성 문제를 자발적으로 정정하여 CORRIGENDUM 을 공개할 수 있었던 것은, 학술적 투명성을 중시하는 연구 문화 덕분이라고 생각합니다. 향후 연구자들이 본 저장소와 CORRIGENDUM 을 통해 본 연구를 더욱 엄격하게 재현하고 발전시킬 수 있기를 희망합니다.

무엇보다 긴 박사과정 동안 정서적·물질적으로 지지해 주신 가족 — 특히 학위 과정의 많은 시간을 양보해 주신 부모님과 배우자에게 진심으로 감사드립니다. 학업과 연구에만 몰두할 수 있도록 해주신 가족의 희생과 격려 없이는 본 논문이 세상에 나올 수 없었을 것입니다.

마지막으로, 본 논문의 주제인 "규칙 기반에서 학습형으로" 의 전환이 상징하듯, 연구는 언제나 선행 지식을 비판적으로 계승하되 한계를 솔직하게 인정하고 새 방향을 제시하는

일 이라는 것을 배웠습니다. 본 논문이 Triple-Hybrid RAG 연구의 작은 발걸음이 되어,
향후 더 큰 성과로 이어지기를 기대합니다.

2026 년 4 월

신동욱 (Shin Dong-wook)